

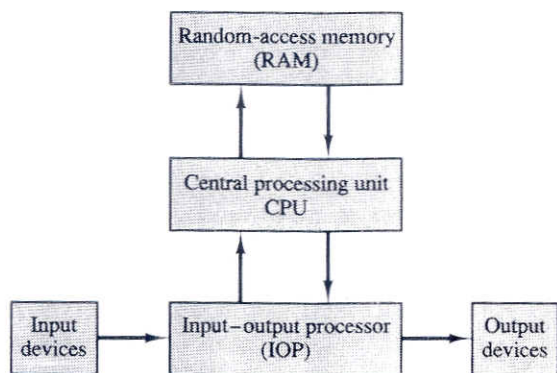
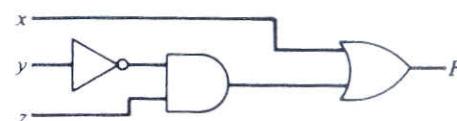


Figure 1-1 Block diagram of a digital computer.

Figure 1-3 Truth table and logic diagram for $F = x + y'z$.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(a) Truth table



(b) Logic diagram

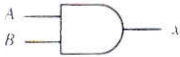
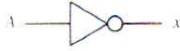
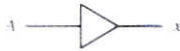
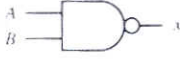
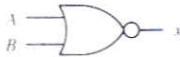
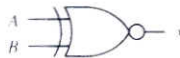
Name	Graphic symbol	Algebraic function	Truth table															
AND		$x = A \cdot B$ or $x = AB$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	0	1	0	0	1	1	1
A	B	x																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$x = A + B$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	1	1	0	1	1	1	1
A	B	x																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$x = A'$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	x	0	1	1	0									
A	x																	
0	1																	
1	0																	
Buffer		$x = A$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	x	0	0	1	1									
A	x																	
0	0																	
1	1																	
NAND		$x = (AB)'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	1	0	1	1	1	0	1	1	1	0
A	B	x																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$x = (A + B)'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	1	0	1	0	1	0	0	1	1	0
A	B	x																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$x = A \oplus B$ or $x = A'B + AB'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	0	0	1	1	1	0	1	1	1	0
A	B	x																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$x = (A \oplus B)'$ or $x = A'B' + AB$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	1	0	1	0	1	0	0	1	1	1
A	B	x																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Figure 1-2 Digital logic gates.

TABLE 1-1 Basic Identities of Boolean Algebra

(1) $x + 0 = x$	(2) $x \cdot 0 = 0$
(3) $x + 1 = 1$	(4) $x \cdot 1 = x$
(5) $x + x = x$	(6) $x \cdot x = x$
(7) $x + x' = 1$	(8) $x \cdot x' = 0$
(9) $x + y = y + x$	(10) $xy = yx$
(11) $x + (y + z) = (x + y) + z$	(12) $x(yz) = (xy)z$
(13) $x(y + z) = xy + xz$	(14) $x + yx = (x + y)(x + z)$
(15) $(x + y)' = x'y'$	(16) $(xy)' = x' + y'$
(17) $(x')' = x$	

Figure 1-4 Two graphic symbols for NOR gate.

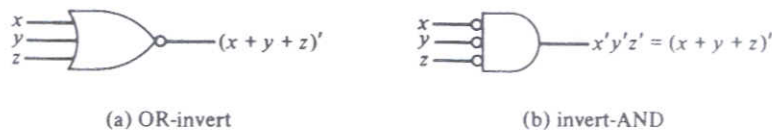


Figure 1-5 Two graphic symbols for NAND gate.

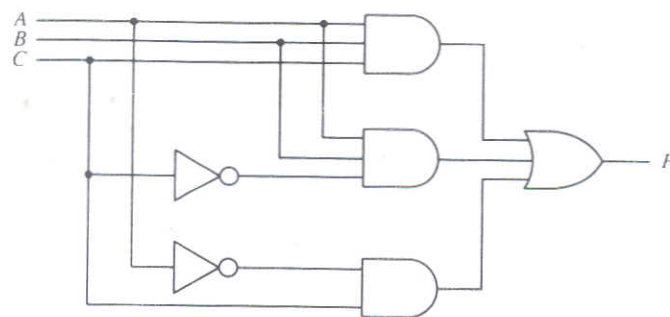
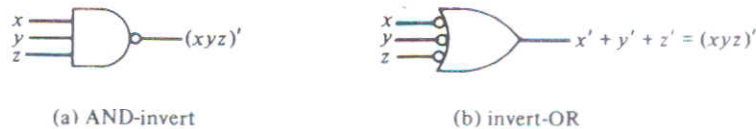
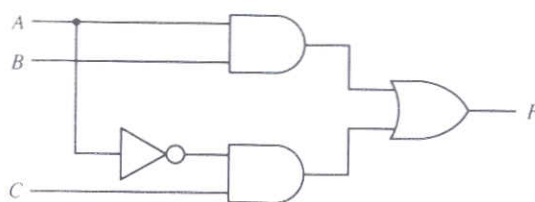
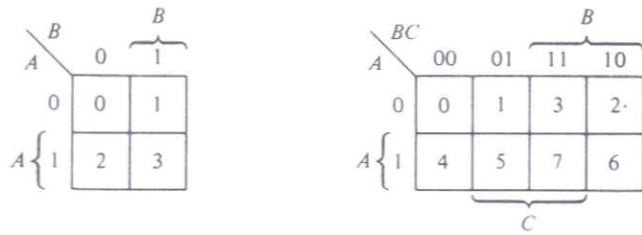
(a) $F = ABC + ABC' + A'C$ (B) $F = AB + A'C$

Figure 1-6 Two logic diagrams for the same Boolean function.



(a) Two-variable map

(b) Three-variable map

(c) Four-variable map

Figure 1-7 Maps for two-, three-, and four-variable functions.

Figure 1-8 Map for $F(A, B, C) = \Sigma(3, 4, 6, 7)$.

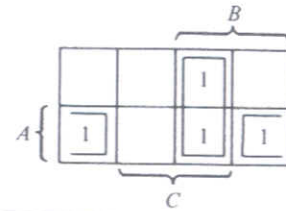


Figure 1-9 Map for $F(A, B, C) = \Sigma(0, 2, 4, 5, 6)$.

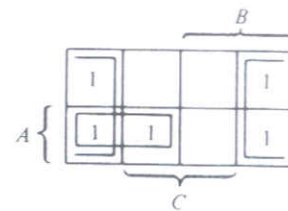


Figure 1-10 Map for $F(A, B, C, D) = \Sigma(0, 1, 2, 6, 8, 9, 10)$.

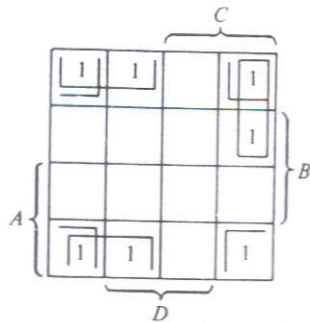


Figure 1-11 Map for $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$.

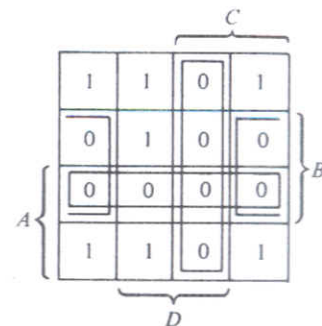
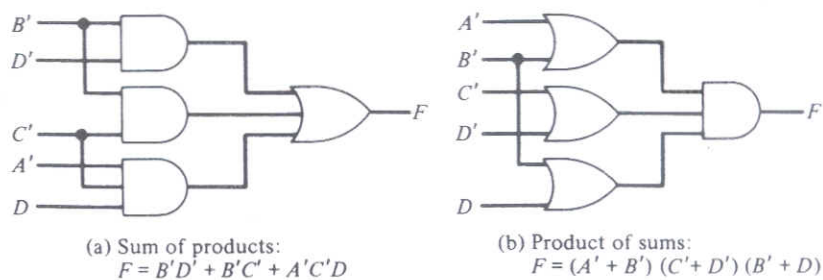


Figure 1-12 Logic diagrams with AND and OR gates.



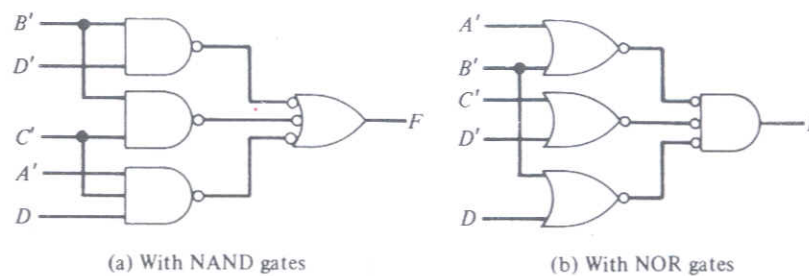


Figure 1-13 Logic diagrams with NAND or NOR gates.

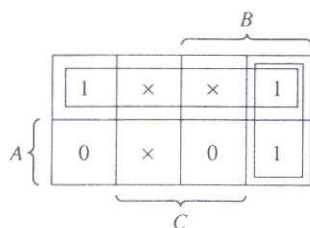


Figure 1-14 Example of map with don't-care conditions.

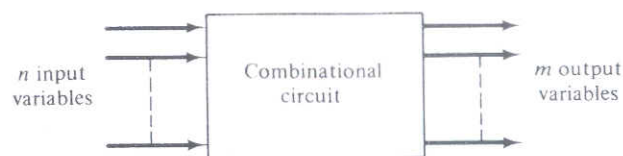
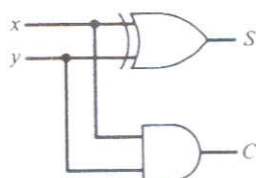


Figure 1-15 Block diagram of a combinational circuit.

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

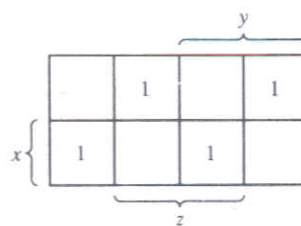
(a) Truth table



(b) Logic diagram

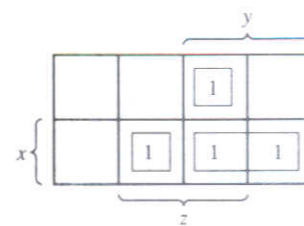
Figure 1-16 Half-adder.

Figure 1-17 Maps for full-adder.



$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$= x \oplus y \oplus z$$



$$C = xy + xz + yz$$

$$= xy + (x'y + xy')z$$

TABLE 1-2 Truth Table for Full-Adder

Inputs			Outputs	
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

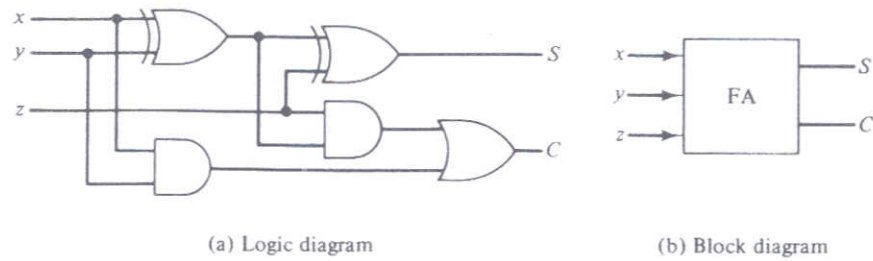


Figure 1-18 Full-adder circuit.

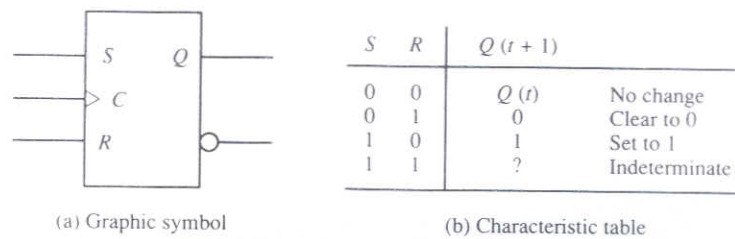


Figure 1-19 SR flip-flop.

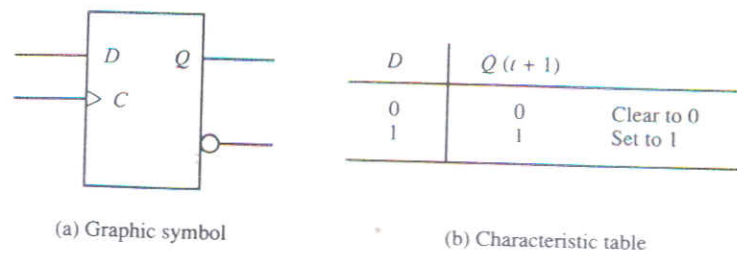


Figure 1-20 D flip-flop.

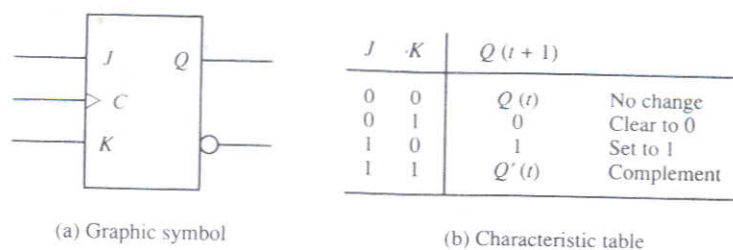


Figure 1-21 JK flip-flop.

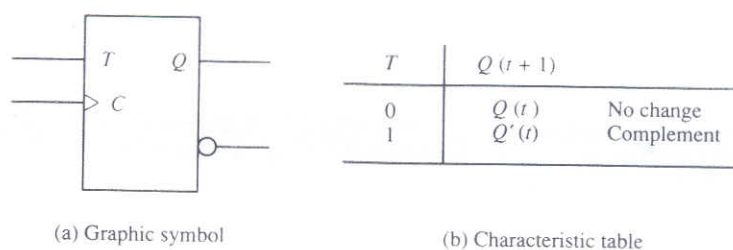


Figure 1-22 T flip-flop.

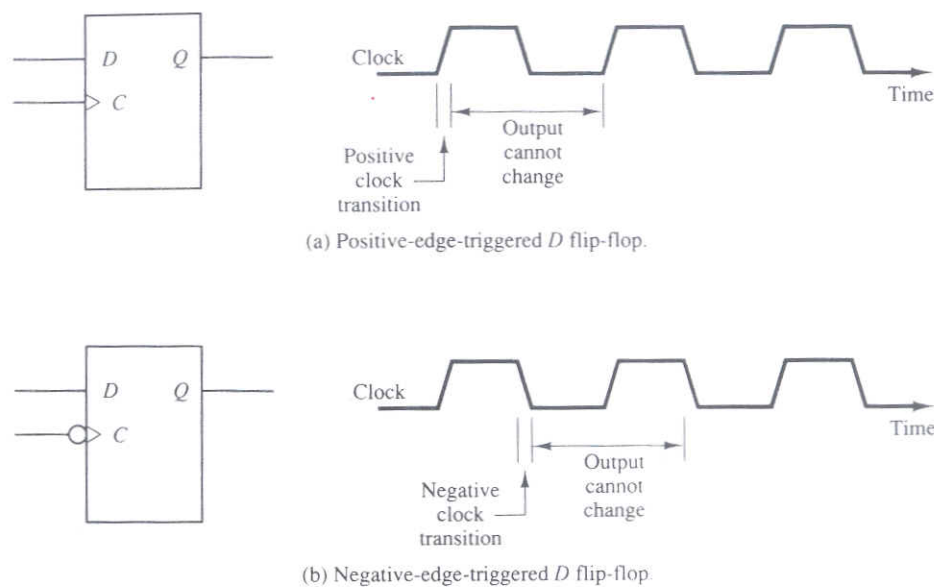


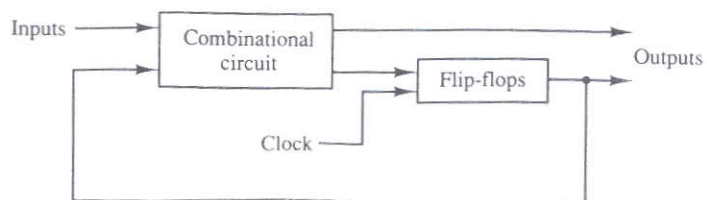
Figure 1-23 Edge-triggered flip-flop.

TABLE 1-3 Excitation Table for Four Flip-Flops

SR flip-flop				D flip-flop		
$Q(t)$	$Q(t+1)$	S	R	$Q(t)$	$Q(t+1)$	D
0	0	0	$\times$	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	0
1	1	$\times$	0	1	1	1

JK flip-flop				T flip-flop		
$Q(t)$	$Q(t+1)$	J	K	$Q(t)$	$Q(t+1)$	T
0	0	0	$\times$	0	0	0
0	1	1	$\times$	0	1	1
1	0	$\times$	1	1	0	1
1	1	$\times$	0	1	1	0

Figure 1-24 Block diagram of a clocked synchronous sequential circuit.



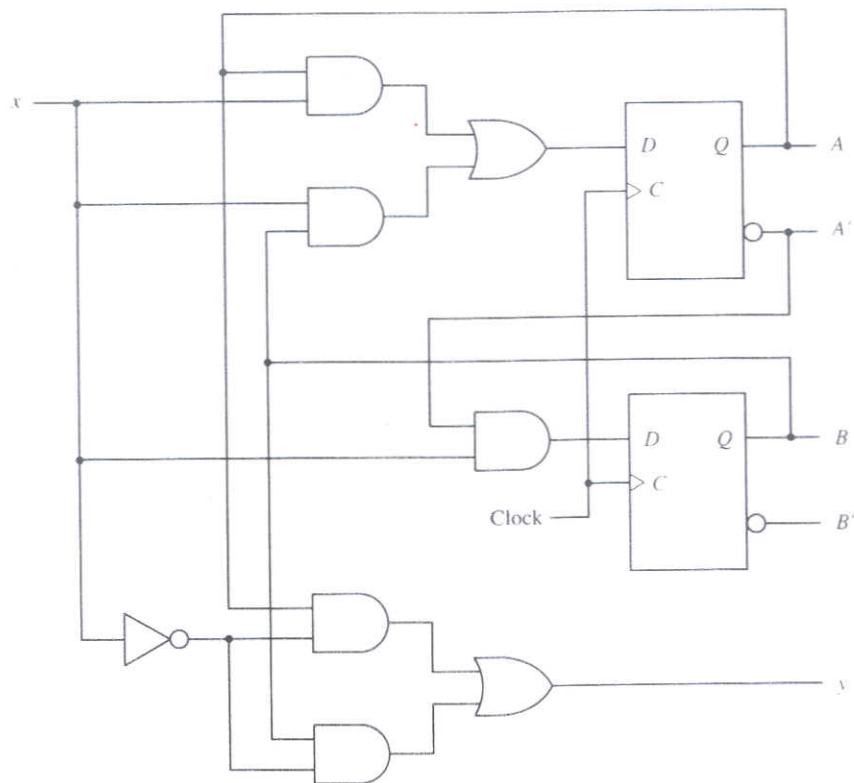


Figure 1-25 Example of a sequential circuit.

TABLE 1-4 State Table for Circuit of Fig. 1-25

Present state		Input	Next state		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

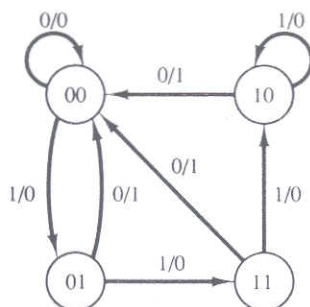


Figure 1-26 State diagrams of sequential circuit.

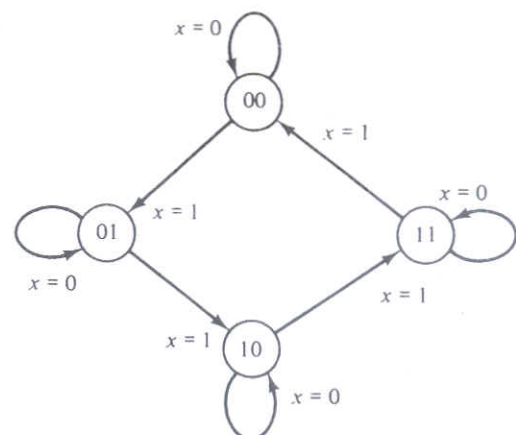


Figure 1-27 State diagram for binary counter.

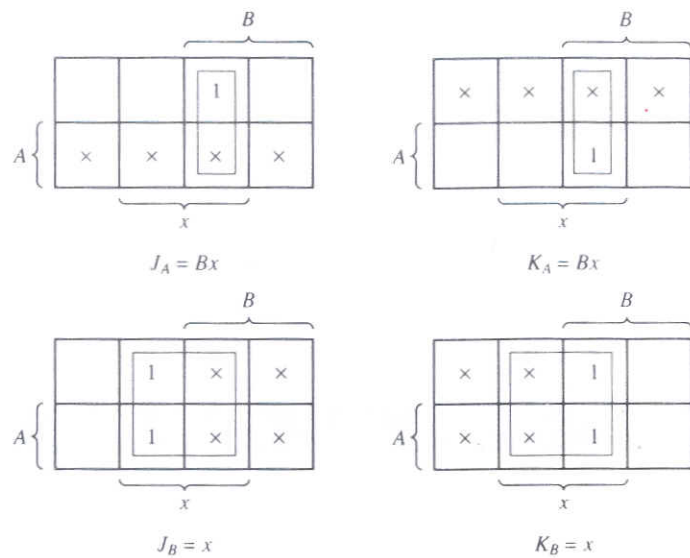


Figure 1-28 Maps for combinational circuit of counter.

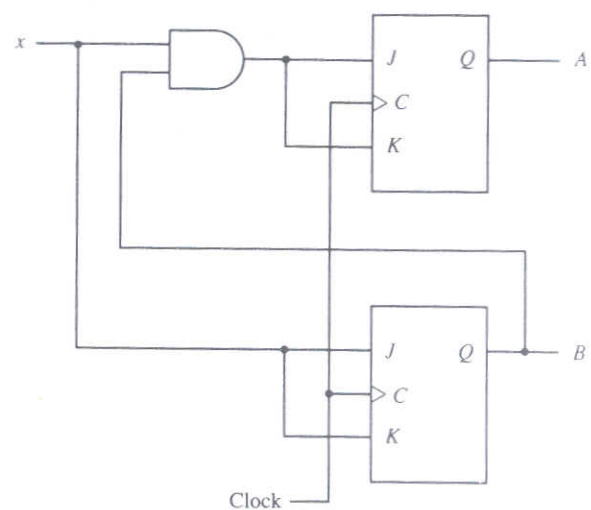


Figure 1-29 Logic diagram of a 2-bit binary counter.

TABLE 1-5 Excitation Table for Binary Counter

Present state		Input	Next state		Flip-flop inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	x	1	x
0	1	0	0	1	0	x	x	0
0	1	1	1	0	1	x	x	1
1	0	0	1	0	x	0	0	x
1	0	1	1	1	x	0	1	x
1	1	0	1	1	x	0	x	0
1	1	1	0	0	x	1	x	1

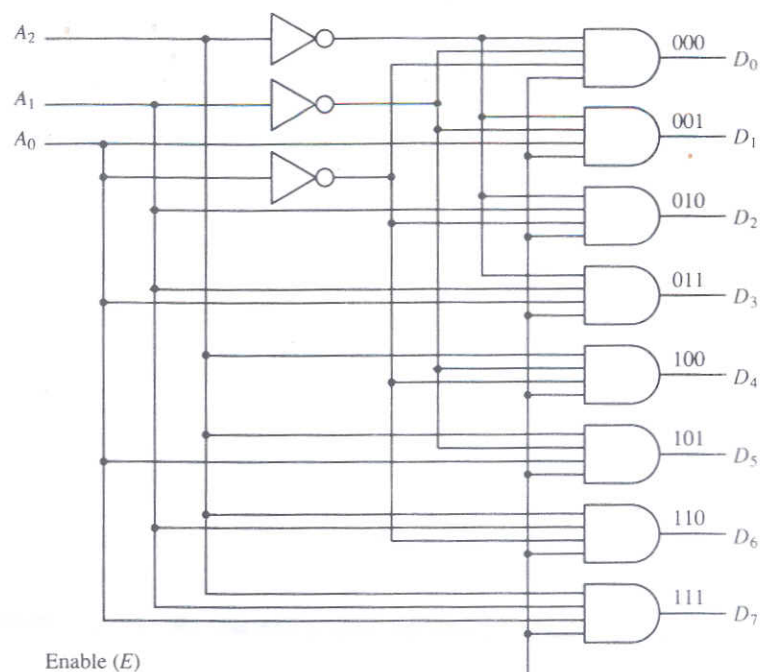


Figure 2-1 3-to-8-line decoder.

TABLE 2-1 Truth Table for 3-to-8-Line Decoder

Enable	Inputs			Outputs								
	E	A_2	A_1	A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	×	×	×	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	0	1	0	0	0	0	0
1	1	1	0	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0	0	0	0	0

Figure 2-2 2-to-4-line decoder with NAND gates.

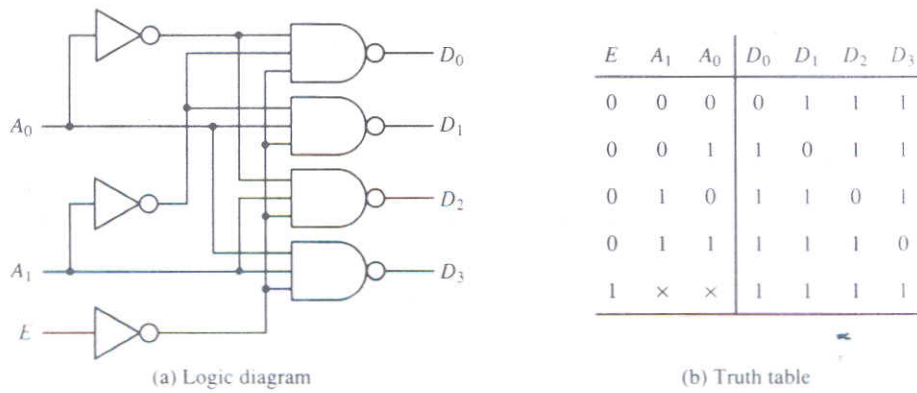
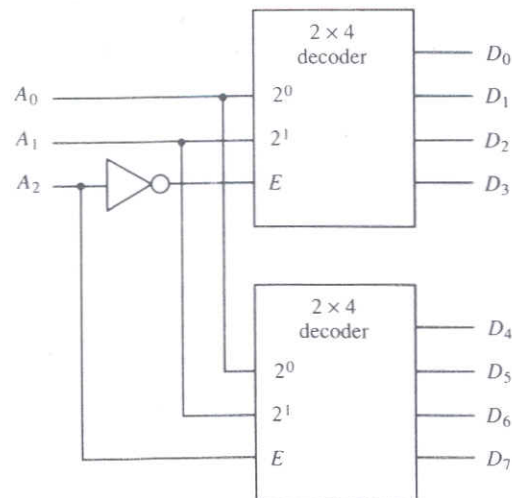
Figure 2-3 A 3×8 decoder constructed with two 2×4 decoders.

Figure 2-4 4-to-1-line multiplexer.

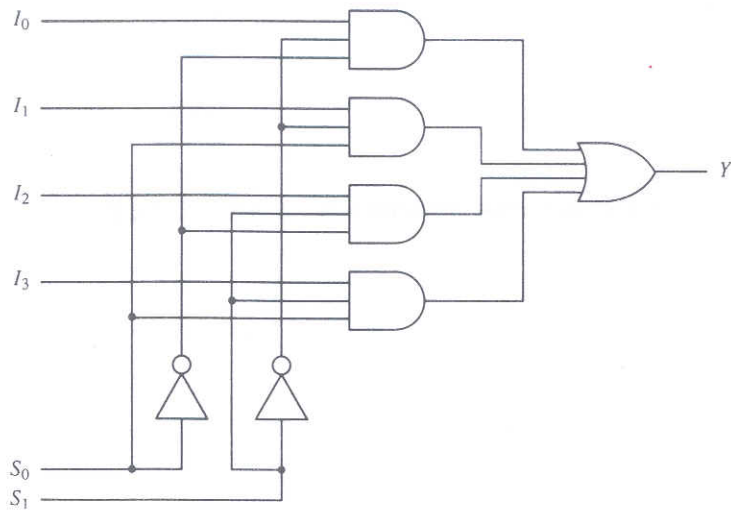
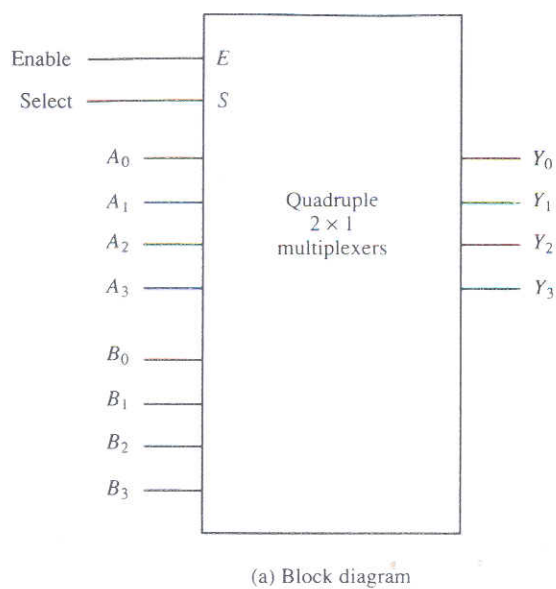


TABLE 2-3 Function Table for 4-to-1-Line Multiplexer

Select		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



(a) Block diagram

E	S	Y
0	$\times$	All 0's
1	0	A
1	1	B

(b) Function table

Figure 2-5 Quaduple 2-to-1 line multiplexers.

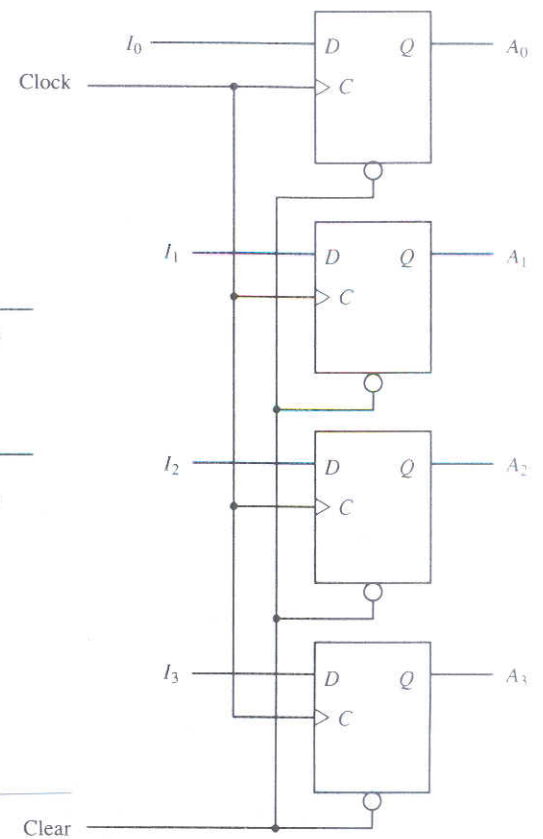


Figure 2-6 4-bit register.

Figure 2-8 4-bit shift register.

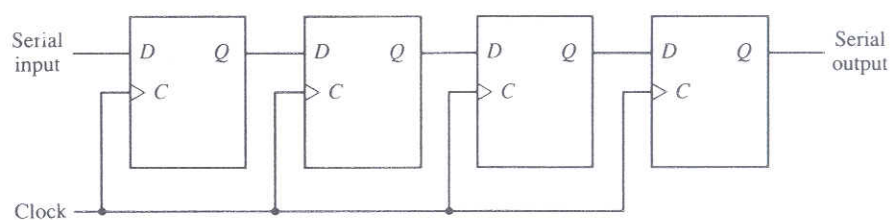


TABLE 2-2 Truth Table for Octal-to-Binary Encoder

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

TABLE 2-4 Function Table for Register of Fig. 2-9

Mode control		Register operation
S_1	S_0	
0	0	No change
0	1	Shift right (down)
1	0	Shift left (up)
1	1	Parallel load

TABLE 2-5 Function Table for the Register of Fig. 2-11

Clock	Clear	Load	Increment	Operation
↑	0	0	0	No change
↑	0	0	1	Increment count by 1
↑	0	1	×	Load inputs I_0 through I_3
↑	1	×	×	Clear outputs to 0

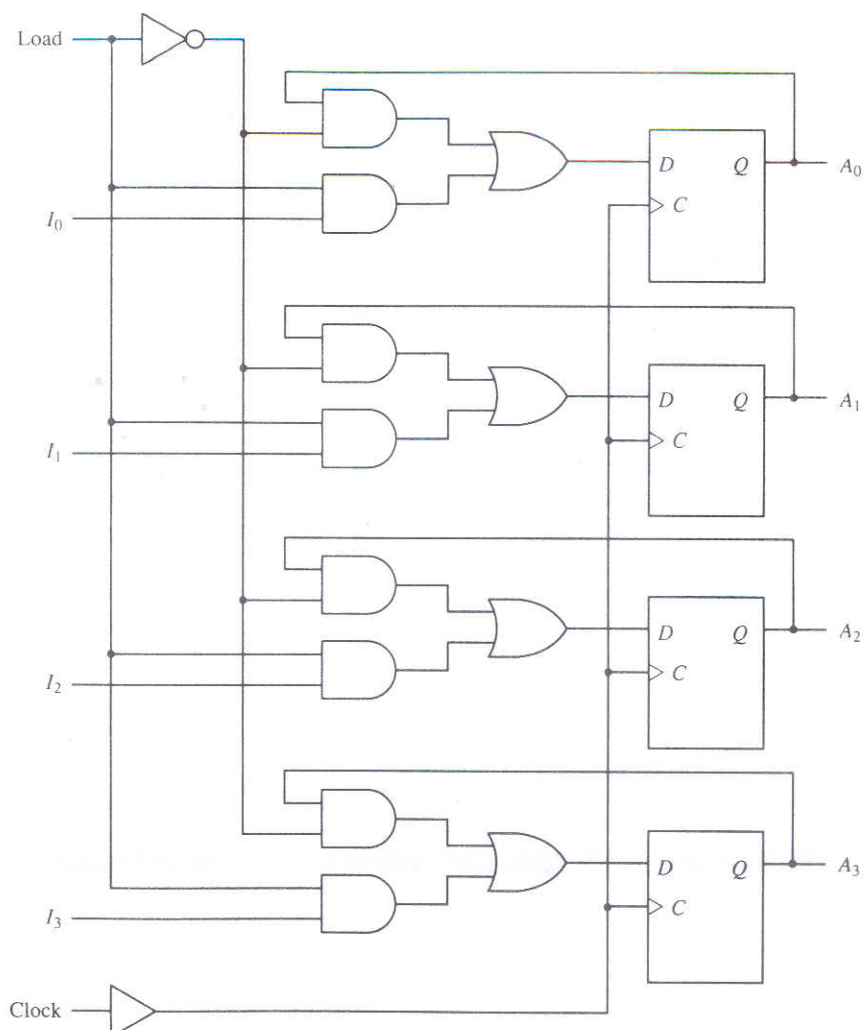


Figure 2-7 4-bit register with parallel load.

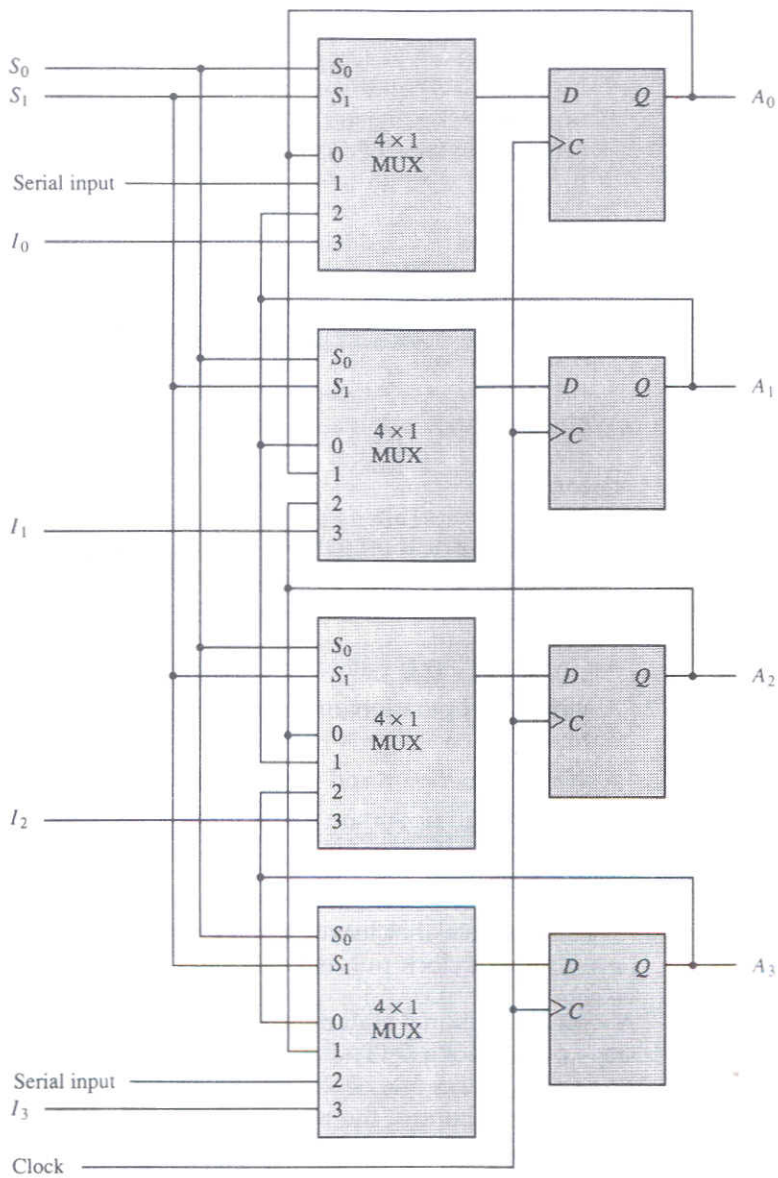


Figure 2-9 Bidirectional shift register with parallel load.

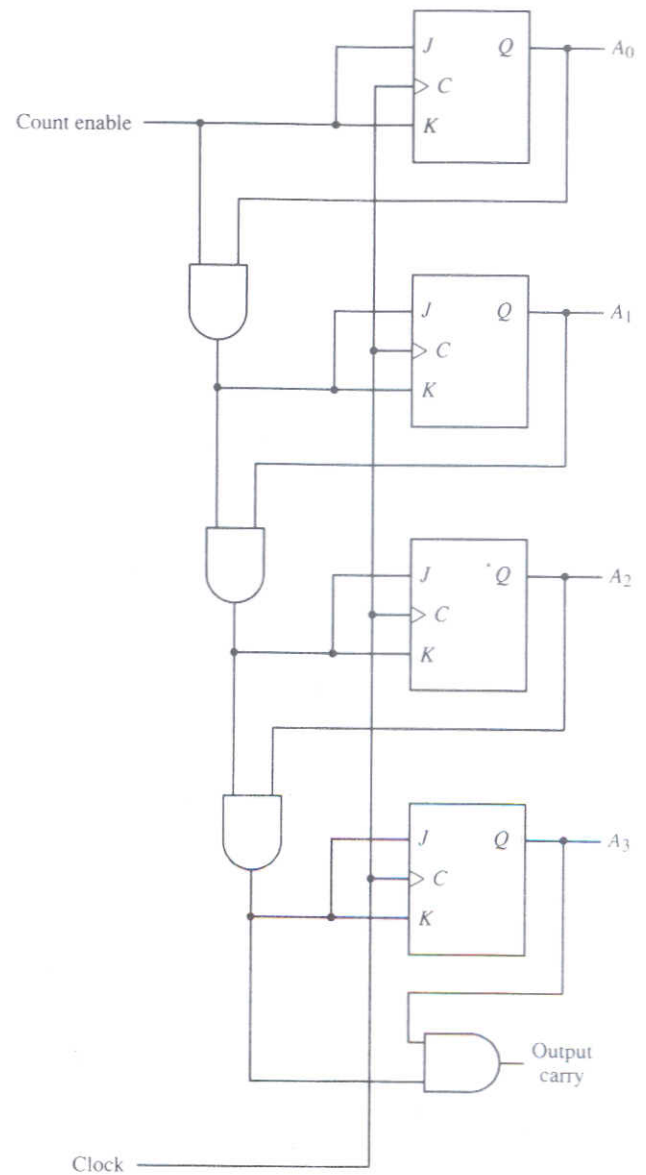


Figure 2-10 4-bit synchronous binary counter.

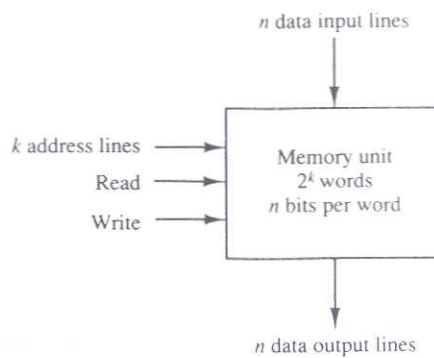


Figure 2-12 Block diagram of random access memory (RAM).

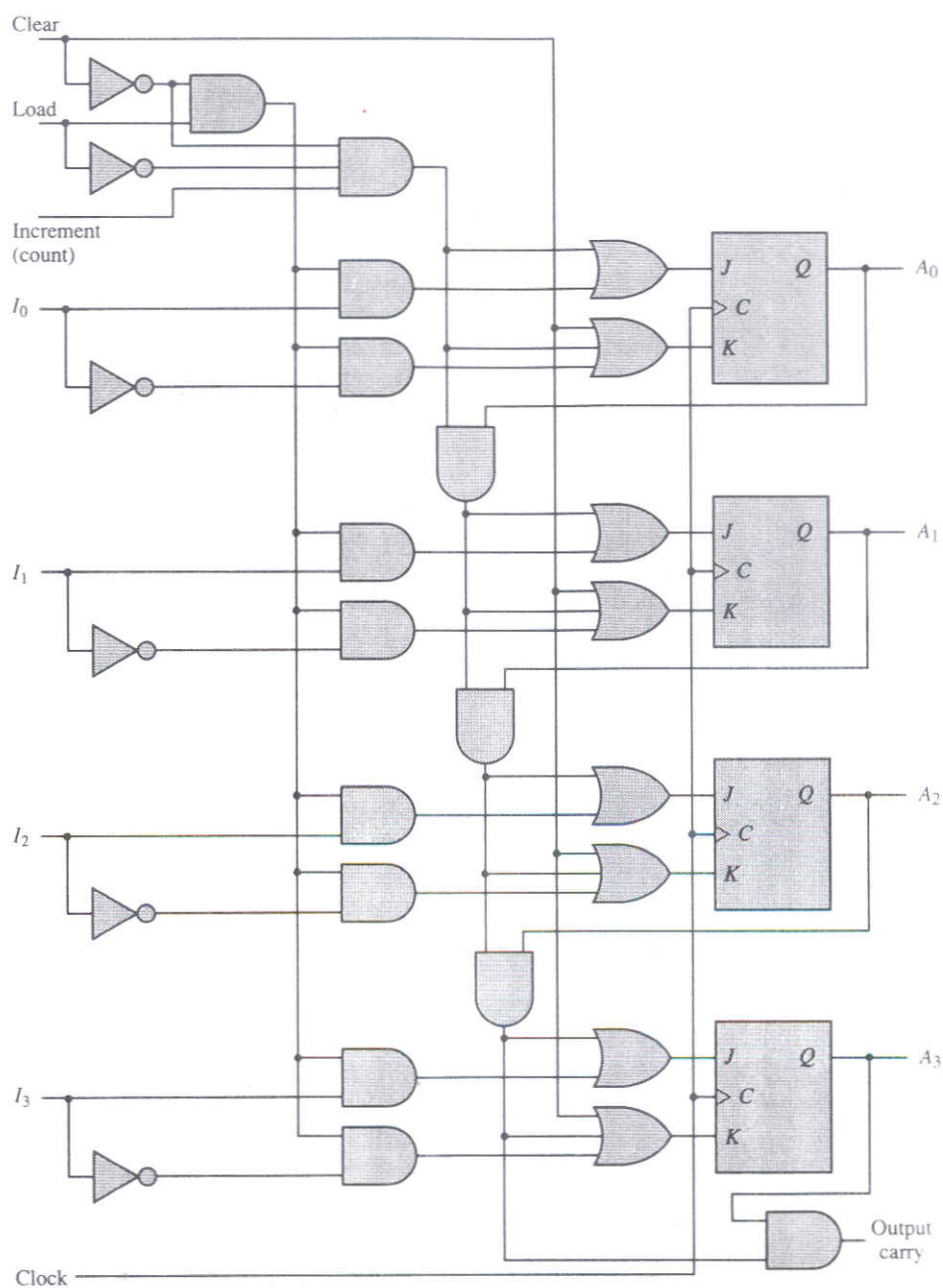


Figure 2-11 4-bit binary counter with parallel load and synchronous clear.

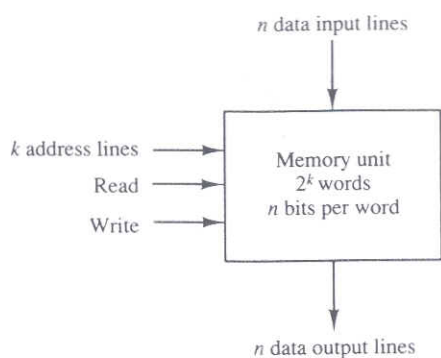


Figure 2-12 Block diagram of random access memory (RAM).

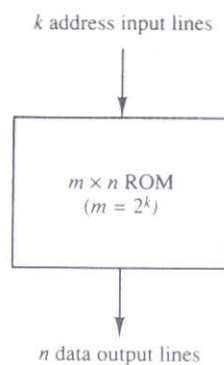


Figure 2-13 Block diagram of read only memory (ROM).

Figure 3-1 Conversion of decimal 41.6875 into binary.

Integer = 41		Fraction = 0.6875
41		0.6875
20	1	<u>2</u>
10	0	1.3750
5	0	<u>x 2</u>
2	1	0.7500
1	0	<u>x 2</u>
0	1	1.5000
		<u>x 2</u>
		1.0000
$(41)_{10} = (101001)_2$		$(0.6875)_{10} = (0.1011)_2$
$(41.6875)_{10} = (101001.1011)_2$		

TABLE 3-1 Binary-Coded Octal Numbers

Octal number	Binary-coded octal	Decimal equivalent	
0	000	0	↑ Code for one octal digit ↓
1	001	1	
2	010	2	
3	011	3	
4	100	4	
5	101	5	
6	110	6	
7	111	7	
10	001 000	8	
11	001 001	9	
12	001 010	10	
24	010 100	20	
62	110 010	50	
143	001 100 011	99	
370	011 111 000	248	

TABLE 3-2 Binary-Coded Hexadecimal Numbers

Hexadecimal number	Binary-coded hexadecimal	Decimal equivalent	
0	0000	0	↑ Code for one hexadecimal digit ↓
1	0001	1	
2	0010	2	
3	0011	3	
4	0100	4	
5	0101	5	
6	0110	6	
7	0111	7	
8	1000	8	
9	1001	9	
A	1010	10	
B	1011	11	
C	1100	12	
D	1101	13	
E	1110	14	
F	1111	15	
14	0001 0100	20	
32	0011 0010	50	
63	0110 0011	99	
F8	1111 1000	248	

TABLE 3-3 Binary-Coded Decimal (BCD) Numbers

Decimal number	Binary-coded decimal (BCD) number	
0	0000	Code for one decimal digit
1	0001	
2	0010	
3	0011	
4	0100	
5	0101	
6	0110	
7	0111	
8	1000	
9	1001	
10	0001 0000	
20	0010 0000	
50	0101 0000	
99	1001 1001	
248	0010 0100 1000	

TABLE 3-6 Four Different Binary Codes for the Decimal Digit

Decimal digit	BCD 8421	2421	Excess-3	Excess-3 gray
0	0000	0000	0011	0010
1	0001	0001	0100	0110
2	0010	0010	0101	0111
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1100
6	0110	1100	1001	1101
7	0111	1101	1010	1111
8	1000	1110	1011	1110
9	1001	1111	1100	1010
Unused bit combinations	1010	0101	0000	0000
	1011	0110	0001	0001
	1100	0111	0010	0011
	1101	1000	1101	1000
	1110	1001	1110	1001
	1111	1010	1111	1011

TABLE 3-4 American Standard Code for Information Interchange (ASCII)

Character	Binary code	Character	Binary code
A	100 0001	0	011 0000
B	100 0010	1	011 0001
C	100 0011	2	011 0010
D	100 0100	3	011 0011
E	100 0101	4	011 0100
F	100 0110	5	011 0101
G	100 0111	6	011 0110
H	100 1000	7	011 0111
I	100 1001	8	011 1000
J	100 1010	9	011 1001
K	100 1011		
L	100 1100		
M	100 1101	space	010 0000
N	100 1110	.	010 1110
O	100 1111	(	010 1000
P	101 0000	+	010 1011
Q	101 0001	\$	010 0100
R	101 0010	*	010 1010
S	101 0011	)	010 1001
T	101 0100	-	010 1101
U	101 0101	/	010 1111
V	101 0110	,	010 1100
W	101 0111	=	011 1101
X	101 1000		
Y	101 1001		
Z	101 1010		

TABLE 3-5 4-Bit Gray Code

Binary code	Decimal equivalent	Binary code	Decimal equivalent
0000	0	1100	8
0001	1	1101	9
0011	2	1111	10
0010	3	1110	11
0110	4	1010	12
0111	5	1011	13
0101	6	1001	14
0100	7	1000	15

TABLE 3-7 Parity Bit Generation

Message xyz	P(odd)	P(even)
000	1	0
001	0	1
010	0	1
011	1	0
100	0	1
101	1	0
110	1	0
111	0	1

1	2	7	5	4	3	Octal							
1	0	1	0	1	1	1	0	1	0	0	1	1	Binary
A			F			6			3			Hexadecimal	

Figure 3-2 Binary, octal, and hexadecimal conversion.

Figure 3-3 Error detection with odd parity bit.

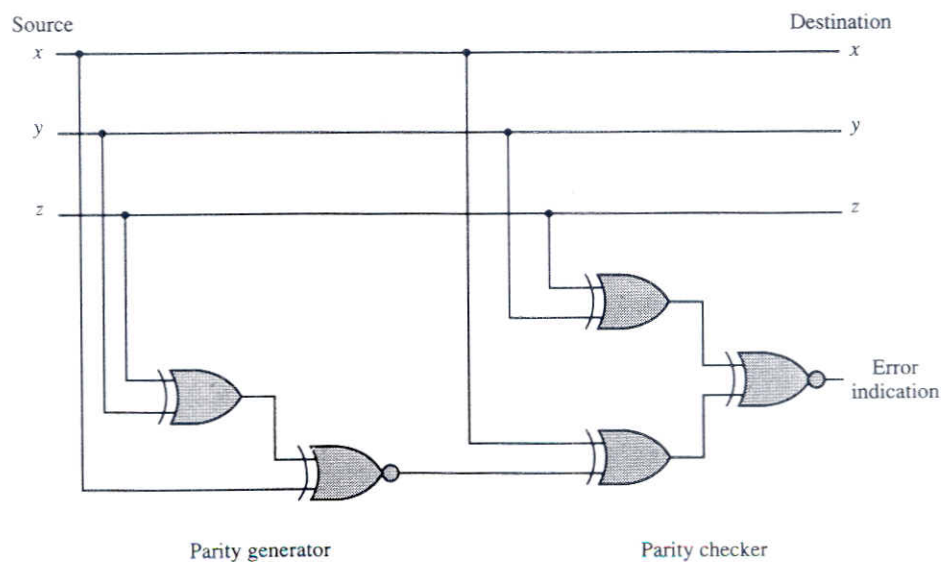


Figure 4-1 Block diagram of register.

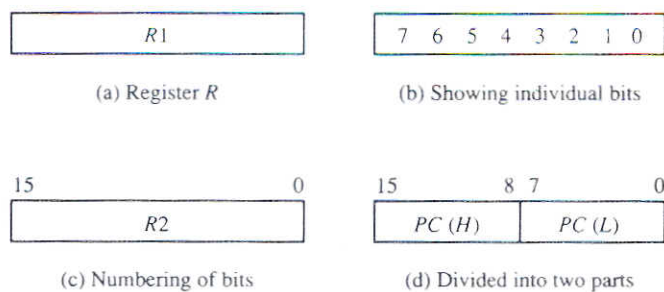
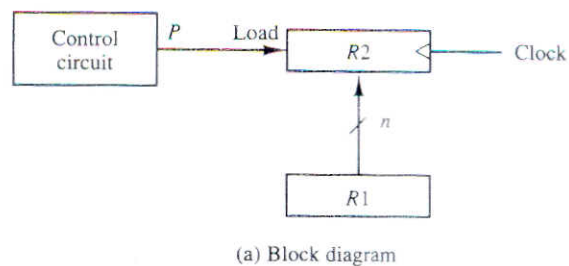
Figure 4-2 Transfer from R1 to R2 when $P = 1$.

TABLE 4-2 Function Table for Bus of Fig. 4-3

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

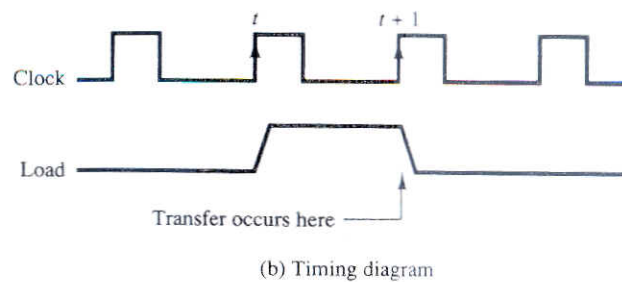


TABLE 4-1 Basic Symbols for Register Transfers

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	MAR, R2
Parentheses ()	Denotes a part of a register	R2(0-7), R2(L)
Arrow $\leftarrow$	Denotes transfer of information	R2 $\leftarrow$ R1
Comma ,	Separates two microoperations	R2 $\leftarrow$ R1, R1 $\leftarrow$ R2

Figure 4-3 Bus system for four registers.

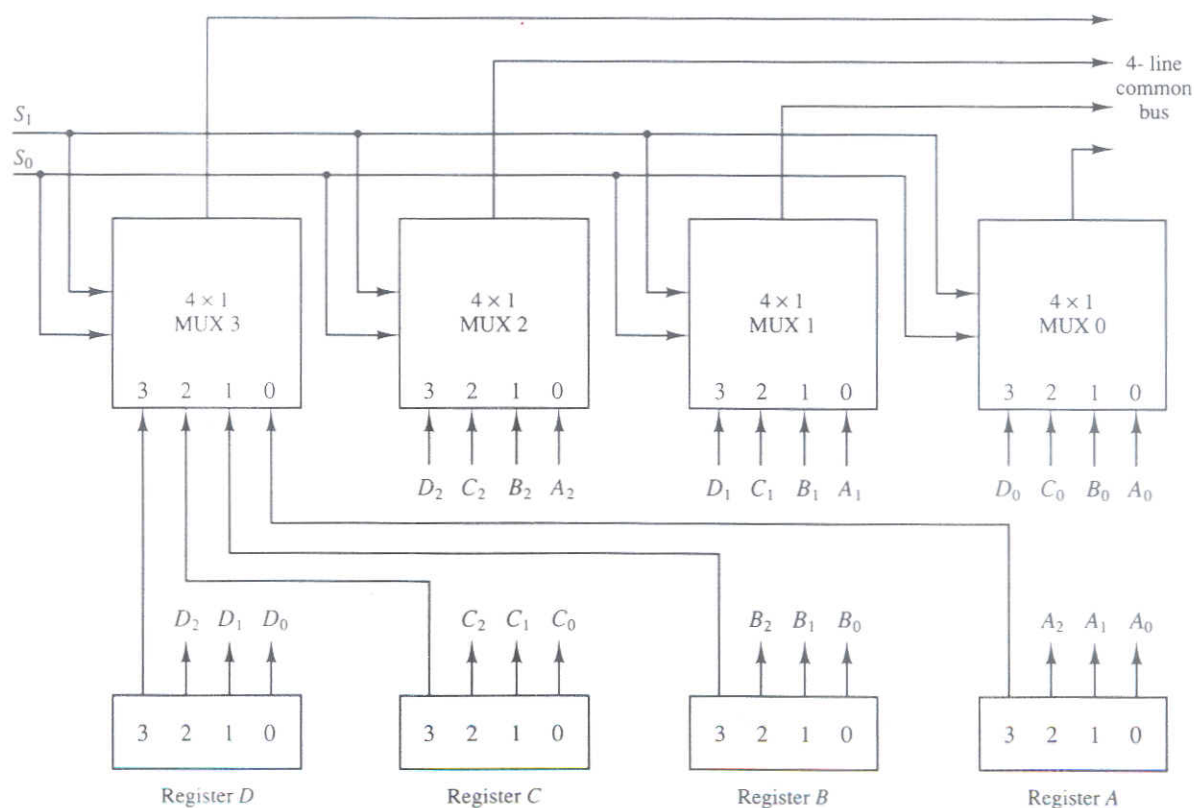


Figure 4-4 Graphic symbols for three-state buffer.

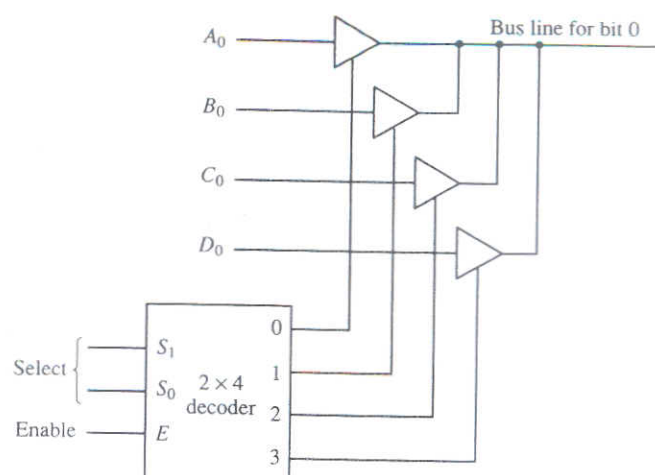


Figure 4-5 Bus line with three state-buffers.

TABLE 4-3 Arithmetic Microoperations

Symbolic designation	Description
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 - R2$	Contents of R1 minus R2 transferred to R3
$R2 \leftarrow \overline{R2}$	Complement the contents of R2 (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement the contents of R2 (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	R1 plus the 2's complement of R2 (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by one
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by one

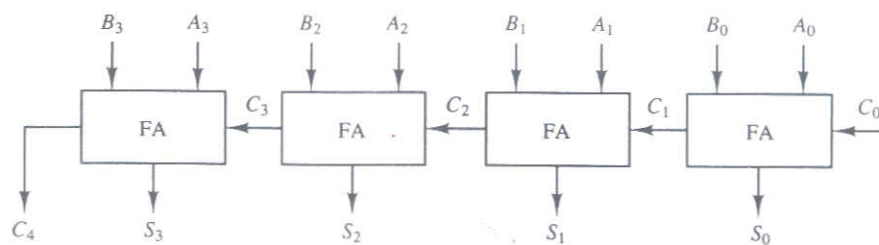


Figure 4-6 4-bit binary adder.

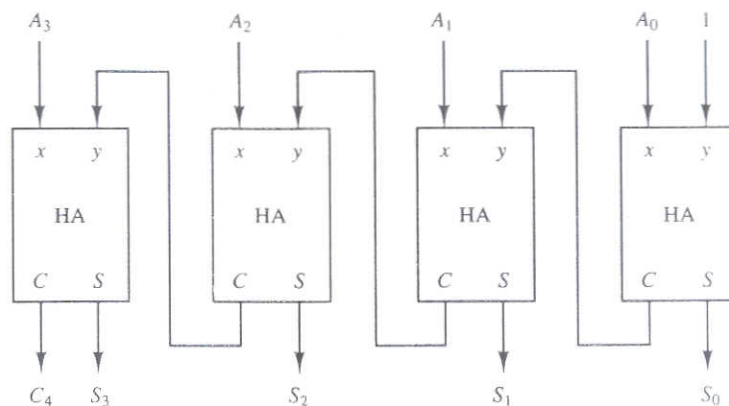


Figure 4-8 4-bit binary incrementer.

Figure 4-10 One stage of logic circuit.

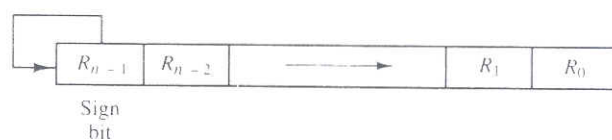
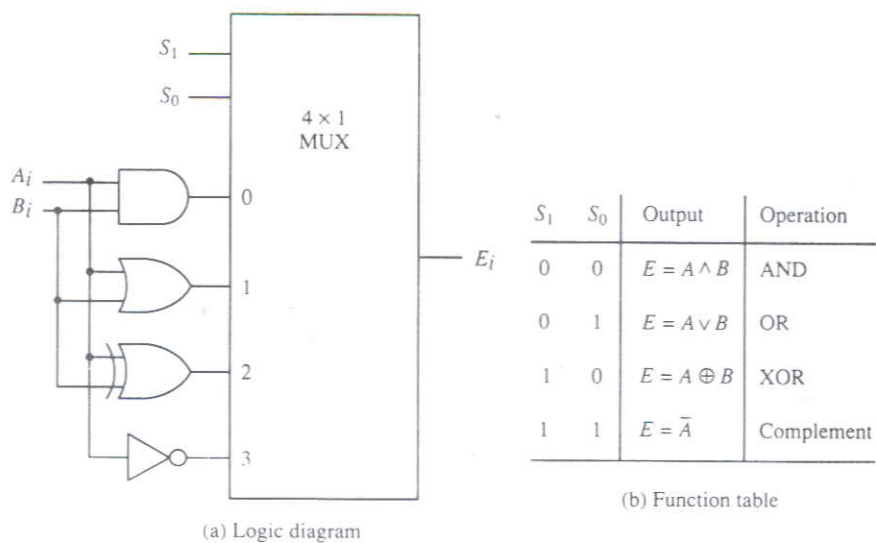


Figure 4-11 Arithmetic shift right.

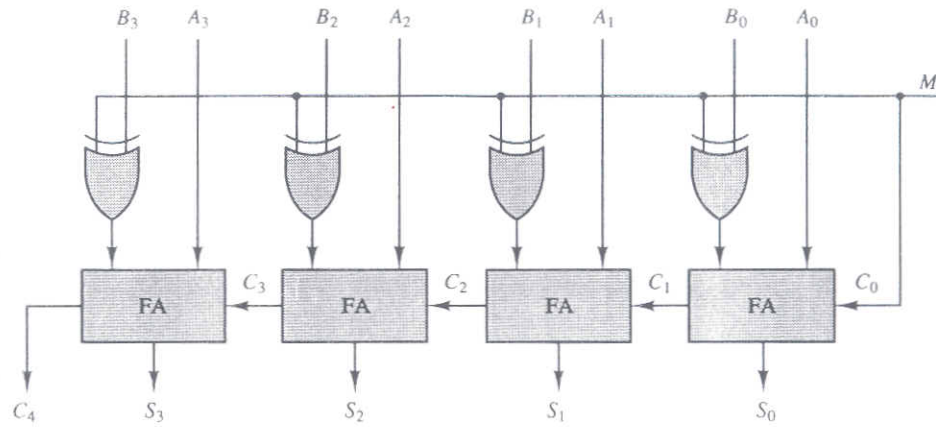


Figure 4-7 4-bit adder-subtractor.

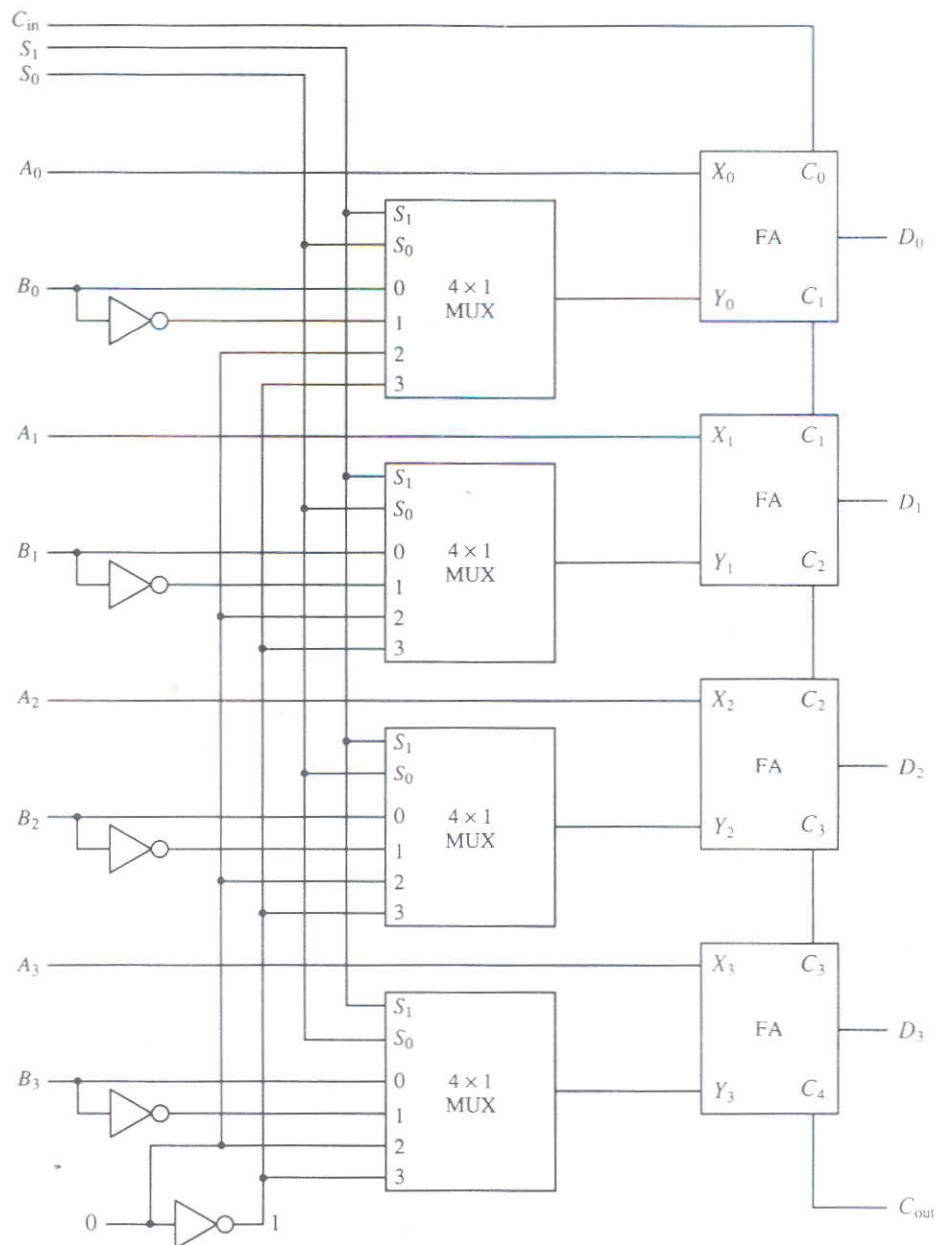


Figure 4-9 4-bit arithmetic circuit.

TABLE 4-4 Arithmetic Circuit Function Table

Select			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S_1	S_0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\overline{B}$	$D = A + \overline{B}$	Subtract with borrow
0	1	1	$\overline{B}$	$D = A + \overline{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

TABLE 4-5 Truth Tables for 16 Functions of Two Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

TABLE 4-6 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

TABLE 4-7 Shift Microoperations

Symbolic designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic shift-left R
$R \leftarrow \text{ashr } R$	Arithmetic shift-right R

TABLE 4-8 Function Table for Arithmetic Logic Shift Unit

Operation select					Operation	Function
S_3	S_2	S_1	S_0	C_{in}		
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	$\times$	$F = A \wedge B$	AND
0	1	0	1	$\times$	$F = A \vee B$	OR
0	1	1	0	$\times$	$F = A \oplus B$	XOR
0	1	1	1	$\times$	$F = \overline{A}$	Complement A
1	0	$\times$	$\times$	$\times$	$F = \text{shr } A$	Shift right A into F
1	1	$\times$	$\times$	$\times$	$F = \text{shl } A$	Shift left A into F

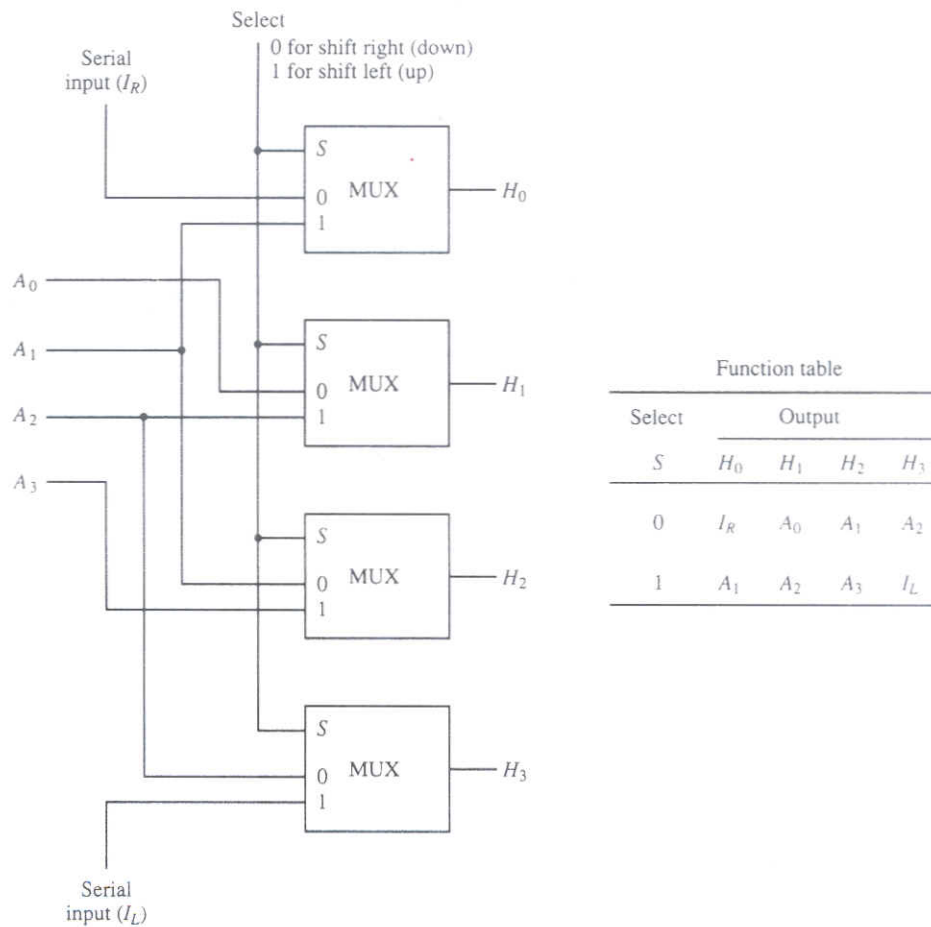


Figure 4-12 4-bit combinational circuit shifter.

Figure 4-13 One stage of arithmetic logic shift unit.

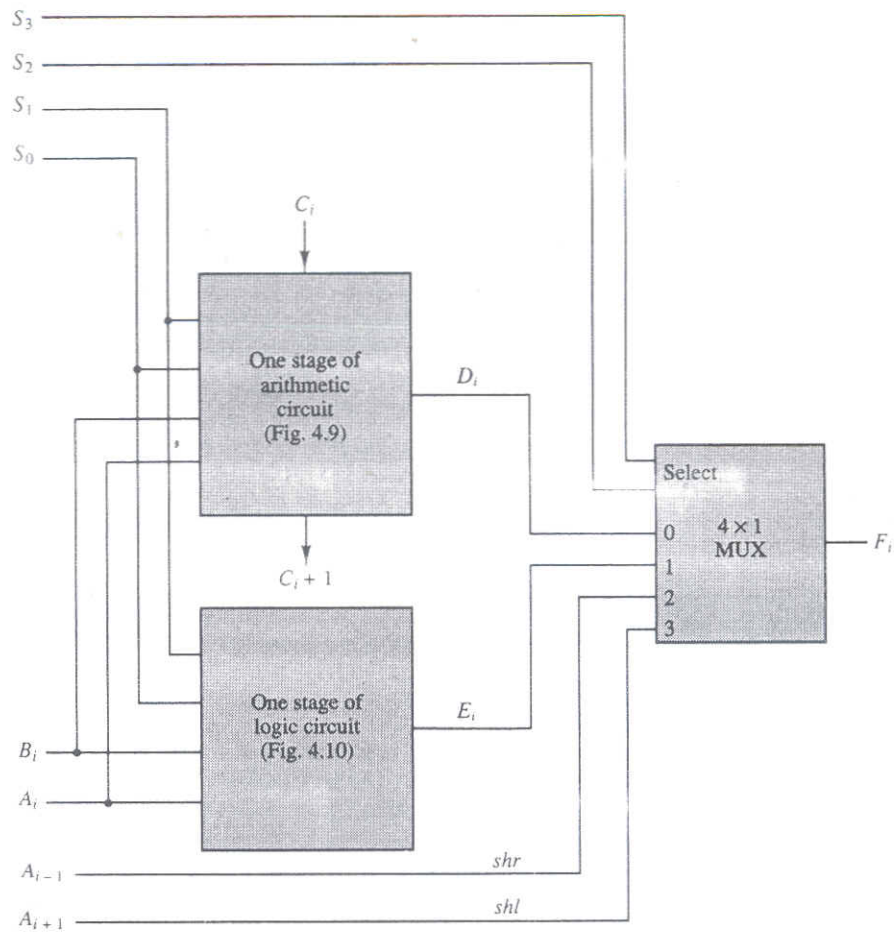


Figure 5-1 Stored program organization.

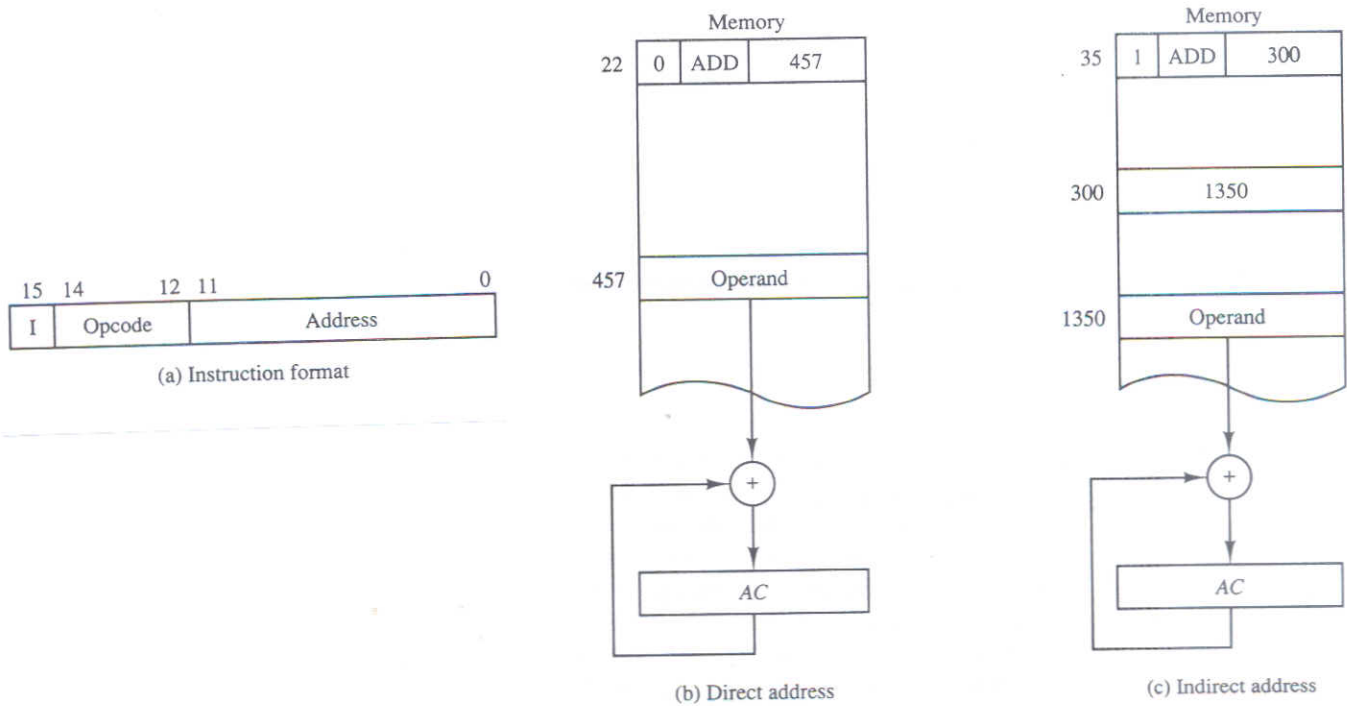
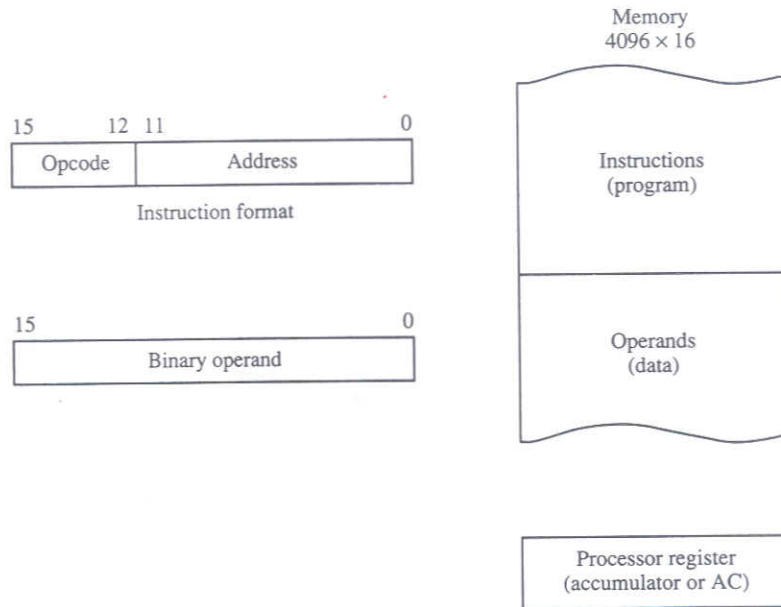


Figure 5-2 Demonstration of direct and indirect address.

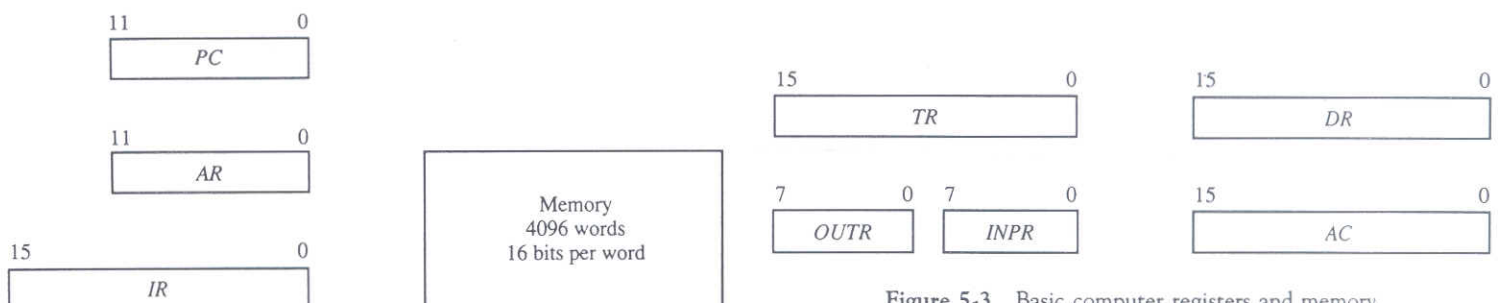


Figure 5-3 Basic computer registers and memory.

TABLE 5-1 List of Registers for the Basic Computer

Register symbol	Number of bits	Register name	Function
<i>DR</i>	16	Data register	Holds memory operand
<i>AR</i>	12	Address register	Holds address for memory
<i>AC</i>	16	Accumulator	Processor register
<i>IR</i>	16	Instruction register	Holds instruction code
<i>PC</i>	12	Program counter	Holds address of instruction
<i>TR</i>	16	Temporary register	Holds temporary data
<i>INPR</i>	8	Input register	Holds input character
<i>OUTR</i>	8	Output register	Holds output character

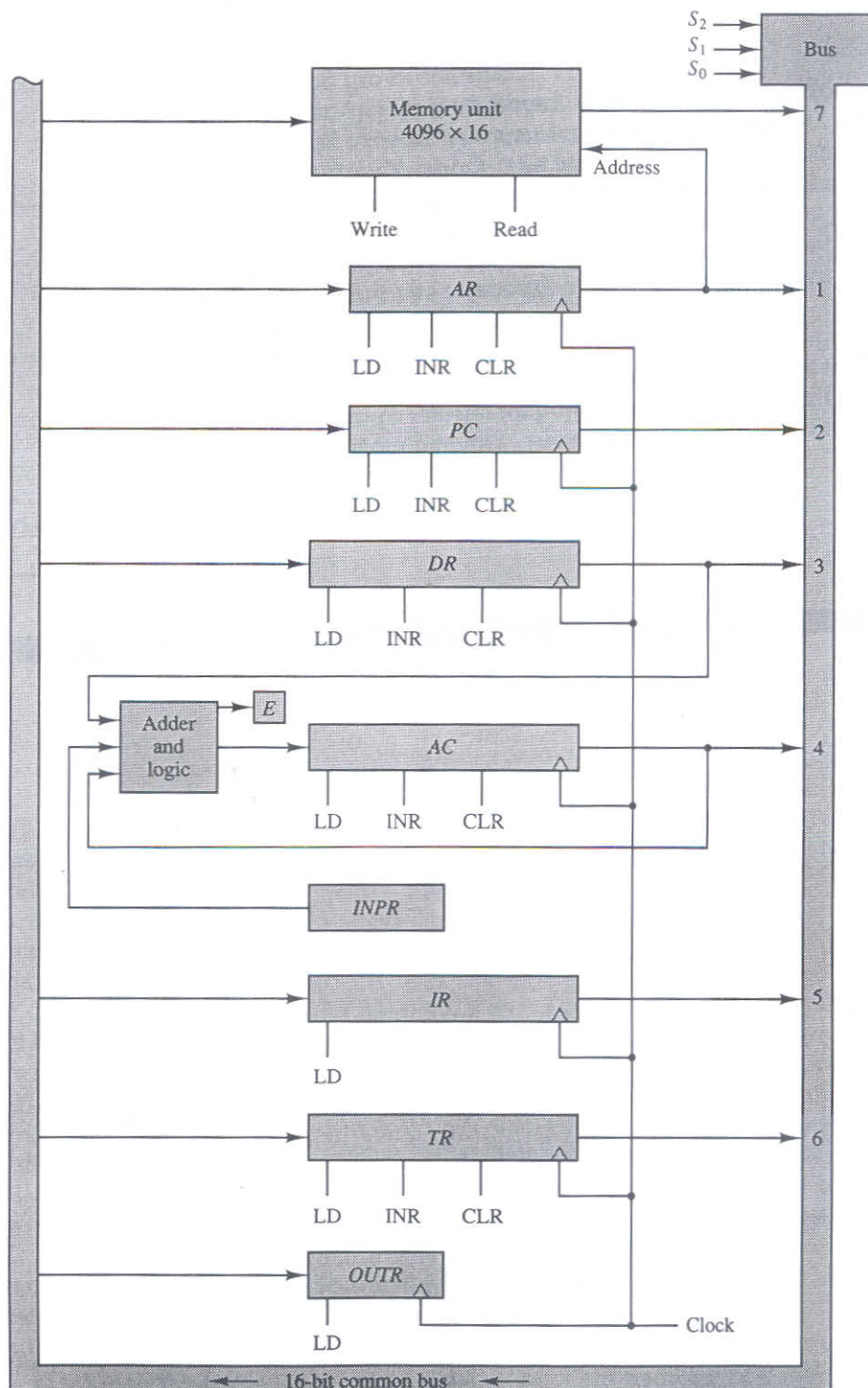


Figure 5-4 Basic computer registers connected to a common bus.

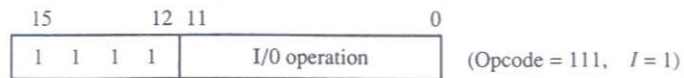
Figure 5-5 Basic computer instruction formats.



(a) Memory – reference instruction



(b) Register – reference instruction



(c) Input – output instruction

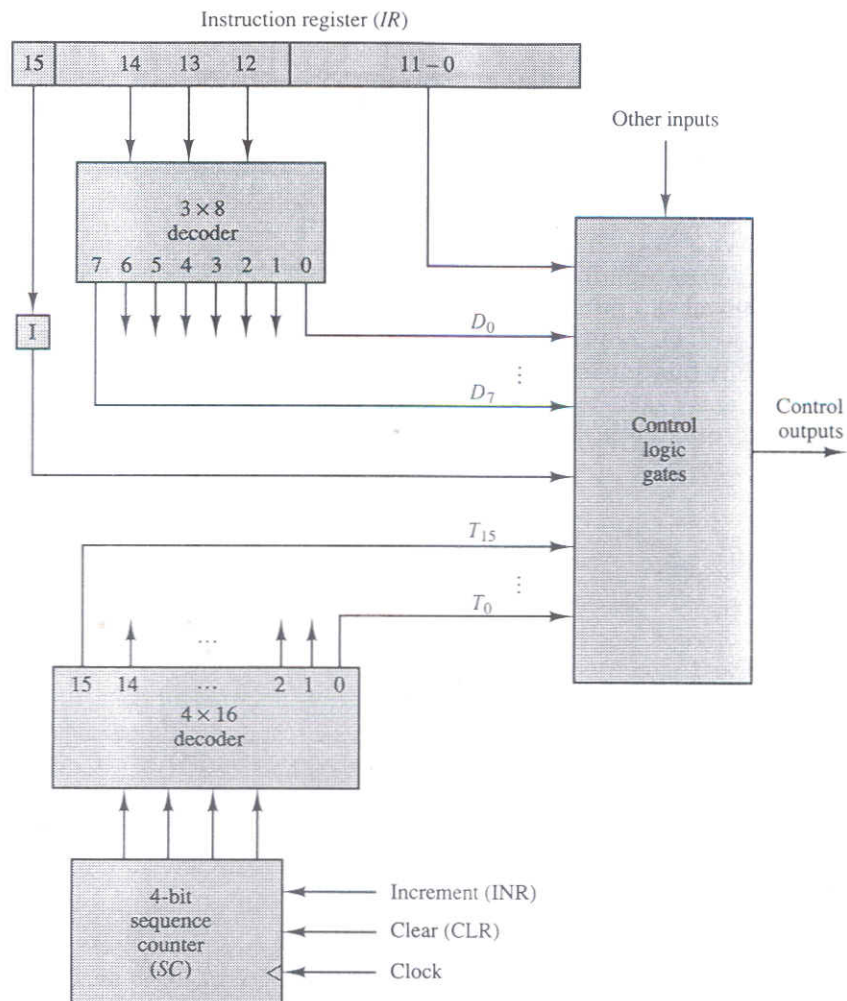


Figure 5-6 Control unit of basic computer.

TABLE 5-2 Basic Computer Instructions

Symbol	Hexadecimal code		Description
	$I = 0$	$I = 1$	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
SNA	7008		Skip next instruction if AC negative
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

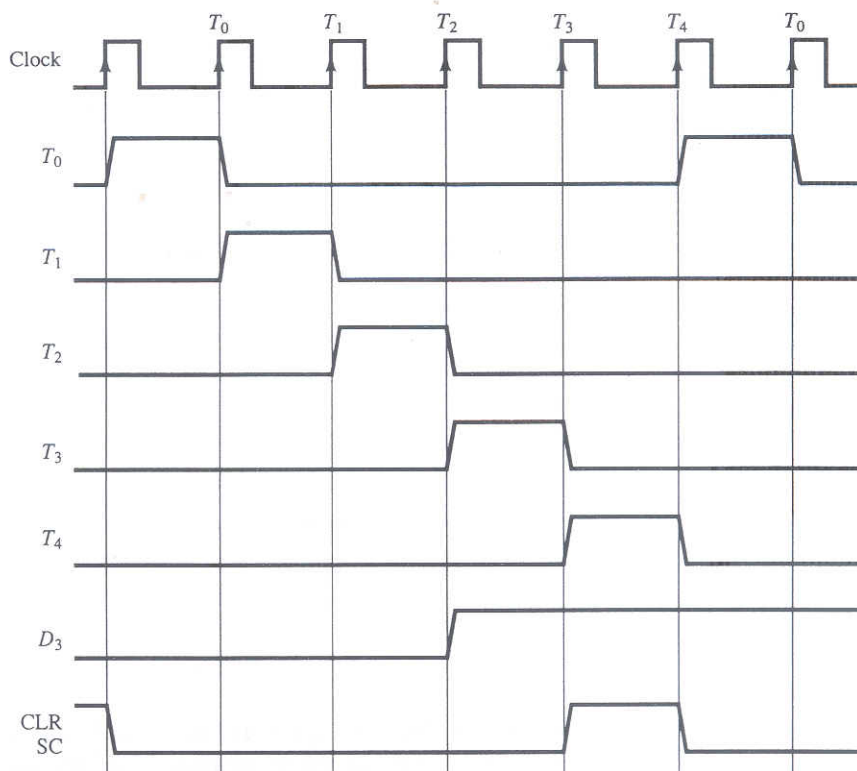


Figure 5-7 Example of control timing signals.

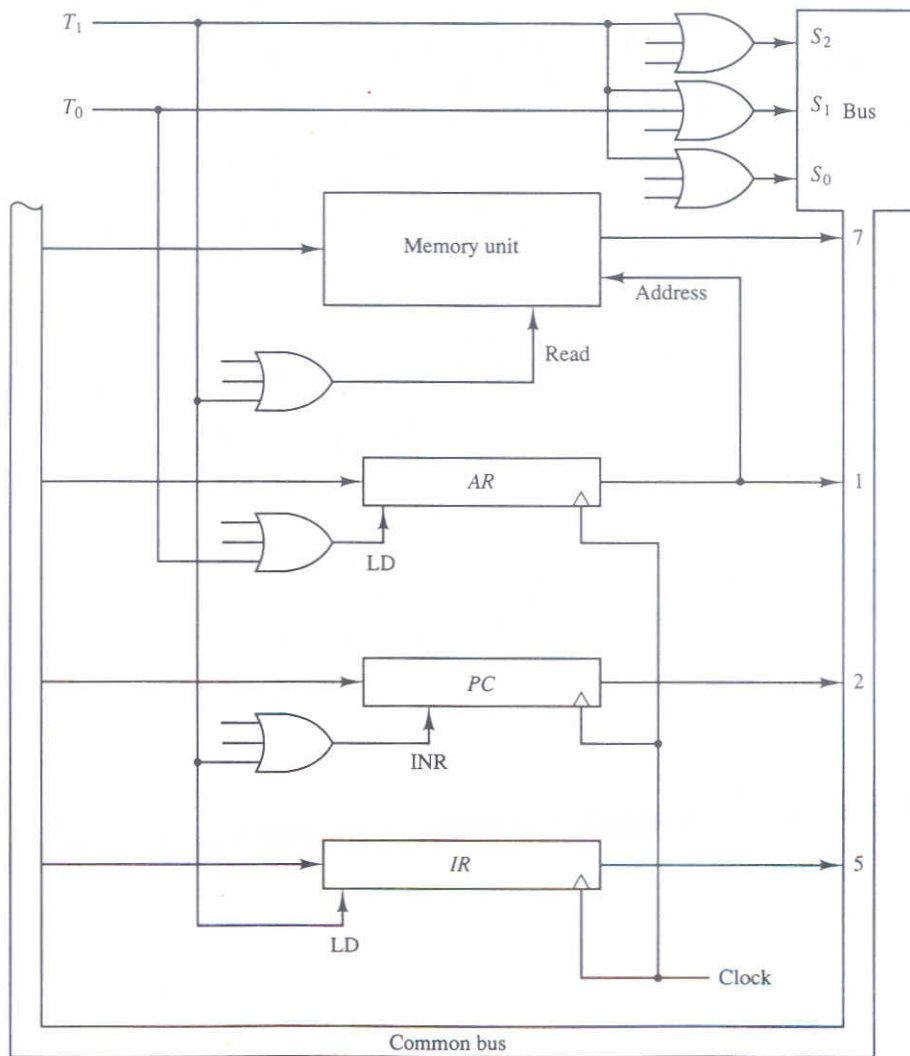


Figure 5-8 Register transfers for the fetch phase.

TABLE 5-3 Execution of Register-Reference Instructions

$D_7I'T_3 = r$ (common to all register-reference instructions)

$IR(i) = B_i$ [bit in $IR(0-11)$ that specifies the operation]

	r :	$SC \leftarrow 0$	Clear SC
CLA	rB_{11} :	$AC \leftarrow 0$	Clear AC
CLE	rB_{10} :	$E \leftarrow 0$	Clear E
CMA	rB_9 :	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8 :	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7 :	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	rB_6 :	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	rB_5 :	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4 :	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3 :	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2 :	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1 :	If $(E = 0)$ then $(PC \leftarrow PC + 1)$	Skip if E zero
HLT	rB_0 :	$S \leftarrow 0$ (S is a start-stop flip-flop)	Halt computer

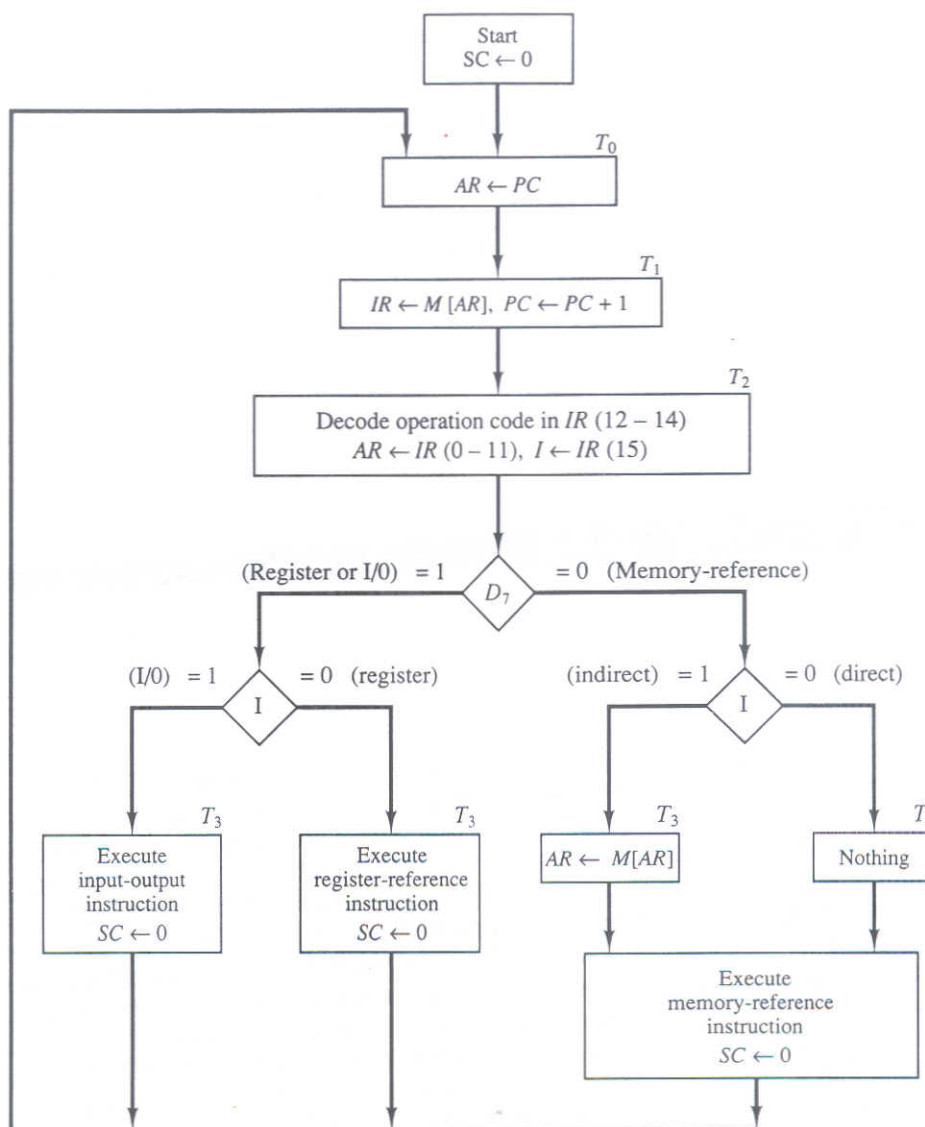
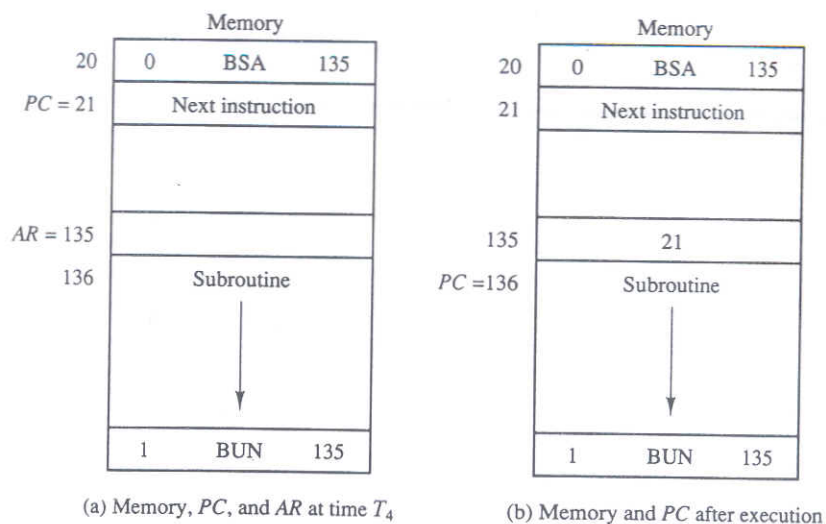


Figure 5-9 Flowchart for instruction cycle (initial configuration).

Figure 5-10 Example of BSA instruction execution.



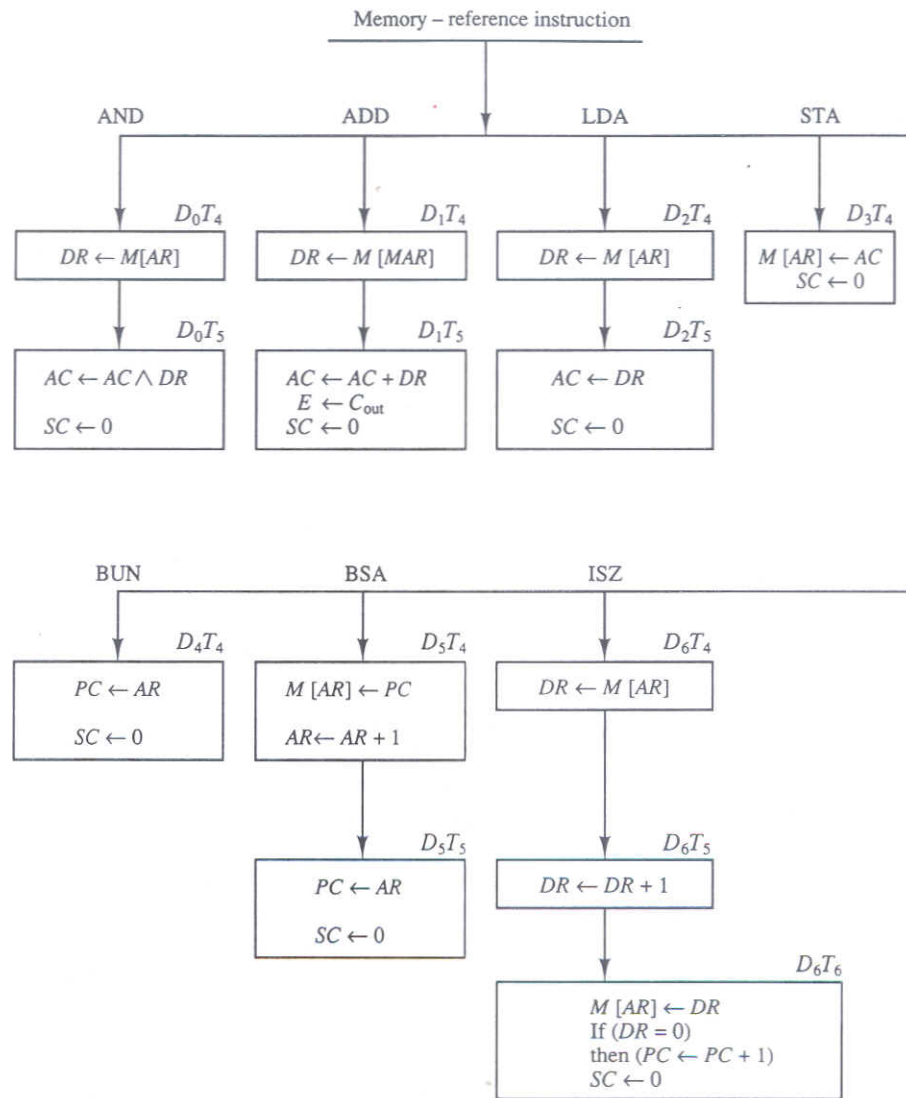


Figure 5-11 Flowchart for memory-reference instructions.

Figure 5-12 Input-output configuration.

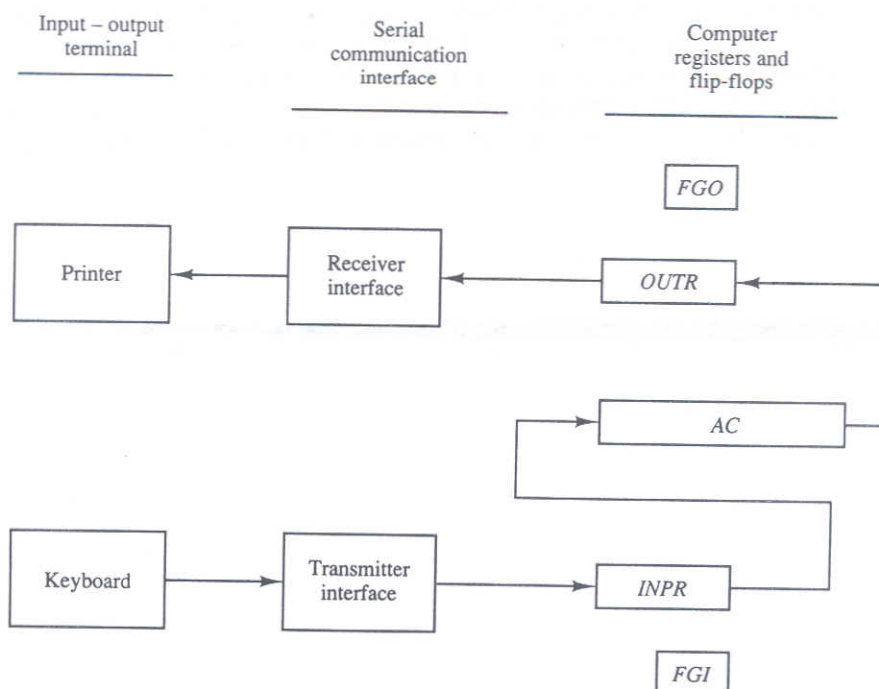


TABLE 5-4 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1,$ If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$

TABLE 5-5 Input-Output Instructions

$D_7IT_3 = p$ (common to all input-output instructions)

$IR(i) = B_i$ [bit in $IR(6-11)$ that specifies the instruction]

	p :	$SC \leftarrow 0$	Clear SC
INP	pB_{11} :	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$	Input character
OUT	pB_{10} :	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$	Output character
SKI	pB_9 :	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$	Skip on input flag
SKO	pB_8 :	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$	Skip on output flag
ION	pB_7 :	$IEN \leftarrow 1$	Interrupt enable on
IOF	pB_6 :	$IEN \leftarrow 0$	Interrupt enable off

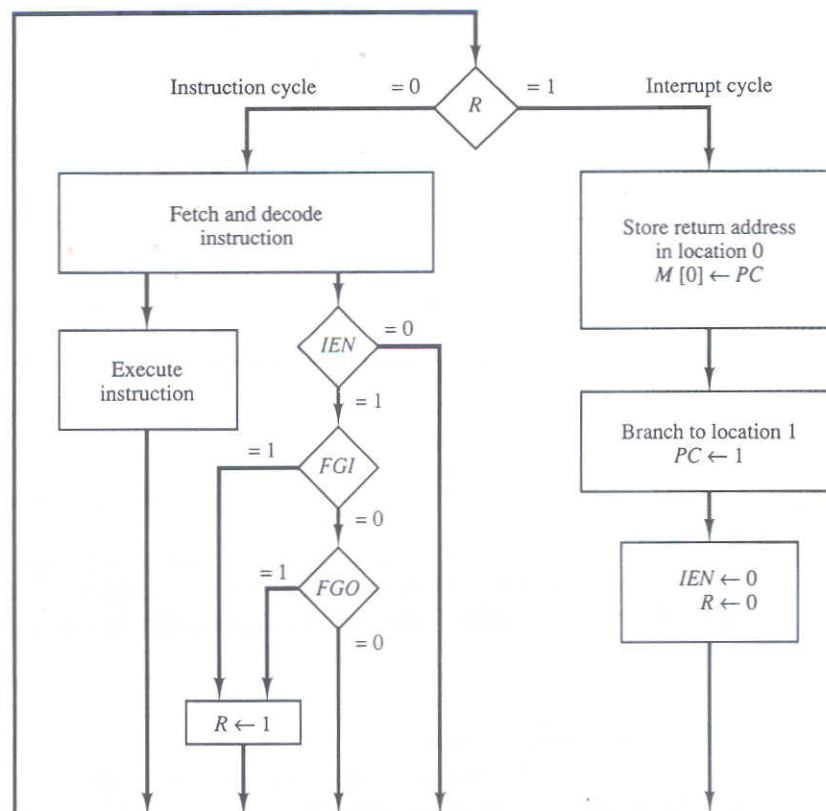


Figure 5-13 Flowchart for interrupt cycle.

Figure 5-14 Demonstration of the interrupt cycle.

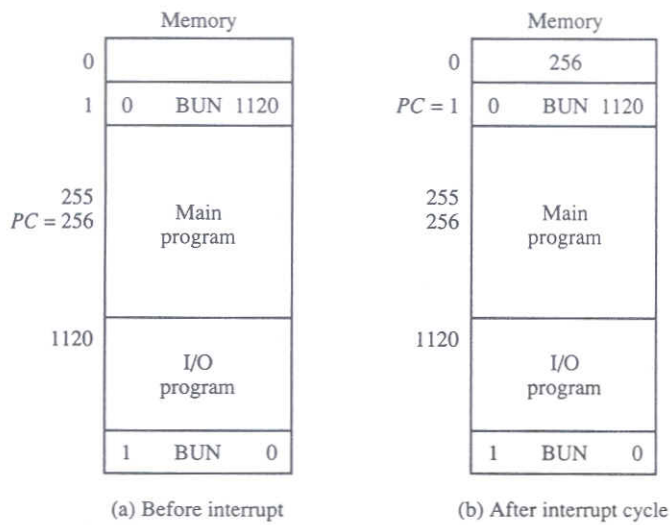


Figure 5-17 Control inputs for IEN.

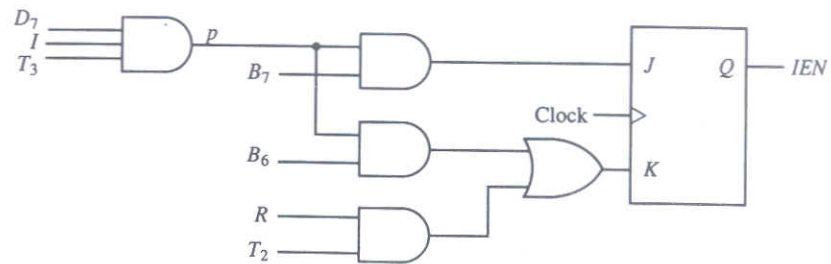


Figure 5-18 Encoder for bus selection inputs.

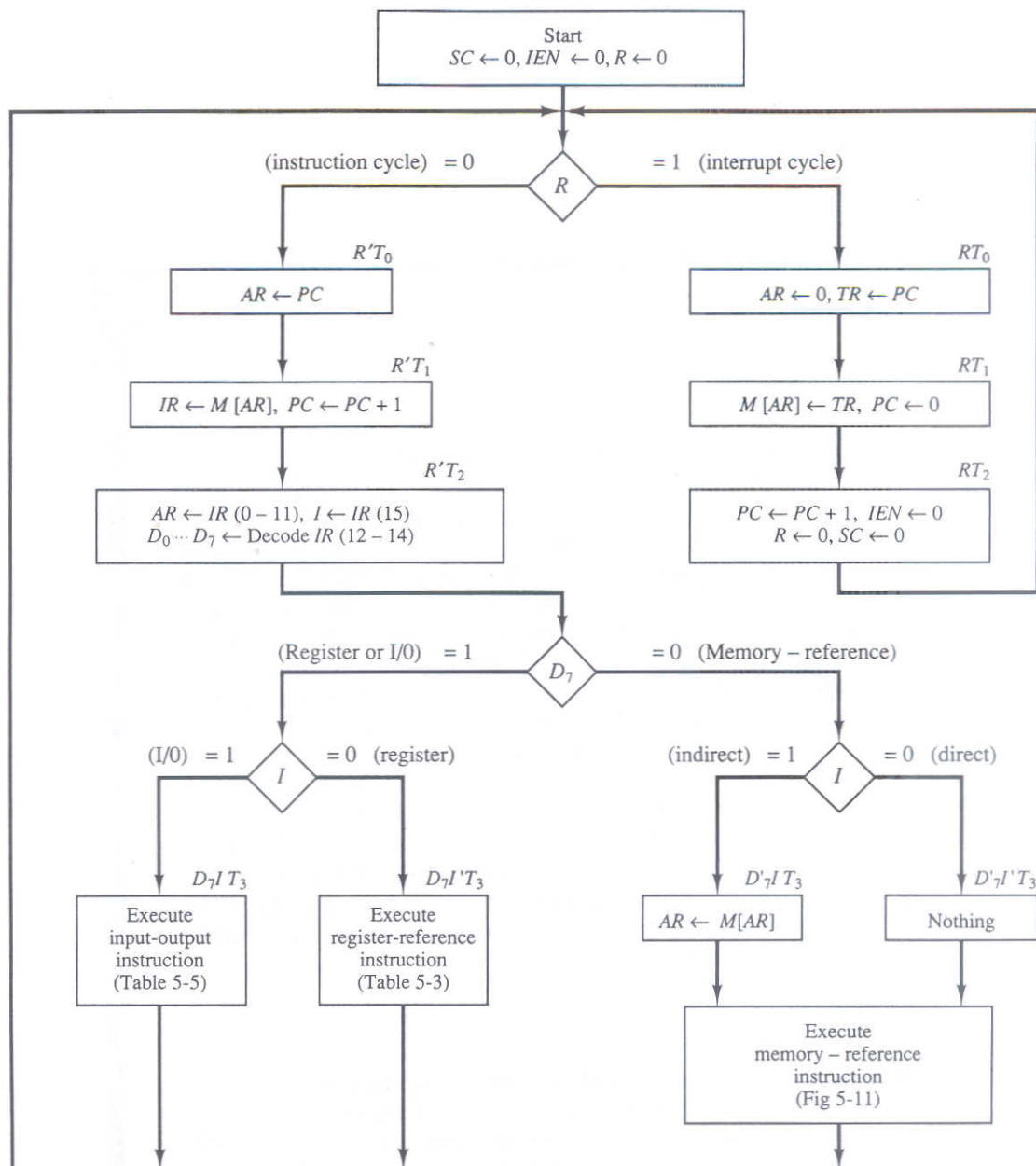
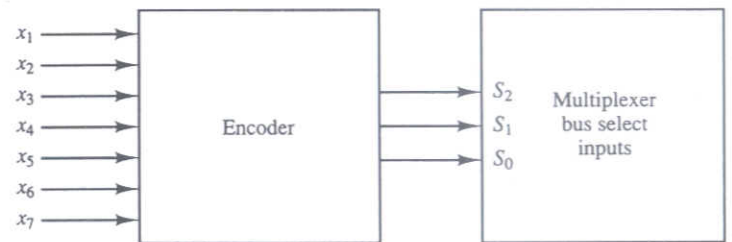


Figure 5-15 Flowchart for computer operation.

Figure 5-16 Control gates associated with AR.

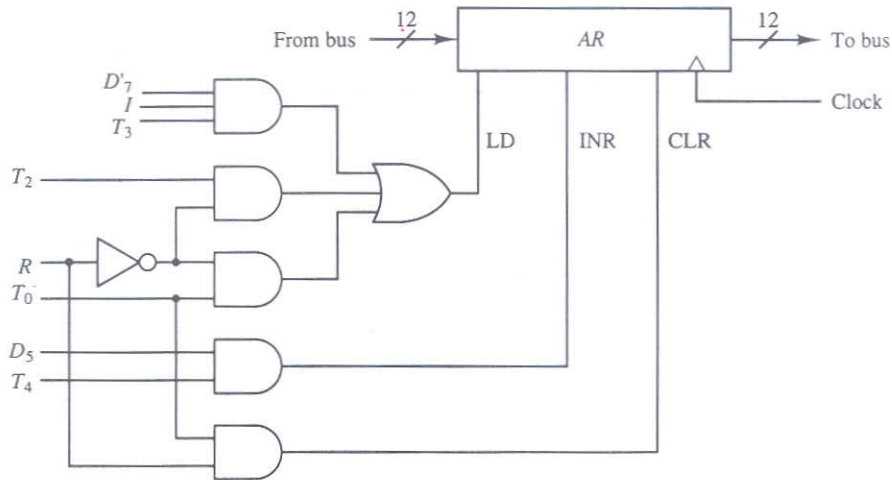


TABLE 5-6 Control Functions and Microoperations for the Basic Computer

Fetch	$R'T_0$: $AR \leftarrow PC$
	$R'T_1$: $IR \leftarrow M[AR], PC \leftarrow PC + 1$
Decode	$R'T_2$: $D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14),$ $AR \leftarrow IR(0-11), I \leftarrow IR(15)$
Indirect	D_7IT_3 : $AR \leftarrow M[AR]$
Interrupt:	
	$T_0T_1T_2(IEN)(FGI + FGO)$: $R \leftarrow 1$
	RT_0 : $AR \leftarrow 0, TR \leftarrow PC$
	RT_1 : $M[AR] \leftarrow TR, PC \leftarrow 0$
	RT_2 : $PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$
Memory-reference:	
AND	D_0T_4 : $DR \leftarrow M[AR]$ D_0T_5 : $AC \leftarrow AC \wedge DR, SC \leftarrow 0$
ADD	D_1T_4 : $DR \leftarrow M[AR]$ D_1T_5 : $AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$
LDA	D_2T_4 : $DR \leftarrow M[AR]$ D_2T_5 : $AC \leftarrow DR, SC \leftarrow 0$
STA	D_3T_4 : $M[AR] \leftarrow AC, SC \leftarrow 0$
BUN	D_4T_4 : $PC \leftarrow AR, SC \leftarrow 0$
BSA	D_5T_4 : $M[AR] \leftarrow PC, AR \leftarrow AR + 1$ D_5T_5 : $PC \leftarrow AR, SC \leftarrow 0$
ISZ	D_6T_4 : $DR \leftarrow M[AR]$ D_6T_5 : $DR \leftarrow DR + 1$ D_6T_6 : $M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$
Register-reference:	
	$D_7I'T_3 = r$ (common to all register-reference instructions)
	$IR(i) = B_i$ ($i = 0, 1, 2, \dots, 11$)
	r : $SC \leftarrow 0$
CLA	rB_{11} : $AC \leftarrow 0$
CLE	rB_{10} : $E \leftarrow 0$
CMA	rB_9 : $AC \leftarrow \overline{AC}$
CME	rB_8 : $E \leftarrow \overline{E}$
CIR	rB_7 : $AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
CIL	rB_6 : $AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
INC	rB_5 : $AC \leftarrow AC + 1$
SPA	rB_4 : If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	rB_3 : If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	rB_2 : If $(AC = 0)$ then $PC \leftarrow PC + 1$
SZE	rB_1 : If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	rB_0 : $S \leftarrow 0$
Input-output:	
	$D_7IT_3 = p$ (common to all input-output instructions)
	$IR(i) = B_i$ ($i = 6, 7, 8, 9, 10, 11$)
	p : $SC \leftarrow 0$
INP	pB_{11} : $AC(0-7) \leftarrow INPR, FGI \leftarrow 0$
OUT	pB_{10} : $OUTR \leftarrow AC(0-7), FGO \leftarrow 0$
SKI	pB_9 : If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$
SKO	pB_8 : If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$
ION	pB_7 : $IEN \leftarrow 1$
IOF	pB_6 : $IEN \leftarrow 0$

TABLE 5-7 Encoder for Bus Selection Circuit

Inputs							Outputs			Register selected for bus
x_1	x_2	x_3	x_4	x_5	x_6	x_7	S_2	S_1	S_0	
0	0	0	0	0	0	0	0	0	0	None
1	0	0	0	0	0	0	0	0	1	AR
0	1	0	0	0	0	0	0	1	0	PC
0	0	1	0	0	0	0	0	1	1	DR
0	0	0	1	0	0	0	1	0	0	AC
0	0	0	0	1	0	0	1	0	1	IR
0	0	0	0	0	1	0	1	1	0	TR
0	0	0	0	0	0	1	1	1	1	Memory

Figure 5-19 Circuits associated with AC.

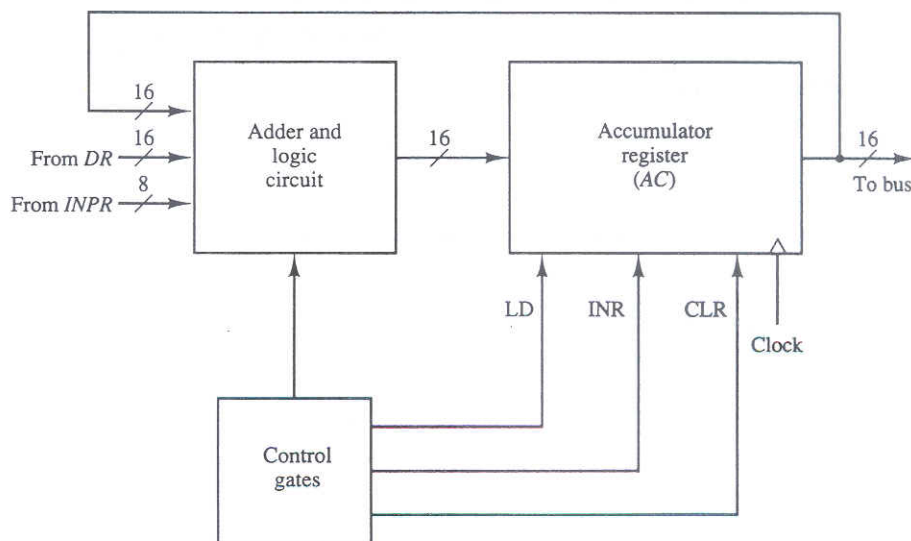


Figure 5-20 Gate structure for controlling the LD, INR, and CLR of AC.

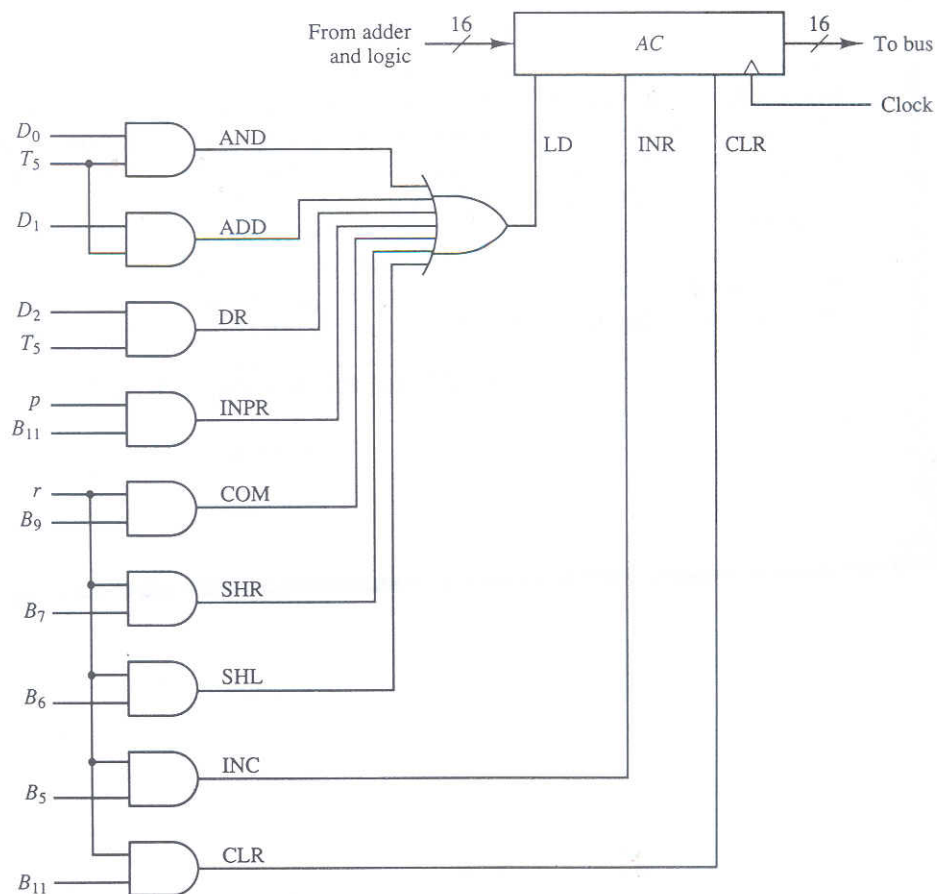


Figure 5-21 One stage of adder and logic circuit.

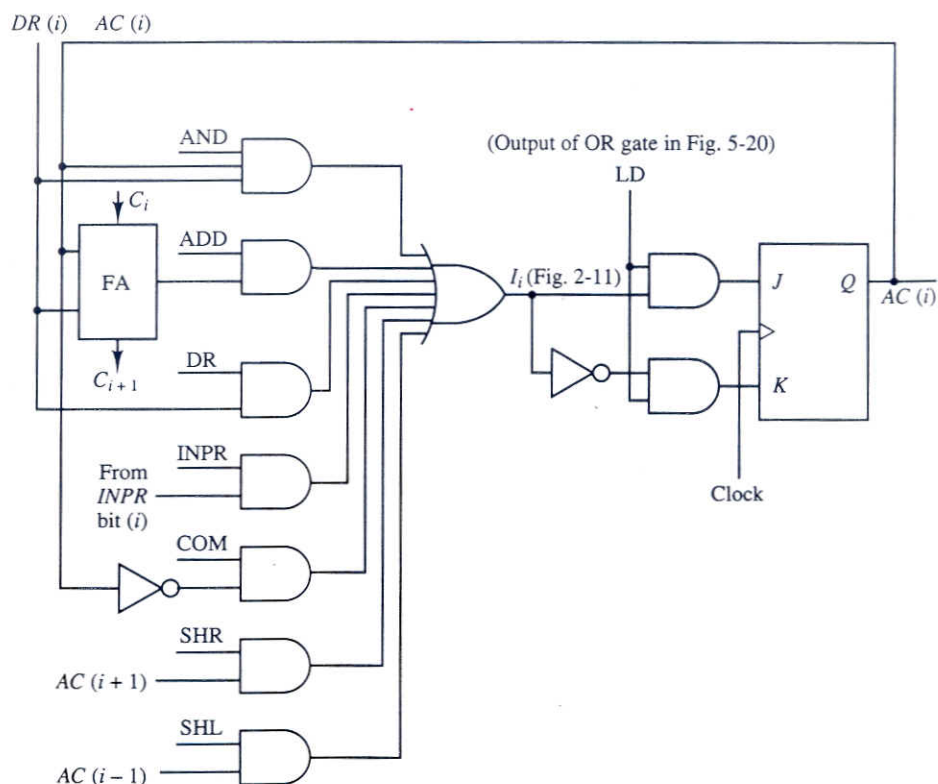


TABLE 6-1 Computer Instructions

Symbol	Hexadecimal code	Description
AND	0 or 8	AND M to AC
ADD	1 or 9	Add M to AC , carry to E
LDA	2 or A	Load AC from M
STA	3 or B	Store AC in M
BUN	4 or C	Branch unconditionally to m
BSA	5 or D	Save return address in m and branch to $m + 1$
ISZ	6 or E	Increment M and skip if zero
CLA	7800	Clear AC
CLE	7400	Clear E
CMA	7200	Complement AC
CME	7100	Complement E
CIR	7080	Circulate right E and AC
CIL	7040	Circulate left E and AC
INC	7020	Increment AC ,
SPA	7010	Skip if AC is positive
SNA	7008	Skip if AC is negative
SZA	7004	Skip if AC is zero
SZE	7002	Skip if E is zero
HLT	7001	Halt computer
INP	F800	Input information and clear flag
OUT	F400	Output information and clear flag
SKI	F200	Skip if input flag is on
SKO	F100	Skip if output flag is on
ION	F080	Turn interrupt on
IOF	F040	Turn interrupt off

TABLE 6-2 Binary Program to Add Two Numbers

Location	Instruction code
0	0010 0000 0000 0100
1	0001 0000 0000 0101
10	0011 0000 0000 0110
11	0111 0000 0000 0001
100	0000 0000 0101 0011
101	1111 1111 1110 1001
110	0000 0000 0000 0000

TABLE 6-3 Hexadecimal Program to Add Two Numbers

Location	Instruction
000	2004
001	1005
002	3006
003	7001
004	0053
005	FFE9
006	0000

TABLE 6-4 Program with Symbolic Operation Codes

Location	Instruction	Comments
000	LDA 004	Load first operand into AC
001	ADD 005	Add second operand to AC
002	STA 006	Store sum in location 006
003	HLT	Halt computer
004	0053	First operand
005	FFE9	Second operand (negative)
006	0000	Store sum here

TABLE 6-5 Assembly Language Program to Add Two Numbers

ORG 0	/Origin of program is location 0
LDA A	/Load operand from location A
ADD B	/Add operand from location B
STA C	/Store sum in location C
HLT	/Halt computer
A, DEC 83	/Decimal operand
B, DEC -23	/Decimal operand
C, DEC 0	/Sum stored in location C
END	/End of symbolic program

TABLE 6-6 Fortran Program to Add Two Numbers

```

INTEGER A, B, C
DATA A,83 B,-23
C = A + B
END

```

TABLE 6-7 Definition of Pseudoinstructions

Symbol	Information for the Assembler
ORG N	Hexadecimal number N is the memory location for the instruction or operand listed in the following line
END	Denotes the end of symbolic program
DEC N	Signed decimal number N to be converted to binary
HEX N	Hexadecimal number N to be converted to binary

TABLE 6-8 Assembly Language Program to Subtract Two Numbers

	ORG 100	/Origin of program is location 100
	LDA SUB	/Load subtrahend to AC
	CMA	/Complement AC
	INC	/Increment AC
	ADD MIN	/Add minuend to AC
	STA DIF	/Store difference
	HLT	/Halt computer
MIN,	DEC 83	/Minuend
SUB,	DEC -23	/Subtrahend
DIF,	HEX 0	/Difference stored here
	END	/End of symbolic program

TABLE 6-9 Listing of Translated Program of Table 6-8

Hexadecimal code		
Location	Content	Symbolic program
		ORG 100
		LDA SUB
		CMA
		INC
		ADD MIN
		STA DIF
		HLT
100	2107	MIN, DEC 83
101	7200	SUB, DEC -23
102	7020	DIF, HEX 0
103	1106	END
104	3108	
105	7001	
106	0053	
107	FFE9	
108	0000	

TABLE 6-10 Hexadecimal Character Code

Character	Code	Character	Code	Character	Code
A	41	Q	51	6	36
B	42	R	52	7	37
C	43	S	53	8	38
D	44	T	54	9	39
E	45	U	55	space	20
F	46	V	56	(	28
G	47	W	57	)	29
H	48	X	58	*	2A
I	49	Y	59	+	2B
J	4A	Z	5A	,	2C
K	4B	0	30	-	2D
L	4C	1	31	.	2E
M	4D	2	32	/	2F
N	4E	3	33	=	3D
O	4F	4	34	CR	0D (carriage return)
P	50	5	35		

TABLE 6-11 Computer Representation of the Line of Code: PL3, LDA SUB I

Memory word	Symbol	Hexadecimal code	Binary representation
1	P L	50 4C	0101 0000 0100 1100
2	3 ,	33 2C	0011 0011 0010 1100
3	L D	4C 44	0100 1100 0100 0100
4	A	41 20	0100 0001 0010 0000
5	S U	53 55	0101 0011 0101 0101
6	B	42 20	0100 0010 0010 0000
7	I CR	49 0D	0100 1001 0000 1101

TABLE 6-12 Address Symbol Table for Program in Table 6-8

Memory word	Symbol or (LC)*	Hexadecimal code	Binary representation
1	M I	4D 49	0100 1101 0100 1001
2	N ,	4E 2C	0100 1110 0010 1100
3	(LC)	01 06	0000 0001 0000 0110
4	S U	53 55	0101 0011 0101 0101
5	B ,	42 2C	0100 0010 0010 1100
6	(LC)	01 07	0000 0001 0000 0111
7	D I	44 49	0100 0100 0100 1001
8	F ,	46 2C	0100 0110 0010 1100
9	(LC)	01 08	0000 0001 0000 1000

* (LC) designates content of location counter.

TABLE 6-13 Symbolic Program to Add 100 Numbers

Line		
1	ORG 100	/Origin of program is HEX 100
2	LDA ADS	/Load first address of operands
3	STA PTR	/Store in pointer
4	LDA NBR	/Load minus 100
5	STA CTR	/Store in counter
6	CLA	/Clear accumulator
7	LOP, ADD PTR I	/Add an operand to AC
8	ISZ PTR	/Increment pointer
9	ISZ CTR	/Increment counter
10	BUN LOP	/Repeat loop again
11	STA SUM	/Store sum
12	HLT	/Halt
13	ADS, HEX 150	/First address of operands
14	PTR, HEX 0	/This location reserved for a pointer
15	NBR, DEC -100	/Constant to initialize counter
16	CTR, HEX 0	/This location reserved for a counter
17	SUM, HEX 0	/Sum is stored here
18	ORG 150	/Origin of operands is HEX 150
19	DEC 75	/First operand
.		
.		
.		
118	DEC 23	/Last operand
119	END	/End of symbolic program

TABLE 6-14 Program to Multiply Two Positive Numbers

	ORG 100	
LOP,	CLE	/Clear E
	LDA Y	/Load multiplier
	CIR	/Transfer multiplier bit to E
	STA Y	/Store shifted multiplier
	SZE	/Check if bit is zero
	BUN ONE	/Bit is one; go to ONE
	BUN ZRO	/Bit is zero; go to ZRO
ONE,	LDA X	/Load multiplicand
	ADD P	/Add to partial product
	STA P	/Store partial product
	CLE	/Clear E
ZRO,	LDA X	/Load multiplicand
	CIL	/Shift left
	STA X	/Store shifted multiplicand
	ISZ CTR	/Increment counter
	BUN LOP	/Counter not zero; repeat loop
	HLT	/Counter is zero; halt
CTR,	DEC -8	/This location serves as a counter
X,	HEX 000F	/Multiplicand stored here
Y,	HEX 000B	/Multiplier stored here
P,	HEX 0	/Product formed here
	END	

TABLE 6-15 Program to Add Two Double-Precision Numbers

	LDA AL	/Load A low
	ADD BL	/Add B low, carry in E
	STA CL	/Store in C low
	CLA	/Clear AC
	CIL	/Circulate to bring carry into AC(16)
	ADD AH	/Add A high and carry
	ADD BH	/Add B high
	STA CH	/Store in C high
	HLT	
AL,	—	/Location of operands
AH,	—	
BL,	—	
BH,	—	
CL,	—	
CH,	—	

TABLE 6-16 Program to Demonstrate the Use of Subroutines

Location			
		ORG 100	/Main program
100		LDA X	/Load X
101		BSA SH4	/Branch to subroutine
102		STA X	/Store shifted number
103		LDA Y	/Load Y
104		BSA SH4	/Branch to subroutine again
105		STA Y	/Store shifted number
106		HLT	
107	X,	HEX 1234	
108	Y,	HEX 4321	
			/Subroutine to shift left 4 times
109	SH4,	HEX 0	/Store return address here
10A		CIL	/Circulate left once
10B		CIL	
10C		CIL	
10D		CIL	/Circulate left fourth time
10E		AND MSK	/Set AC(13-16) to zero
10F		BUN SH4 I	/Return to main program
110	MSK,	HEX FFF0	/Mask operand
		END	

TABLE 6-17 Program to Demonstrate Parameter Linkage

Location			
		ORG 200	
200		LDA X	/Load first operand into AC
201		BSA OR	/Branch to subroutine OR
202		HEX 3AF6	/Second operand stored here
203		STA Y	/Subroutine returns here
204		HLT	
205	X,	HEX 7B95	/First operand stored here
206	Y,	HEX 0	/Result stored here
207	OR,	HEX 0	/Subroutine OR
208		CMA	/Complement first operand
209		STA TMP	/Store in temporary location
20A		LDA OR I	/Load second operand
20B		CMA	/Complement second operand
20C		AND TMP	/AND complemented first operand
20D		CMA	/Complement again to get OR
20E		ISZ OR	/Increment return address
20F		BUN OR I	/Return to main program
210	TMP,	HEX 0	/Temporary storage
		END	

TABLE 6-18 Subroutine to Move a Block of Data

		/Main program
	BSA MVE	/Branch to subroutine
	HEX 100	/First address of source data
	HEX 200	/First address of destination data
	DEC -16	/Number of items to move
	HLT	
MVE,	HEX 0	/Subroutine MVE
	LDA MVE I	/Bring address of source
	STA PT1	/Store in first pointer
	ISZ MVE	/Increment return address
	LDA MVE I	/Bring address of destination
	STA PT2	/Store in second pointer
	ISZ MVE	/Increment return address
	LDA MVE I	/Bring number of items
	STA CTR	/Store in counter
	ISZ MVE	/Increment return address
LOP,	LDA PT1 I	/Load source item
	STA PT2 I	/Store in destination
	ISZ PT1	/Increment source pointer
	ISZ PT2	/Increment destination pointer
	ISZ CTR	/Increment counter
	BUN LOP	/Repeat 16 times
	BUN MVE I	/Return to main program
PT1,	—	
PT2,	—	
CTR,	—	

TABLE 6-19 Programs to Input and Output One Character

(a) Input a character:

CIF,	SKI	/Check input flag
	BUN CIF	/Flag=0, branch to check again
	INP	/Flag=1, input character
	OUT	/Print character
	STA CHR	/Store character
	HLT	

CHR, — /Store character here

(b) Output one character:

	LDA CHR	/Load character into AC
COF,	SKO	/Check output flag
	BUN COF	/Flag=0, branch to check again
	OUT	/Flag=1, output character
	HLT	
CHR,	HEX 0057	/Character is "W"

TABLE 6-20 Subroutine to Input and Pack Two Characters

IN2,	—	/Subroutine entry
FST,	SKI	
	BUN FST	
	INP	/Input first character
	OUT	
	BSA SH4	/Shift left four times
	BSA SH4	/Shift left four more times
SCD,	SKI	
	BUN SCD	
	INP	/Input second character
	OUT	
	BUN IN2 I	/Return

TABLE 6-21 Program to Store Input Characters in a Buffer

	LDA ADS	/Load first address of buffer
	STA PTR	/Initialize pointer
LOP,	BSA IN2	/Go to subroutine IN2 (Table 6-20)
	STA PTR I	/Store double character word in buffer
	ISZ PTR	/Increment pointer
	BUN LOP	/Branch to input more characters
	HLT	
ADS,	HEX 500	/First address of buffer
PTR,	HEX 0	/Location for pointer

TABLE 6-22 Program to Compare Two Words

	LDA WD1	/Load first word
	CMA	
	INC	/Form 2's complement
	ADD WD2	/Add second word
	SZA	/Skip if AC is zero
	BUN UEQ	/Branch to "unequal" routine
	BUN EQL	/Branch to "equal" routine
WD1,	—	
WD2,	—	

TABLE 6-23 Program to Service an Interrupt

Location			
0	ZRO,	—	/Return address stored here
1		BUN SRV	/Branch to service routine
100		CLA	/Portion of running program
101		ION	/Turn on interrupt facility
102		LDA X	
103		ADD Y	/Interrupt occurs here
104		STA Z	/Program returns here after interrupt
.		.	
.		.	
.		.	/Interrupt service routine
200	SRV,	STA SAC	/Store content of AC
		CIR	/Move E into AC(1)
		STA SE	/Store content of E
		SKI	/Check input flag
		BUN NXT	/Flag is off, check next flag
		INP	/Flag is on, input character
		OUT	/Print character
		STA PT1 I	/Store it in input buffer
		ISZ PT1	/Increment input pointer
	NXT,	SKO	/Check output flag
		BUN EXT	/Flag is off, exit
		LDA PT2 I	/Load character from output buffer
		OUT	/Output character
		ISZ PT2	/Increment output pointer
	EXT,	LDA SE	/Restore value of AC(1)
		CIL	/Shift it to E
		LDA SAC	/Restore content of AC
		ION	/Turn interrupt on
		BUN ZRO I	/Return to running program
	SAC,	—	/AC is stored here
	SE,	—	/E is stored here
	PT1,	—	/Pointer of input buffer
	PT2,	—	/Pointer of output buffer

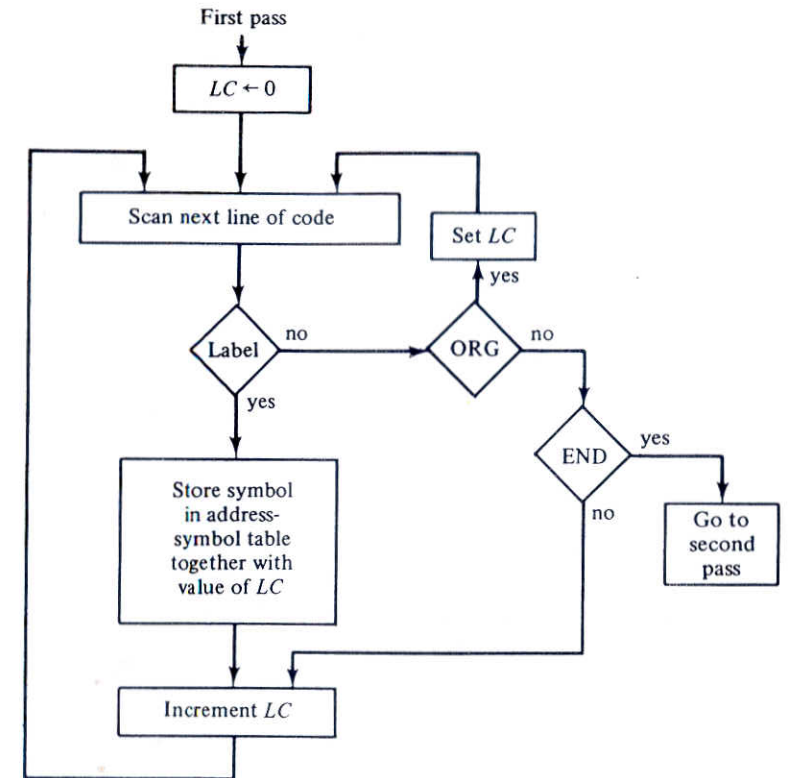


Figure 6-1 Flowchart for first pass of assembler.

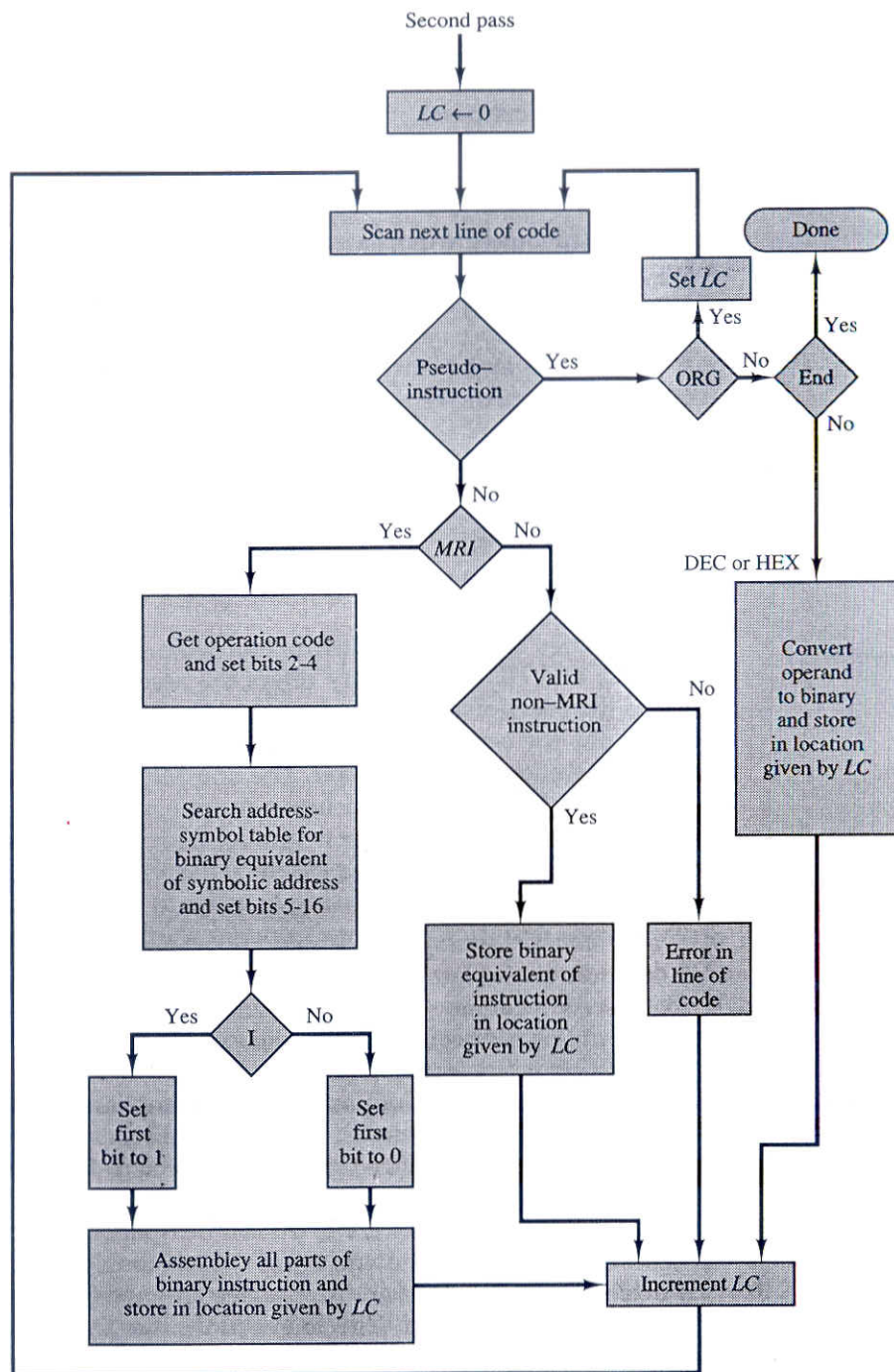


Figure 6-2 Flowchart for second pass of assembler.

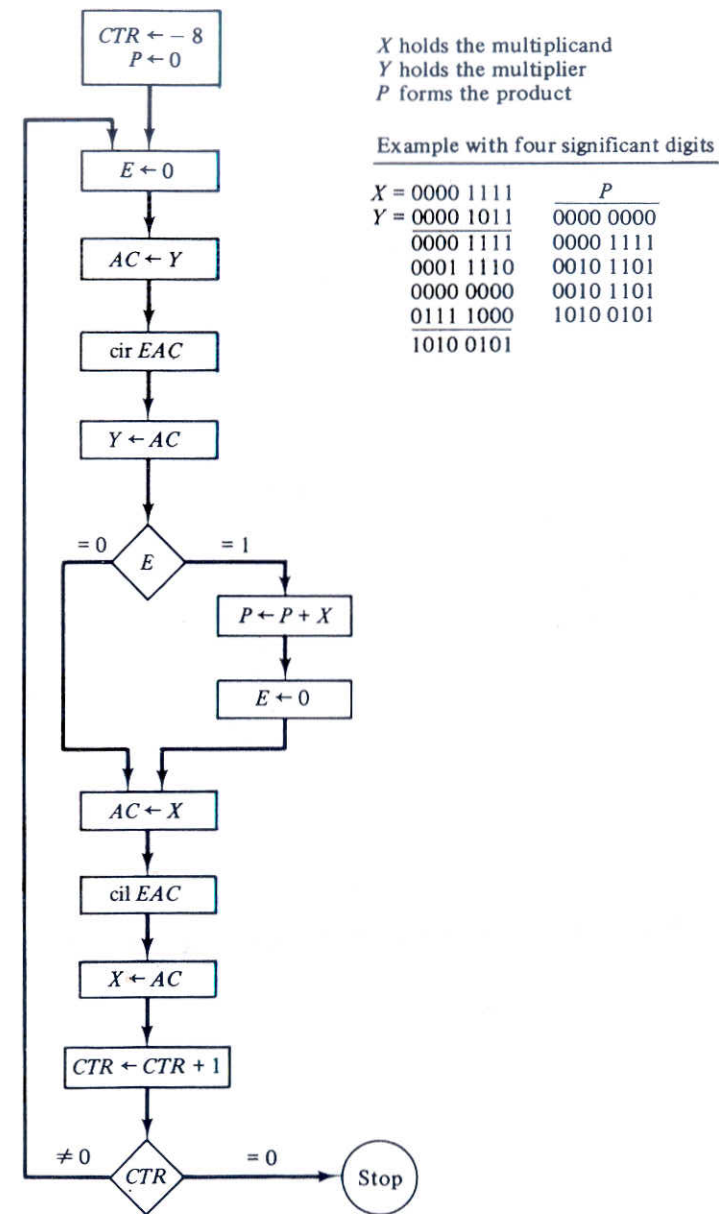


Figure 6-3 Flowchart for multiplication program.

Figure 7-1 Microprogrammed control organization.

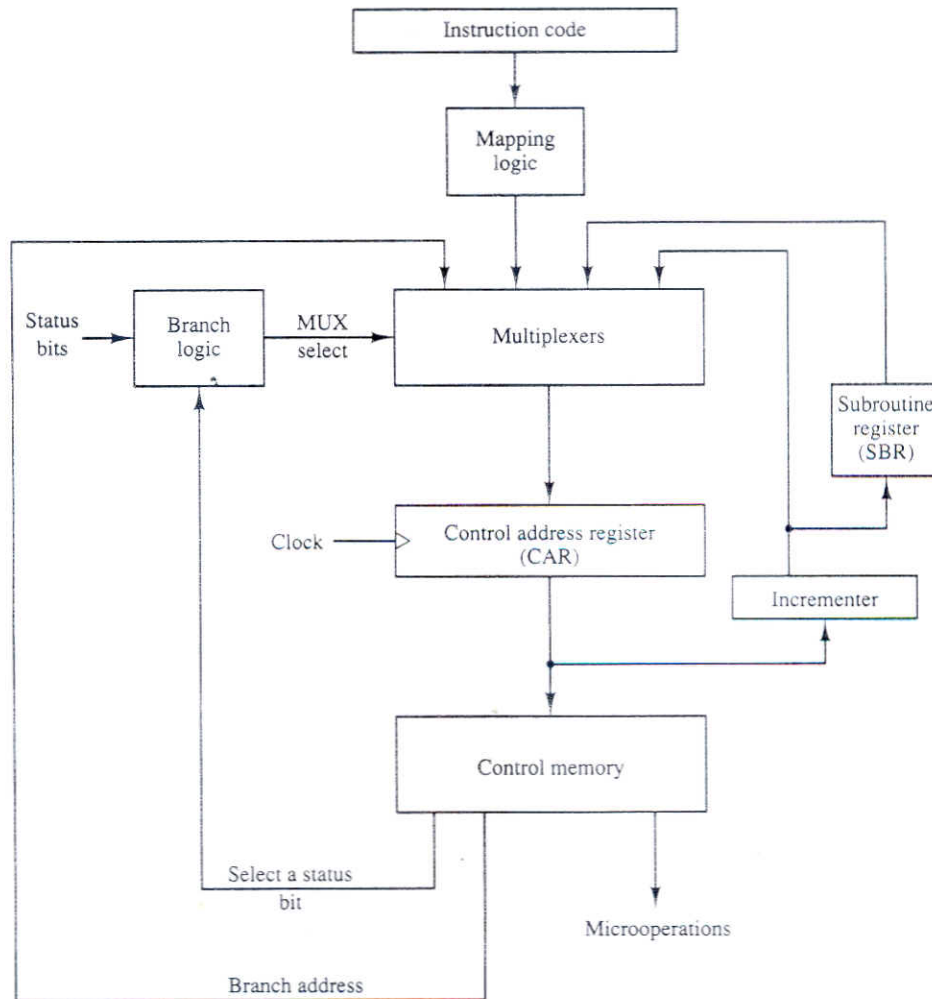
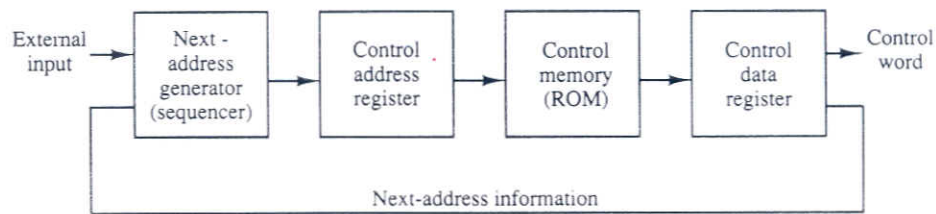
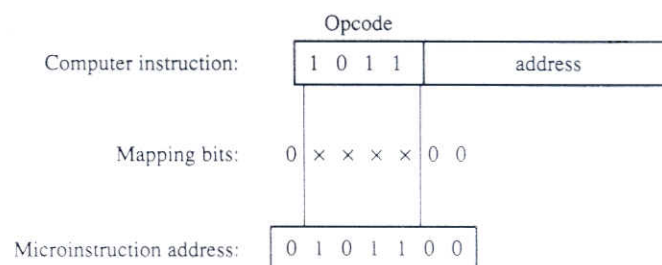


Figure 7-2 Selection of address for control memory.

Figure 7-3 Mapping from instruction code to microinstruction address.



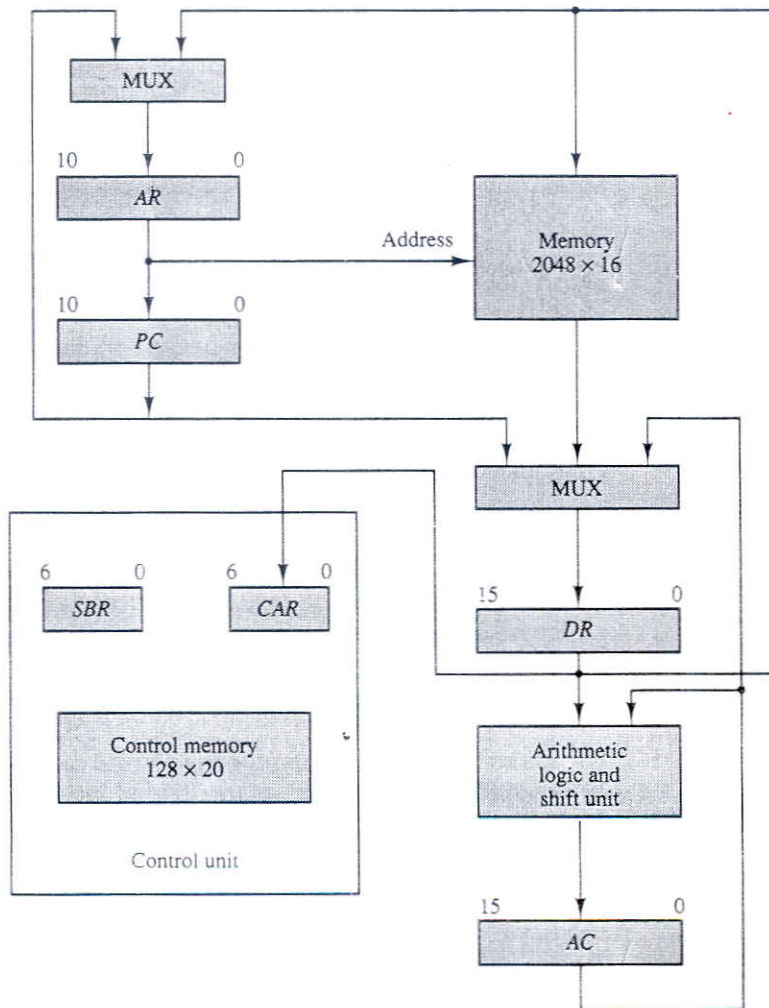


Figure 7-4 Computer hardware configuration.

Figure 7-5 Computer instructions.

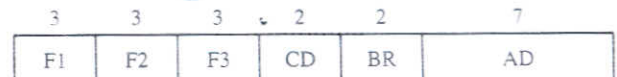


(a) Instruction format

Symbol	Opcode	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	If $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

EA is the effective address

(b) Four computer instructions



F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

Figure 7-6 Microinstruction code format (20 bits).

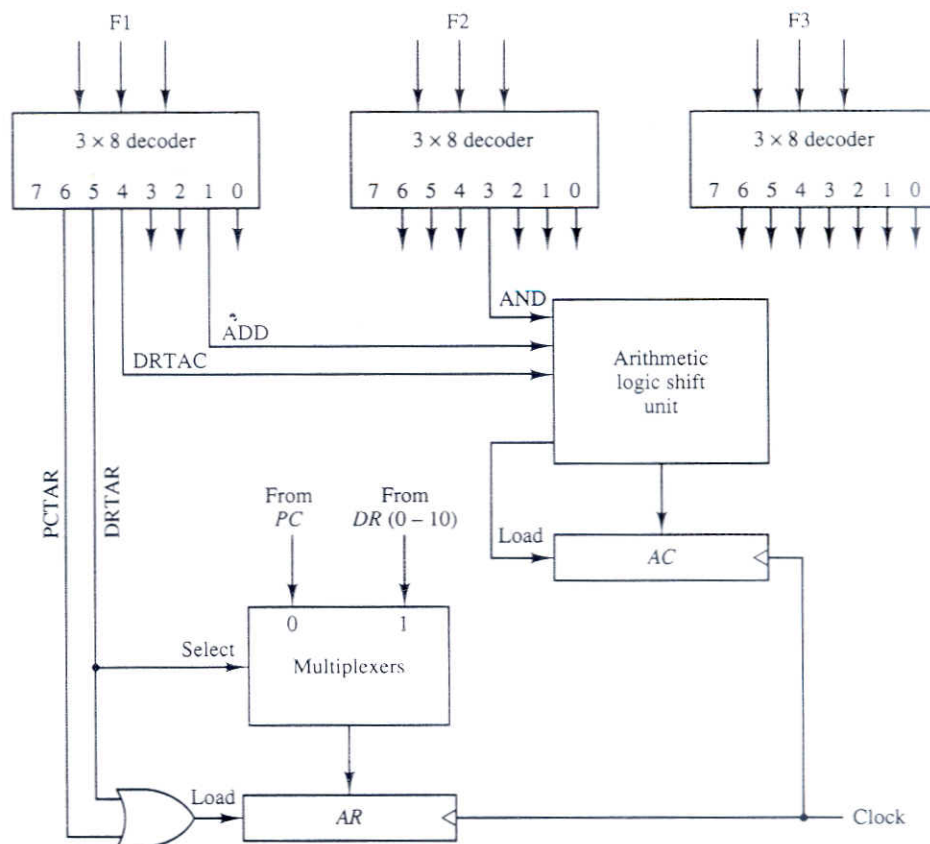


Figure 7-7 Decoding of microoperation fields.

TABLE 7-2 Symbolic Microprogram (Partial)

Label	Microoperations	CD	BR	AD
ADD:	ORG 0			
	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
BRANCH:	ORG 4			
	NOP	S	JMP	OVER
OVER:	NOP	U	JMP	FETCH
	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
STORE:	ORG 8			
	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH
EXCHANGE:	ORG 12			
	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
FETCH:	ORG 64			
	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	

TABLE 7-3 Binary Microprogram for Control Memory (Partial)

Micro Routine	Address		Binary Microinstruction					
	Decimal	Binary	F1	F2	F3	CD	BR	AD
ADD	0	0000000	000	000	000	01	01	1000011
	1	0000001	000	100	000	00	00	0000010
	2	0000010	001	000	000	00	00	1000000
	3	0000011	000	000	000	00	00	1000000
BRANCH	4	0000100	000	000	000	10	00	0000110
	5	0000101	000	000	000	00	00	1000000
	6	0000110	000	000	000	01	01	1000011
	7	0000111	000	000	110	00	00	1000000
STORE	8	0001000	000	000	000	01	01	1000011
	9	0001001	000	101	000	00	00	0001010
	10	0001010	111	000	000	00	00	1000000
EXCHANGE	11	0001011	000	000	000	00	00	1000000
	12	0001100	000	000	000	01	01	1000011
	13	0001101	001	000	000	00	00	0001110
	14	0001110	100	101	000	00	00	0001111
	15	0001111	111	000	000	00	00	1000000
FETCH	64	1000000	110	000	000	00	00	1000001
	65	1000001	000	100	101	00	00	1000010
	66	1000010	101	000	000	00	11	0000000
INDRCT	67	1000011	000	100	000	00	00	1000100
	68	1000100	101	000	000	00	10	0000000

TABLE 7-4 Input Logic Truth Table for Microprogram Sequencer

BR Field	Input			MUX 1		Load SBR
	I_1	I_0	T	S_1	S_0	L
0 0	0	0	0	0	0	0
0 0	0	0	1	0	1	0
0 1	0	1	0	0	0	0
0 1	0	1	1	0	1	1
1 0	1	0	×	1	0	0
1 1	1	1	×	1	1	0

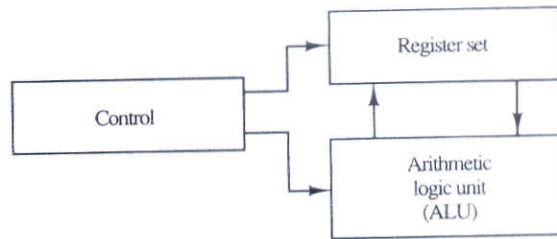


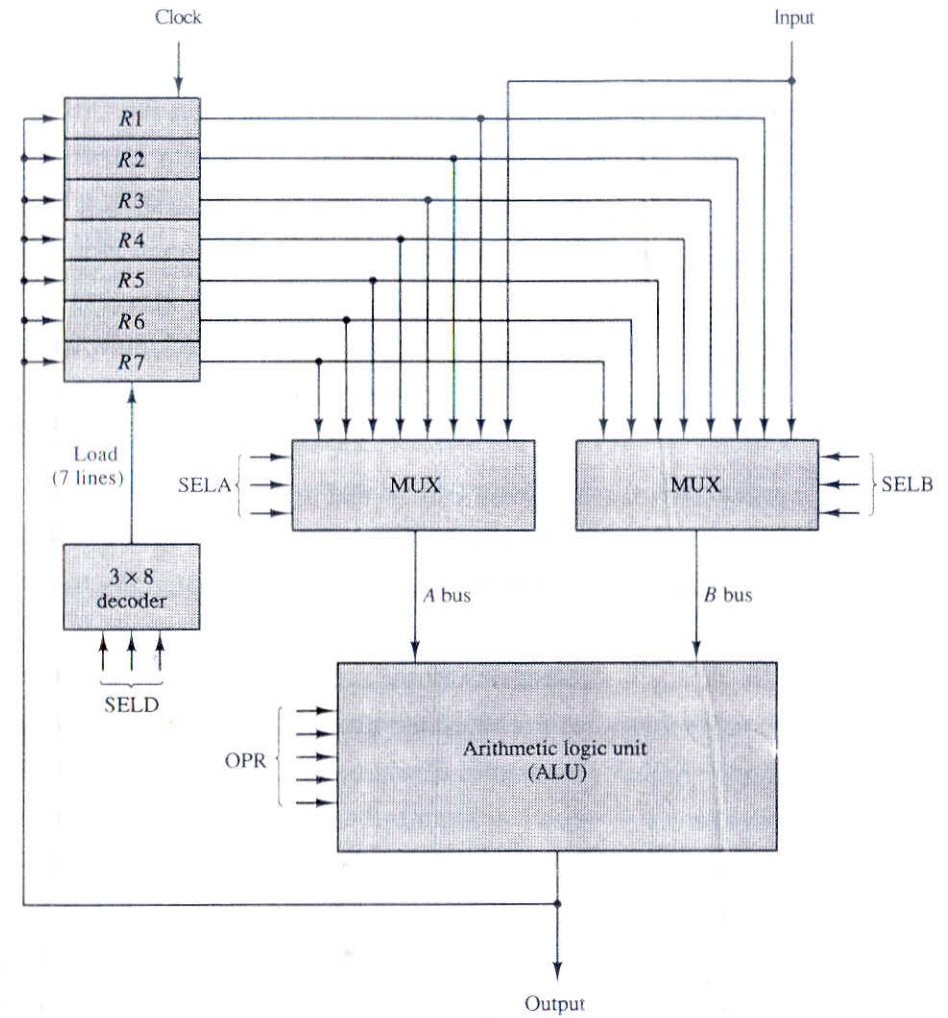
Figure 8-1 Major components of CPU.

TABLE 8-1 Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

TABLE 8-2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA



(a) Block diagram



(b) Control word

Figure 8-2 Register set with common ALU.

TABLE 8-3 Examples of Microoperations for the CPU

Microoperation	Symbolic Designation				Control Word
	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001 000 111 00000
$\text{Output} \leftarrow R2$	R2	—	None	TSFA	010 000 000 00000
$\text{Output} \leftarrow \text{Input}$	Input	—	None	TSFA	000 000 000 00000
$R4 \leftarrow \text{shl } R4$	R4	—	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

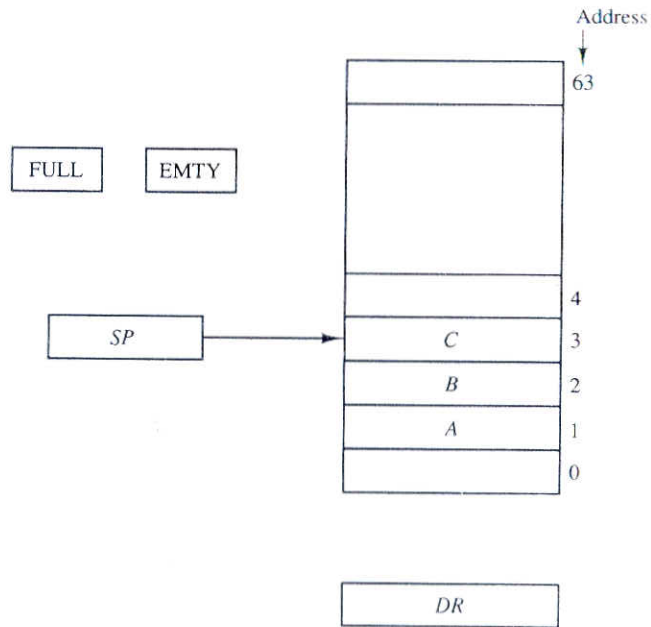


Figure 8-3 Block diagram of a 64-word stack.

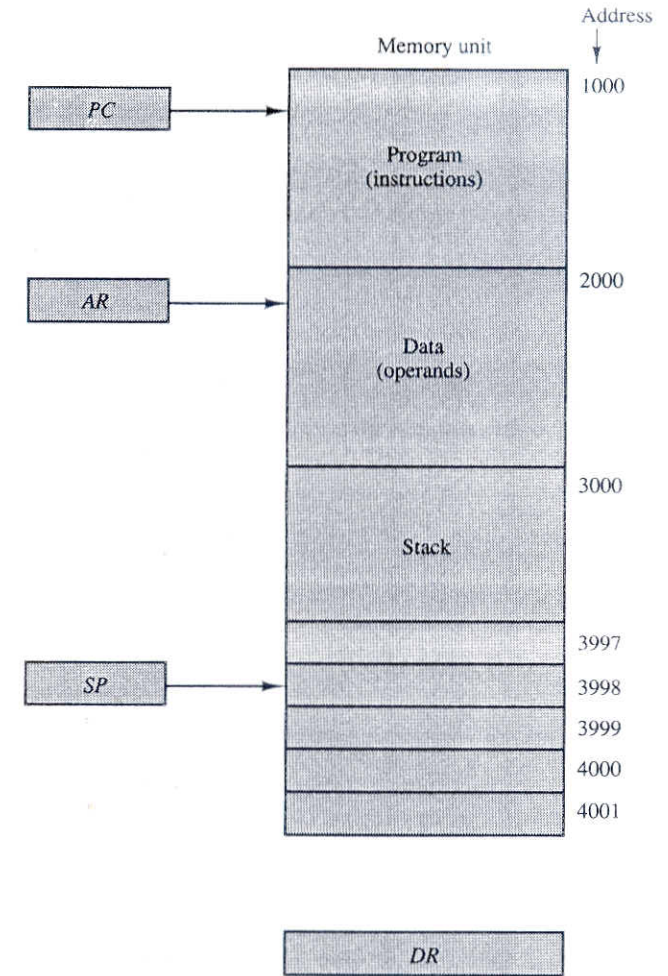


Figure 8-4 Computer memory with program, data, and stack segments.

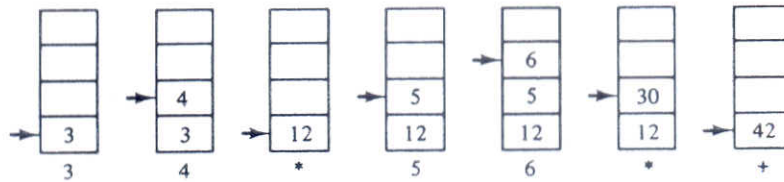
Figure 8-5 Stack operations to evaluate $3 \cdot 4 + 5 \cdot 6$.

Figure 8-6 Instruction format with mode field.

Opcode	Mode	Address
--------	------	---------

Address	Memory
200	Load to AC Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

Figure 8-7 Numerical example for addressing modes.

TABLE 8-4 Tabular List of Numerical Example

Addressing Mode	Effective Address	Content of AC
Direct address	500	800
Immediate operand	201	500
Indirect address	800	300
Relative address	702	325
Indexed address	600	900
Register	—	400
Register indirect	400	700
Autoincrement	400	700
Autodecrement	399	450

TABLE 8-5 Typical Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

TABLE 8-6 Eight Addressing Modes for the Load Instruction

Mode	Assembly Convention	Register Transfer
Direct address	LD ADR	$AC \leftarrow M[ADR]$
Indirect address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative address	LD \$ADR	$AC \leftarrow M[PC + ADR]$
Immediate operand	LD #NBR	$AC \leftarrow NBR$
Index addressing	LD ADR(X)	$AC \leftarrow M[ADR + XR]$
Register	LD R1	$AC \leftarrow R1$
Register indirect	LD (R1)	$AC \leftarrow M[R1]$
Autoincrement	LD (R1)+	$AC \leftarrow M[R1], R1 \leftarrow R1 + 1$

TABLE 8-7 Typical Arithmetic Instructions

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

TABLE 8-8 Typical Logical and Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

TABLE 8-9 Typical Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

TABLE 8-10 Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

TABLE 8-11 Conditional Branch Instructions

Mnemonic	Branch condition	Tested condition
BZ	Branch if zero	$Z = 1$
BNZ	Branch if not zero	$Z = 0$
BC	Branch if carry	$C = 1$
BNC	Branch if no carry	$C = 0$
BP	Branch if plus	$S = 0$
BM	Branch if minus	$S = 1$
BV	Branch if overflow	$V = 1$
BNV	Branch if no overflow	$V = 0$
<i>Unsigned compare conditions ($A - B$)</i>		
BHI	Branch if higher	$A > B$
BHE	Branch if higher or equal	$A \geq B$
BLO	Branch if lower	$A < B$
BLOE	Branch if lower or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$
<i>Signed compare conditions ($A - B$)</i>		
BGT	Branch if greater than	$A > B$
BGE	Branch if greater or equal	$A \geq B$
BLT	Branch if less than	$A < B$
BLE	Branch if less or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

TABLE 8-12 Instruction Set of Berkeley RISC I

Opcode	Operands	Register Transfer	Description
Data manipulation instructions			
ADD	R_s, S_2, R_d	$R_d \leftarrow R_s + S_2$	Integer add
ADDC	R_s, S_2, R_d	$R_d \leftarrow R_s + S_2 + \text{carry}$	Add with carry
SUB	R_s, S_2, R_d	$R_d \leftarrow R_s - S_2$	Integer subtract
SUBC	R_s, S_2, R_d	$R_d \leftarrow R_s - S_2 - \text{carry}$	Subtract with carry
SUBR	R_s, S_2, R_d	$R_d \leftarrow S_2 - R_s$	Subtract reverse
SUBCR	R_s, S_2, R_d	$R_d \leftarrow S_2 - R_s - \text{carry}$	Subtract with carry
AND	R_s, S_2, R_d	$R_d \leftarrow R_s \wedge S_2$	AND
OR	R_s, S_2, R_d	$R_d \leftarrow R_s \vee S_2$	OR
XOR	R_s, S_2, R_d	$R_d \leftarrow R_s \oplus S_2$	Exclusive-OR
SLL	R_s, S_2, R_d	$R_d \leftarrow R_s \text{ shifted by } S_2$	Shift-left
SRL	R_s, S_2, R_d	$R_d \leftarrow R_s \text{ shifted by } S_2$	Shift-right logical
SRA	R_s, S_2, R_d	$R_d \leftarrow R_s \text{ shifted by } S_2$	Shift-right arithmetic
Data transfer instructions			
LDL	$(R_s)S_2, R_d$	$R_d \leftarrow M[R_s + S_2]$	Load long
LDSU	$(R_s)S_2, R_d$	$R_d \leftarrow M[R_s + S_2]$	Load short unsigned
LDSS	$(R_s)S_2, R_d$	$R_d \leftarrow M[R_s + S_2]$	Load short signed
LDBU	$(R_s)S_2, R_d$	$R_d \leftarrow M[R_s + S_2]$	Load byte unsigned
LDBS	$(R_s)S_2, R_d$	$R_d \leftarrow M[R_s + S_2]$	Load byte signed
LDHI	R_d, Y	$R_d \leftarrow Y$	Load immediate high
STL	$R_d, (R_s)S_2$	$M[R_s + S_2] \leftarrow R_d$	Store long
STS	$R_d, (R_s)S_2$	$M[R_s + S_2] \leftarrow R_d$	Store short
STB	$R_d, (R_s)S_2$	$M[R_s + S_2] \leftarrow R_d$	Store byte
GETPSW	R_d	$R_d \leftarrow PSW$	Load status word
PUTPSW	R_d	$PSW \leftarrow R_d$	Set status word
Program control instructions			
JMP	COND, $S_2(R_s)$	$PC \leftarrow R_s + S_2$	Conditional jump
JMPR	COND, Y	$PC \leftarrow PC + Y$	Jump relative
CALL	$R_d, S_2(R_s)$	$R_d \leftarrow PC$ $PC \leftarrow R_s + S_2$	Call subroutine
CALLR	R_d, Y	$CWP \leftarrow CWP - 1$ $R_d \leftarrow PC$	and change window
		$PC \leftarrow PC + Y$ $CWP \leftarrow CWP - 1$	Call relative and change window
RET	R_d, S_2	$PC \leftarrow R_d + S_2$ $CWP \leftarrow CWP + 1$	Return and change window
CALLINT	R_d	$R_d \leftarrow PC$ $CWP \leftarrow CWP - 1$	Disable interrupts
RETINT	R_d, S_2	$PC \leftarrow R_d + S_2$ $CWP \leftarrow CWP + 1$	Enable interrupts
GTLPC	R_d	$R_d \leftarrow PC$	Get last PC

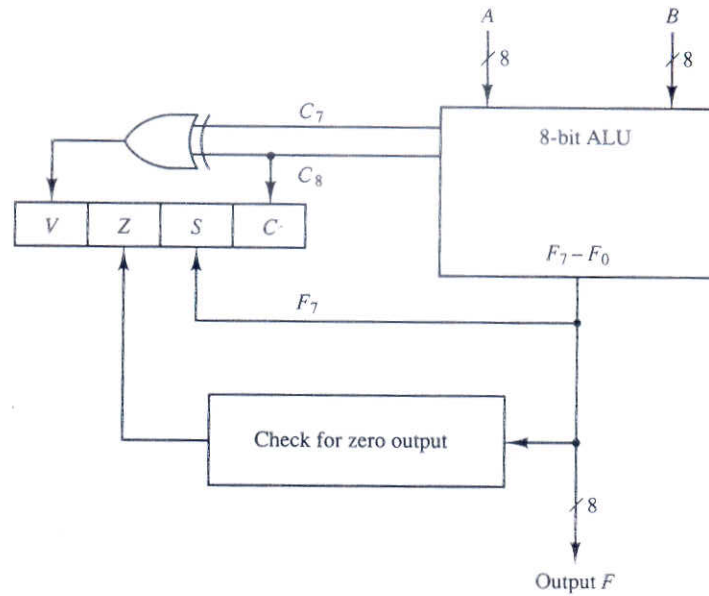


Figure 8-8 Status register bits.

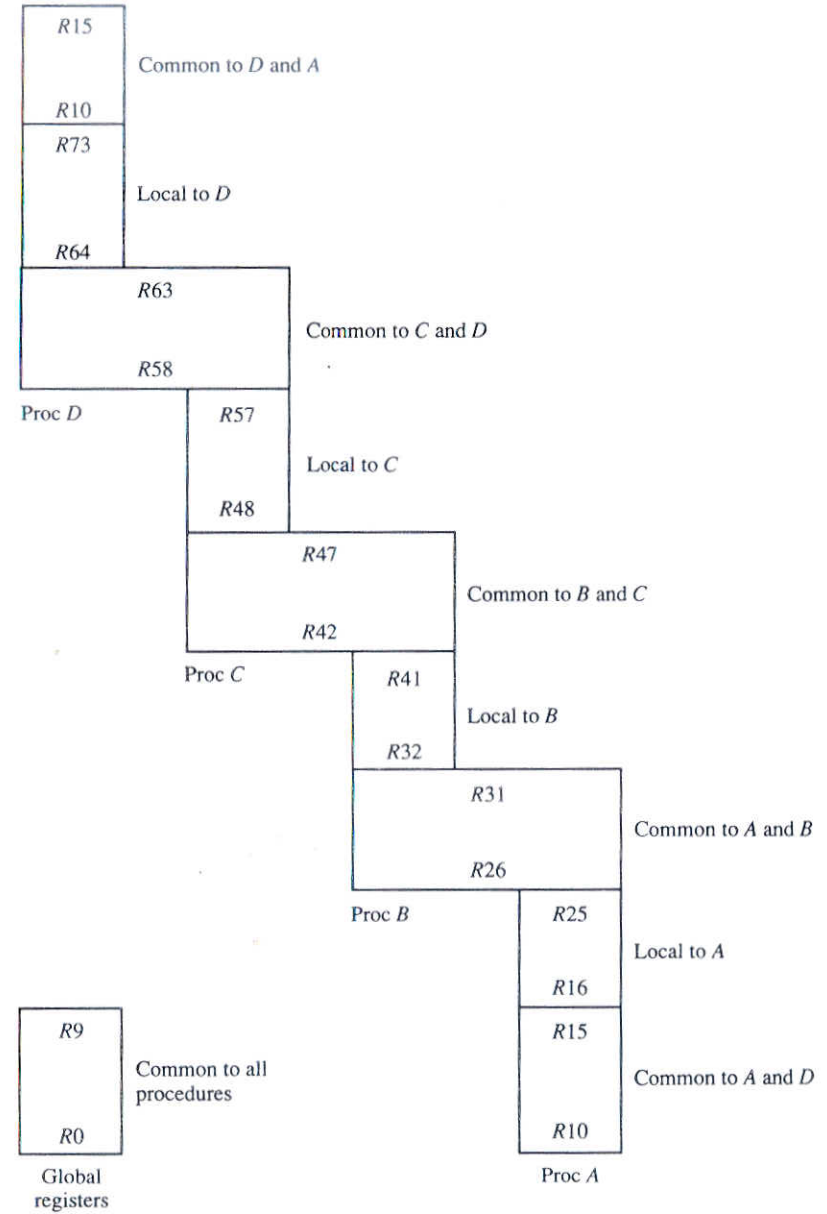
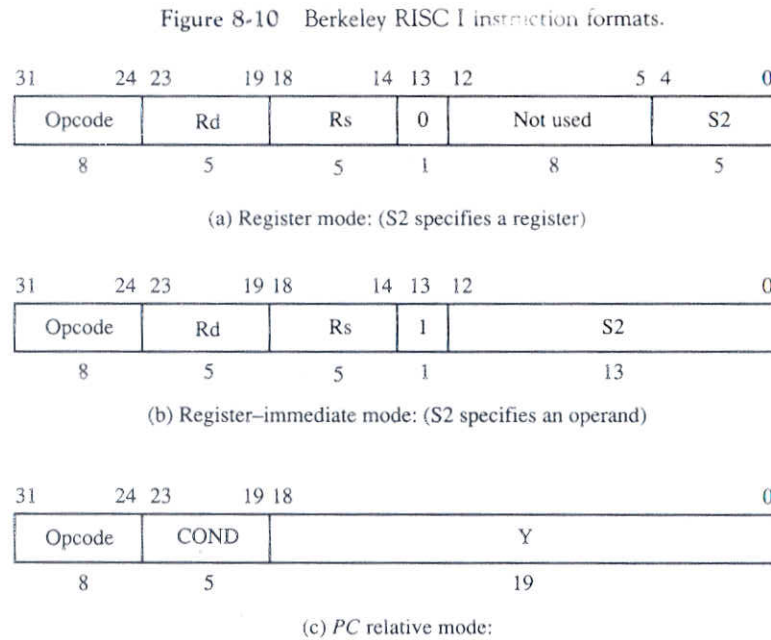


Figure 8-9 Overlapped register windows.

Figure 9-1 Processor with multiple functional units.

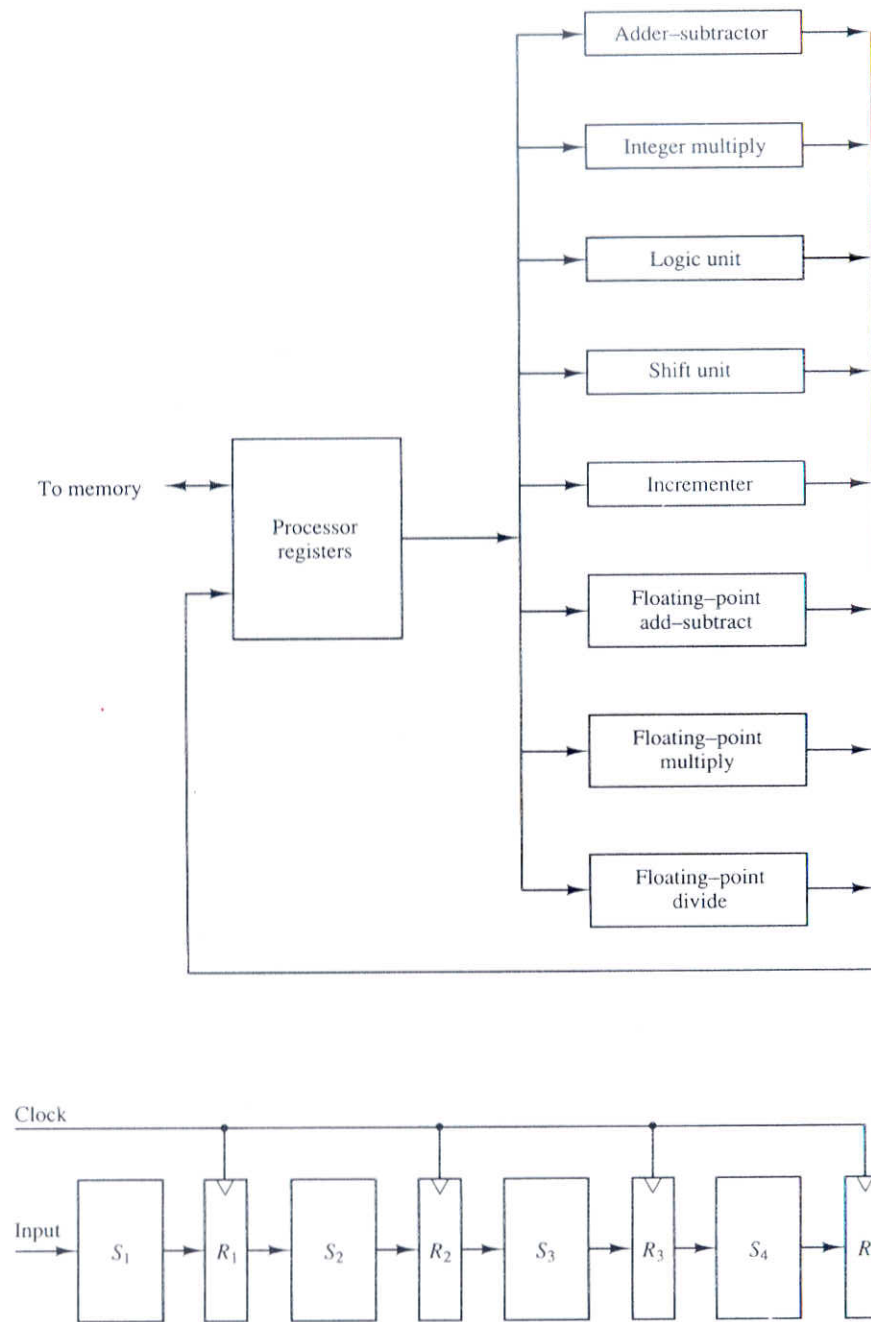


Figure 9-3 Four-segment pipeline.

TABLE 9-1 Content of Registers in Pipeline Example

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

Figure 9-2 Example of pipeline processing.

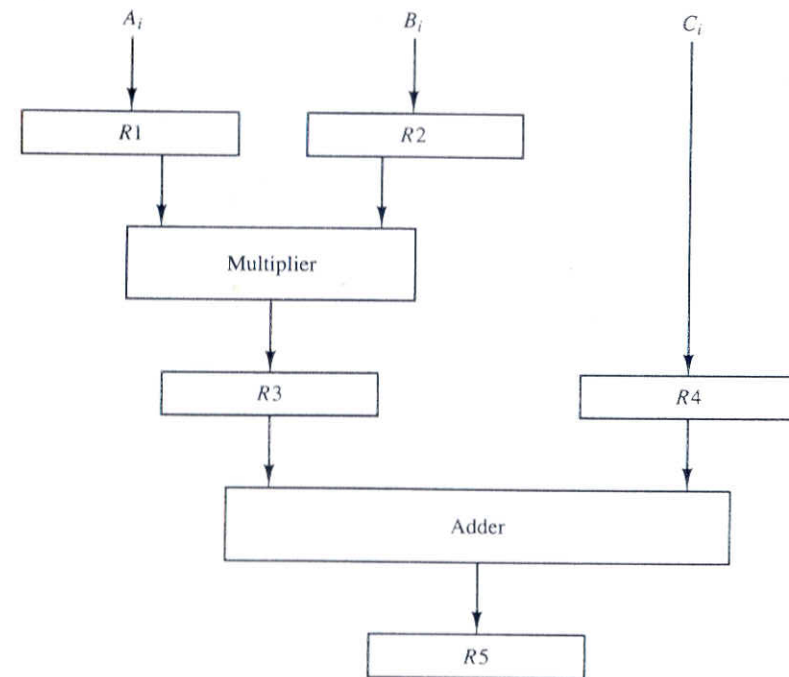


Figure 9-4 Space-time diagram for pipeline.

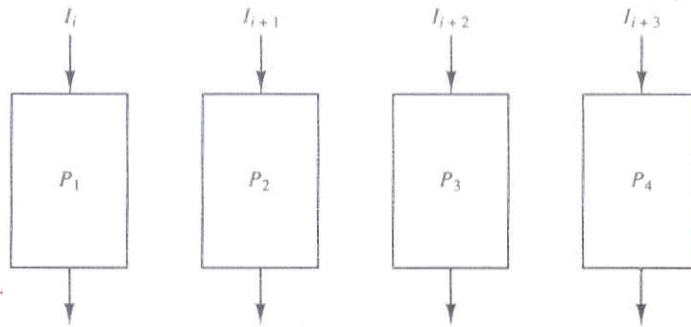
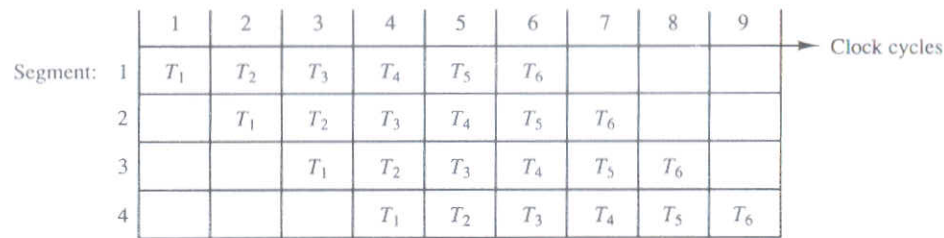


Figure 9-5 Multiple functional units in parallel.

Step:		1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction:	1	FI	DA	FO	EX									
	2		FI	DA	FO	EX								
(Branch)	3			FI	DA	FO	EX							
	4				FI	-	-	FI	DA	FO	EX			
	5					-	-	-	FI	DA	FO	EX		
	6									FI	DA	FO	EX	
	7										FI	DA	FO	EX

Figure 9-8 Timing of instruction pipeline.

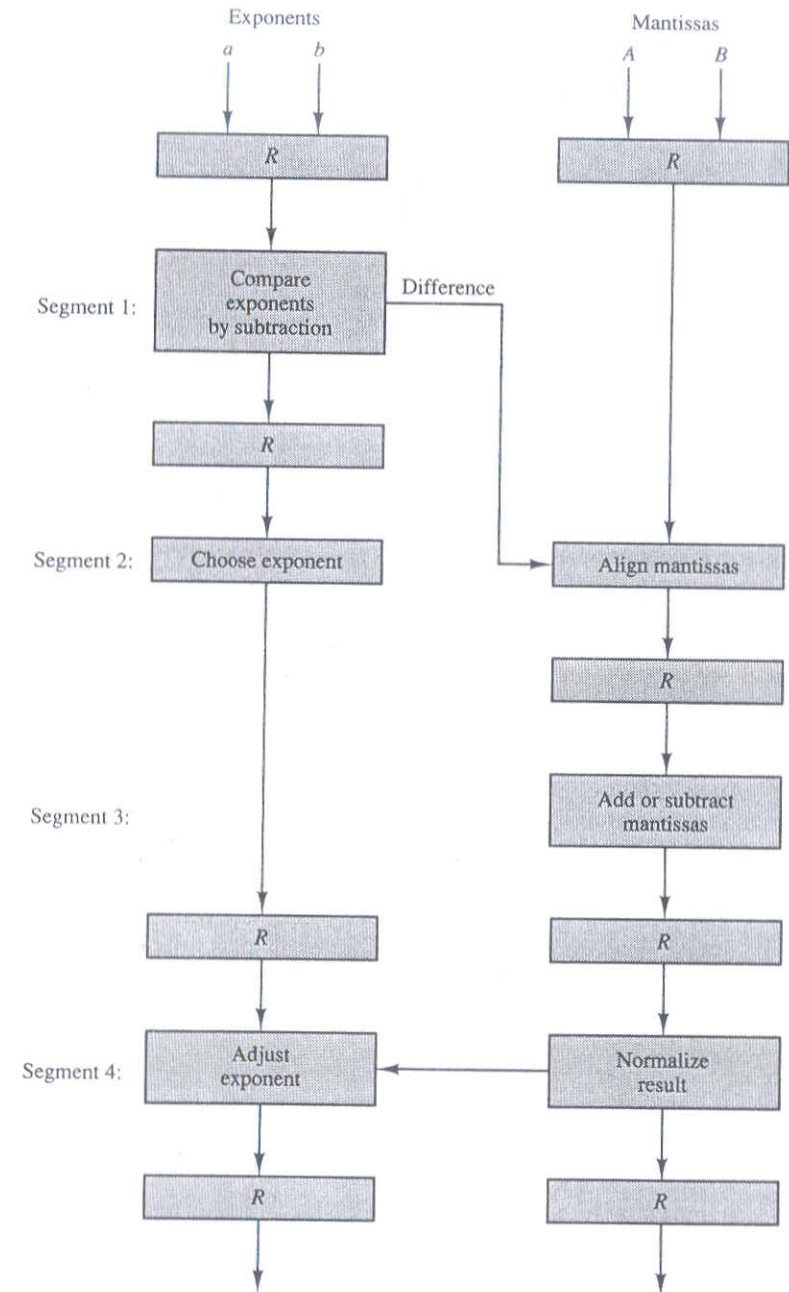


Figure 9-6 Pipeline for floating-point addition and subtraction.

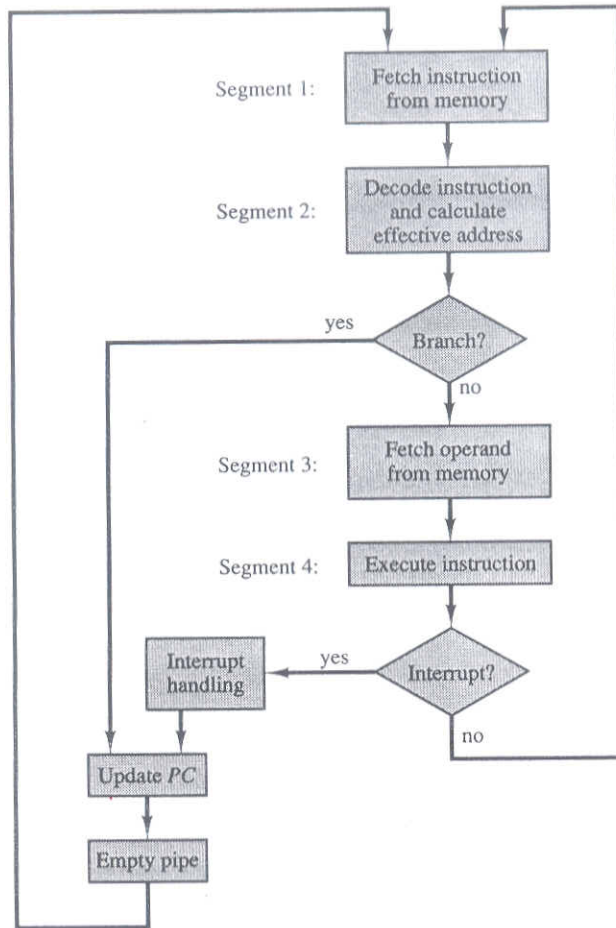


Figure 9-7 Four-segment CPU pipeline.

Clock cycles:	1	2	3	4	5	6
1. Load R1	I	A	E			
2. Load R2		I	A	E		
3. Add R1 + R2			I	A	E	
4. Store R3				I	A	E

(a) Pipeline timing with data conflict

Clock cycle:	1	2	3	4	5	6	7
1. Load R1	I	A	E				
2. Load R2		I	A	E			
3. No-operation			I	A	E		
4. Add R1 + R2				I	A	E	
5. Store R3					I	A	E

(b) Pipeline timing with delayed load

Figure 9-9 Three-segment pipeline timing.

Clock cycles:	1	2	3	4	5	6	7	8	9	10
1. Load	I	A	E							
2. Increment		I	A	E						
3. Add			I	A	E					
4. Subtract				I	A	E				
5. Branch to X					I	A	E			
6. No-operation						I	A	E		
7. No-operation							I	A	E	
8. Instruction in X								I	A	E

(a) Using no-operation instructions

Clock cycles:	1	2	3	4	5	6	7	8
1. Load	I	A	E					
2. Increment		I	A	E				
3. Branch to X			I	A	E			
4. Add				I	A	E		
5. Subtract					I	A	E	
6. Instruction in X						I	A	E

(b) Rearranging the instructions

Figure 9-10 Example of delayed branch.

Figure 9-11 Instruction format for vector processor.

Operation code	Base address source 1	Base address source 2	Base address destination	Vector length
----------------	-----------------------	-----------------------	--------------------------	---------------

Figure 9-12 Pipeline for calculating an inner product.

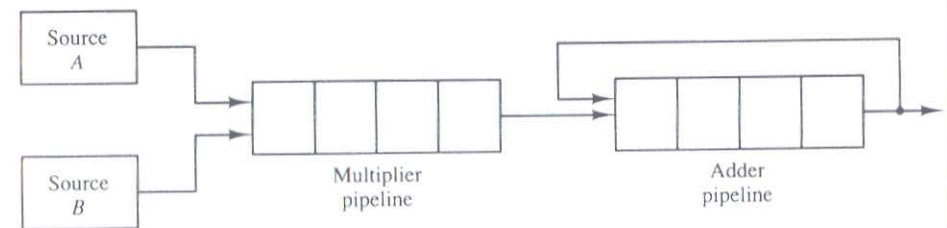


Figure 9-13 Multiple module memory organization.

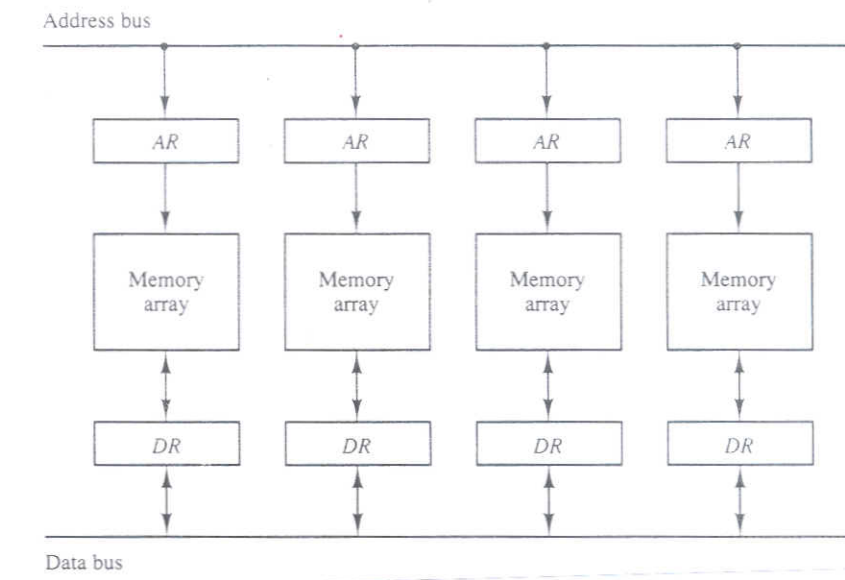


Figure 9-14 Attached array processor with host computer.

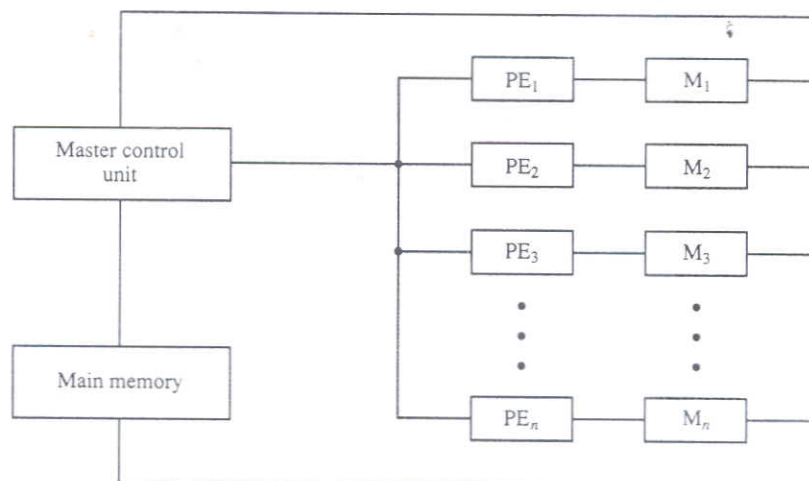
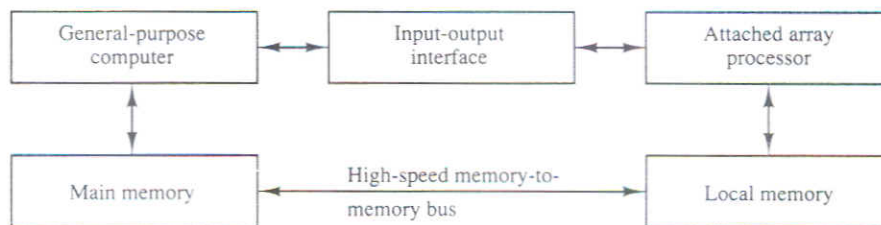


Figure 9-15 SIMD array processor organization.

TABLE 10-1 Addition and Subtraction of Signed-Magnitude Numbers

Operation	Add Magnitudes	Subtract Magnitudes		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

TABLE 10-2 Numerical Example for Binary Multiplier

Multiplicand $B = 10111$	E	A	Q	SC
Multiplier in Q	0	00000	10011	101
$Q_n = 1$; add B		<u>10111</u>		
First partial product	0	10111		
Shift right EAQ	0	01011	11001	100
$Q_n = 1$; add B		<u>10111</u>		
Second partial product	1	00010		
Shift right EAQ	0	10001	01100	011
$Q_n = 0$; shift right EAQ	0	01000	10110	010
$Q_n = 0$; shift right EAQ	0	00100	01011	001
$Q_n = 1$; add B		<u>10111</u>		
Fifth partial product	0	11011		
Shift right EAQ	0	01101	10101	000
Final product in $AQ = 0110110101$				

TABLE 10-3 Example of Multiplication with Booth Algorithm

$Q_n Q_{n+1}$	$BR = 10111$ $\overline{BR} + 1 = 01001$	AC	QR	Q_{n+1}	SC
	Initial	00000	10011	0	101
1 0	Subtract BR	<u>01001</u>			
		01001			
	ashr	00100	11001	1	100
1 1	ashr	00010	01100	1	011
0 1	Add BR	<u>10111</u>			
		11001			
	ashr	11100	10110	0	010
0 0	ashr	11110	01011	0	001
1 0	Subtract BR	<u>01001</u>			
		00111			
	ashr	00011	10101	1	000

TABLE 10-4 Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

TABLE 10-5 Decimal Arithmetic Microoperation Symbols

Symbolic Designation	Description
$A \leftarrow A + B$	Add decimal numbers and transfer sum into A
$\overline{B}$	9's complement of B
$A \leftarrow A + \overline{B} + 1$	Content of A plus 10's complement of B into A
$Q_L \leftarrow Q_L + 1$	Increment BCD number in Q_L
dshr A	Decimal shift-right register A
dshl A	Decimal shift-left register A

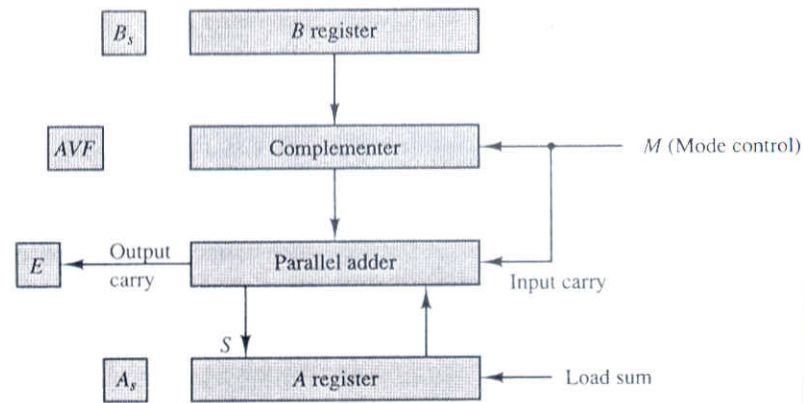


Figure 10-1 Hardware for signed-magnitude addition and subtraction.

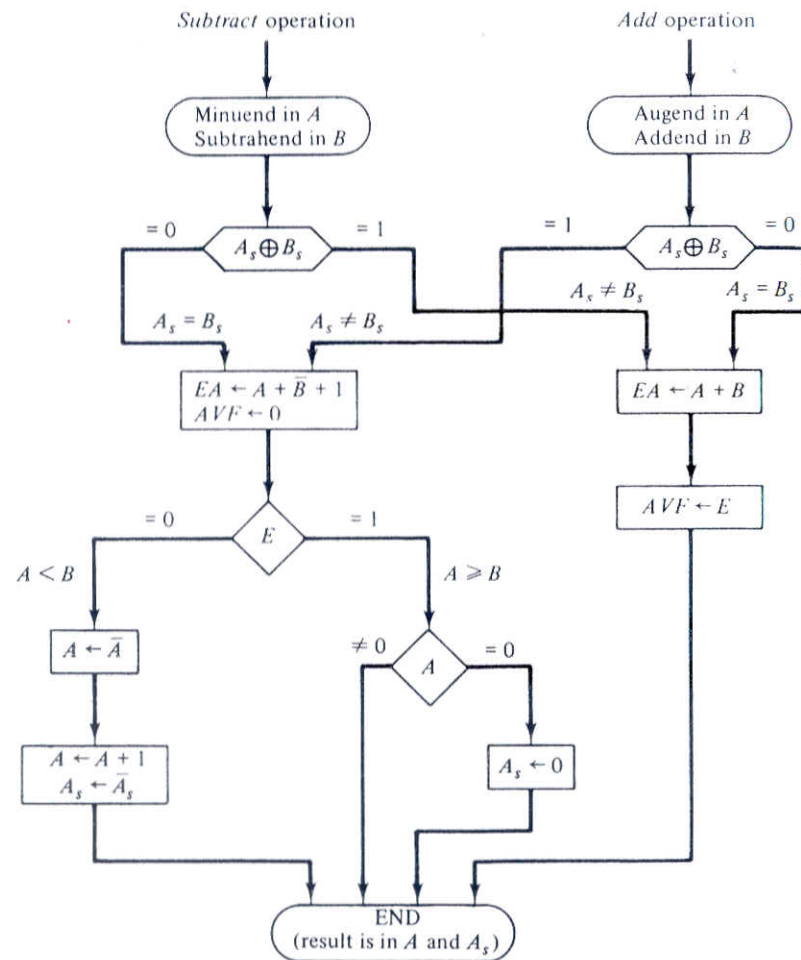


Figure 10-2 Flowchart for add and subtract operations.

Figure 10-3 Hardware for signed-2's complement addition and subtraction.

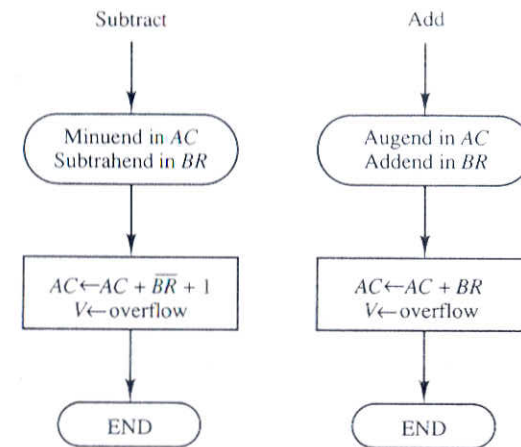
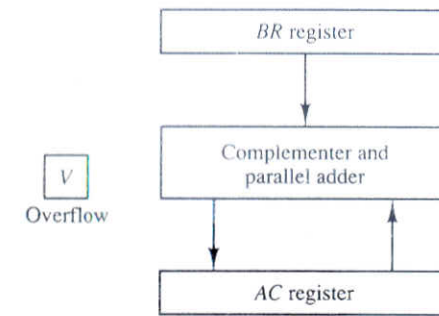


Figure 10-4 Algorithm for adding and subtracting numbers in signed-2's complement representation.

Figure 10-5 Hardware for multiply operation.

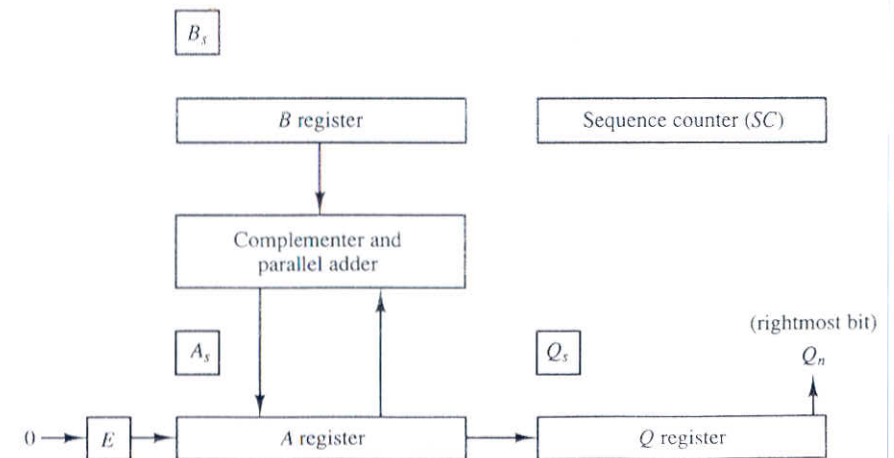


Figure 10-6 Flowchart for multiply operation.

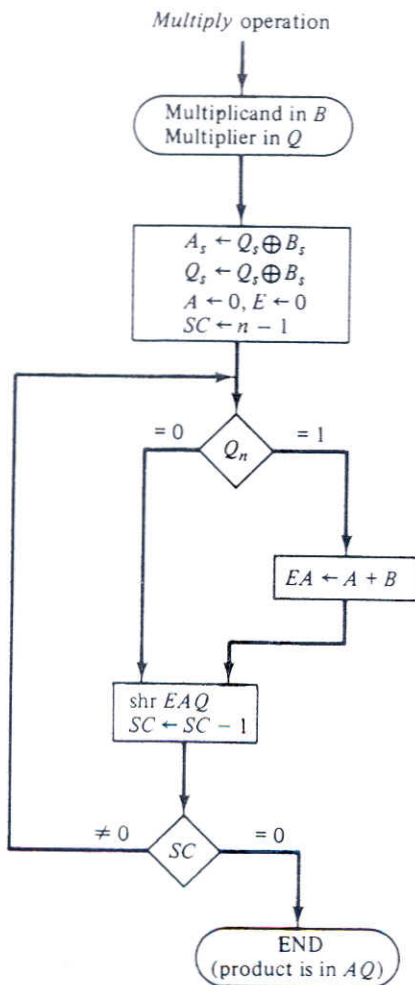


Figure 10-9 2-bit by 2-bit array multiplier.

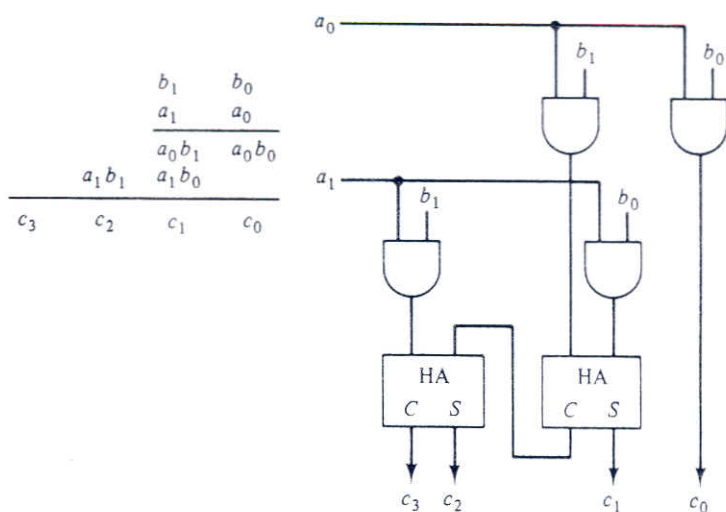


Figure 10-7 Hardware for Booth algorithm.

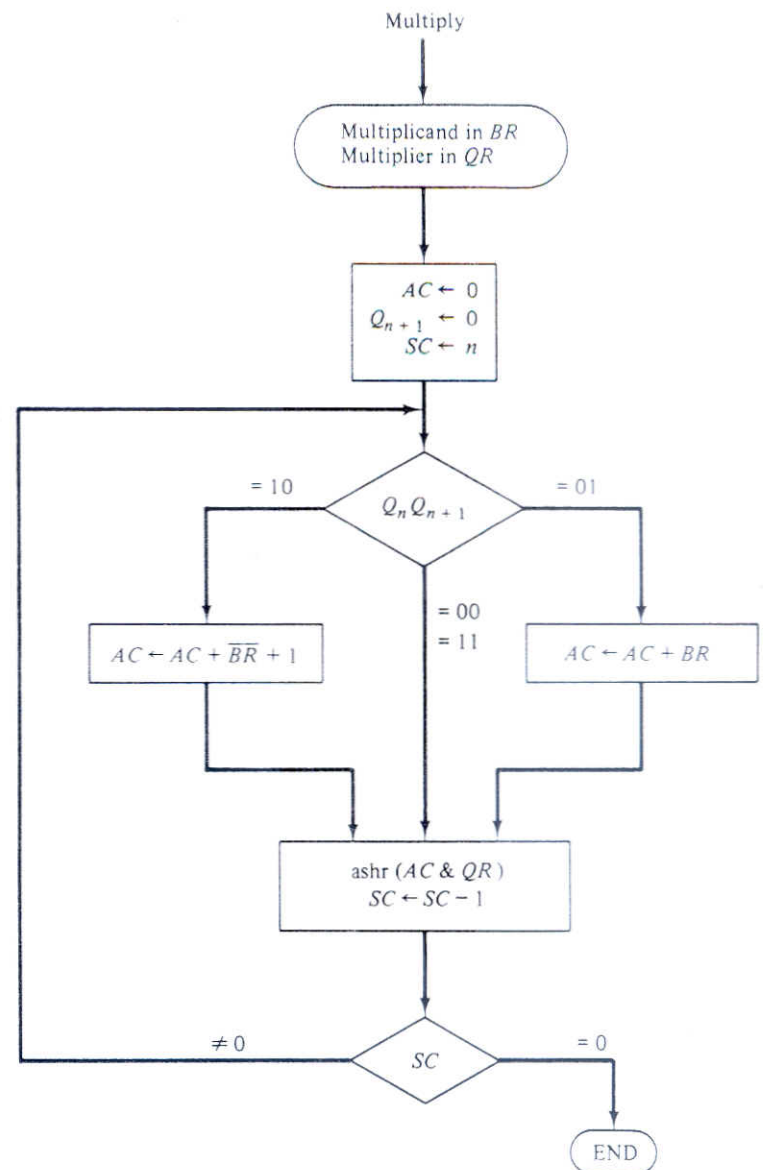
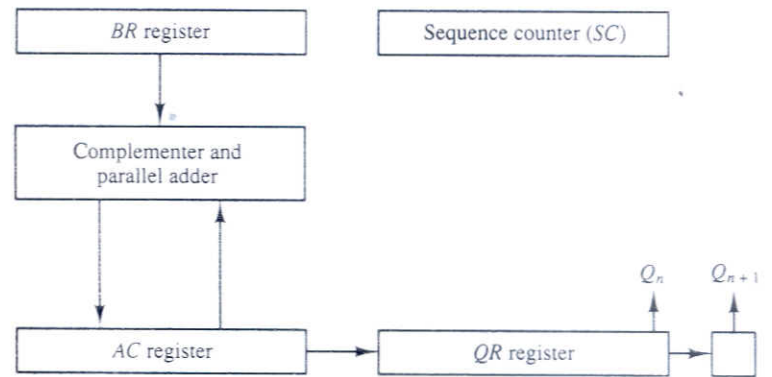


Figure 10-8 Booth algorithm for multiplication of signed-2's complement numbers.

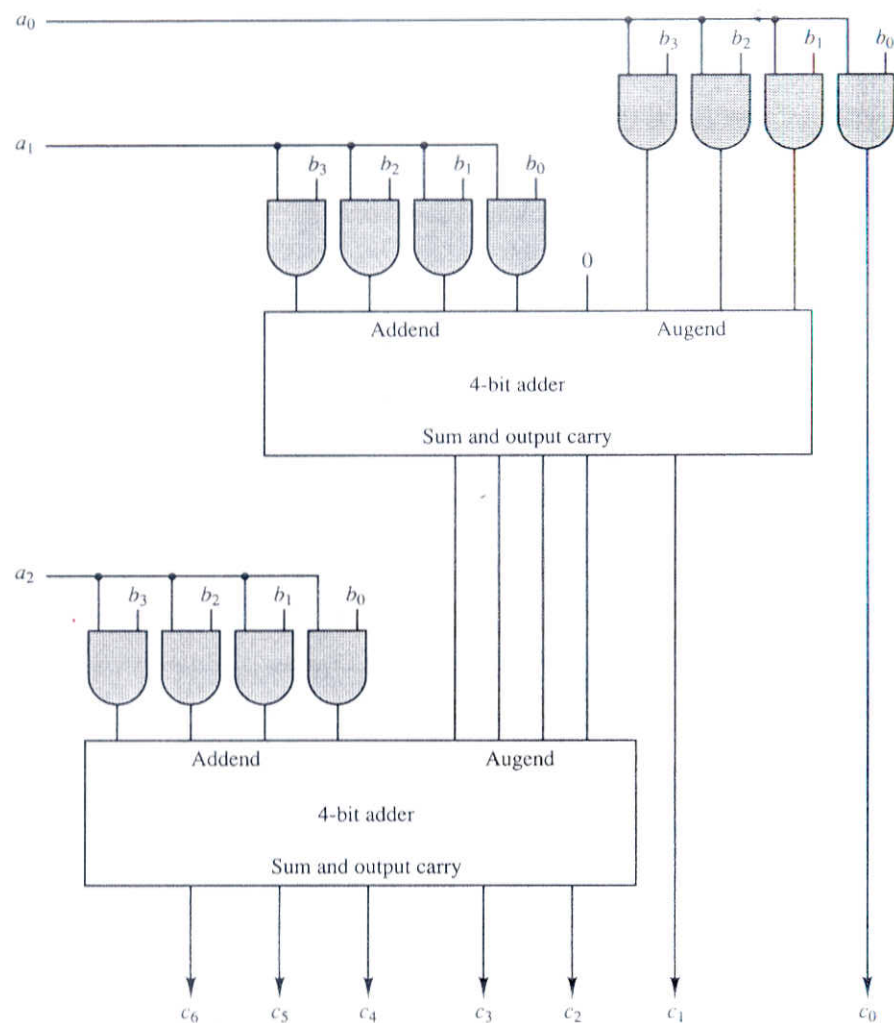


Figure 10-10 4-bit by 3-bit array multiplier.

Figure 10-11 Example of binary division.

Divisor:	11010	Quotient = Q
B = 10001	011100000	Dividend = A
	01110	5 bits of A < B, quotient has 5 bits
	011100	6 bits of A ≥ B
	-10001	Shift right B and subtract; enter 1 in Q
	-010110	7 bits of remainder ≥ B
	--10001	Shift right B and subtract; enter 1 in Q
	--001010	Remainder < B; enter 0 in Q; shift right B
	---010100	Remainder ≥ B
	----10001	Shift right B and subtract; enter 1 in Q
	----000110	Remainder < B; enter 0 in Q
	-----00110	Final remainder

Divisor B = 10001,

 $\bar{B} + 1 = 01111$

	E	A	Q	SC
Dividend.		01110	00000	
shl EAQ	0	11100	00000	5
add $\bar{B} + 1$		01111		
E = 1	1	01011		
Set $Q_n = 1$	1	01011	00001	4
shl EAQ	0	10110	00010	
Add $\bar{B} + 1$		01111		
E = 1	1	00101		
Set $Q_n = 1$	1	00101	00011	3
shl EAQ	0	01010	00110	
Add $\bar{B} + 1$		01111		
E = 0; leave $Q_n = 0$	0	11001	00110	
Add B		10001		
Restore remainder	1	01010		2
shl EAQ	0	10100	01100	
Add $\bar{B} + 1$		01111		
E = 1	1	00011		
Set $Q_n = 1$	1	00011	01101	1
shl EAQ	0	00110	11010	
Add $\bar{B} + 1$		01111		
E = 0; leave $Q_n = 0$	0	10101	11010	
Add B		10001		
Restore remainder	1	00110	11010	0
Neglect E				
Remainder in A:		00110		
Quotient in Q:			11010	

Figure 10-12 Example of binary division with digital hardware.

Figure 10-13 Flowchart for divide operation.

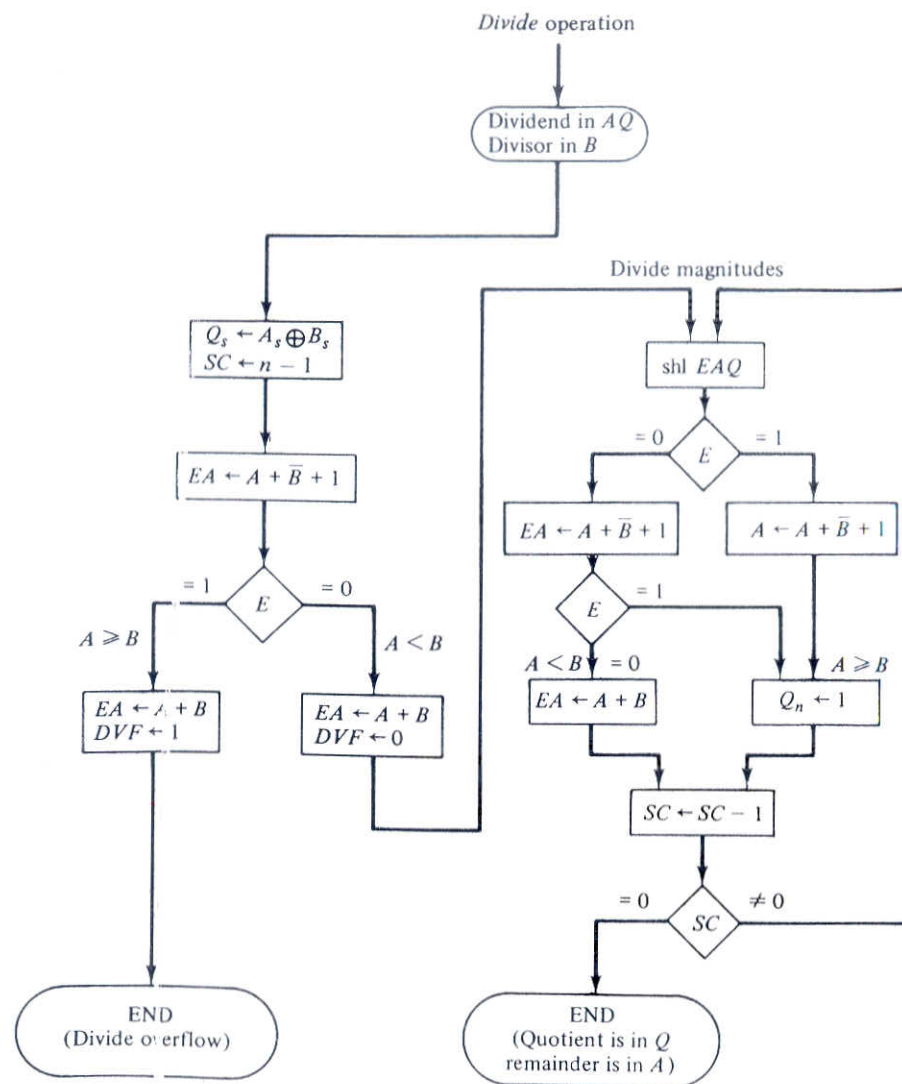


Figure 10-14 Registers for floating-point arithmetic operations.

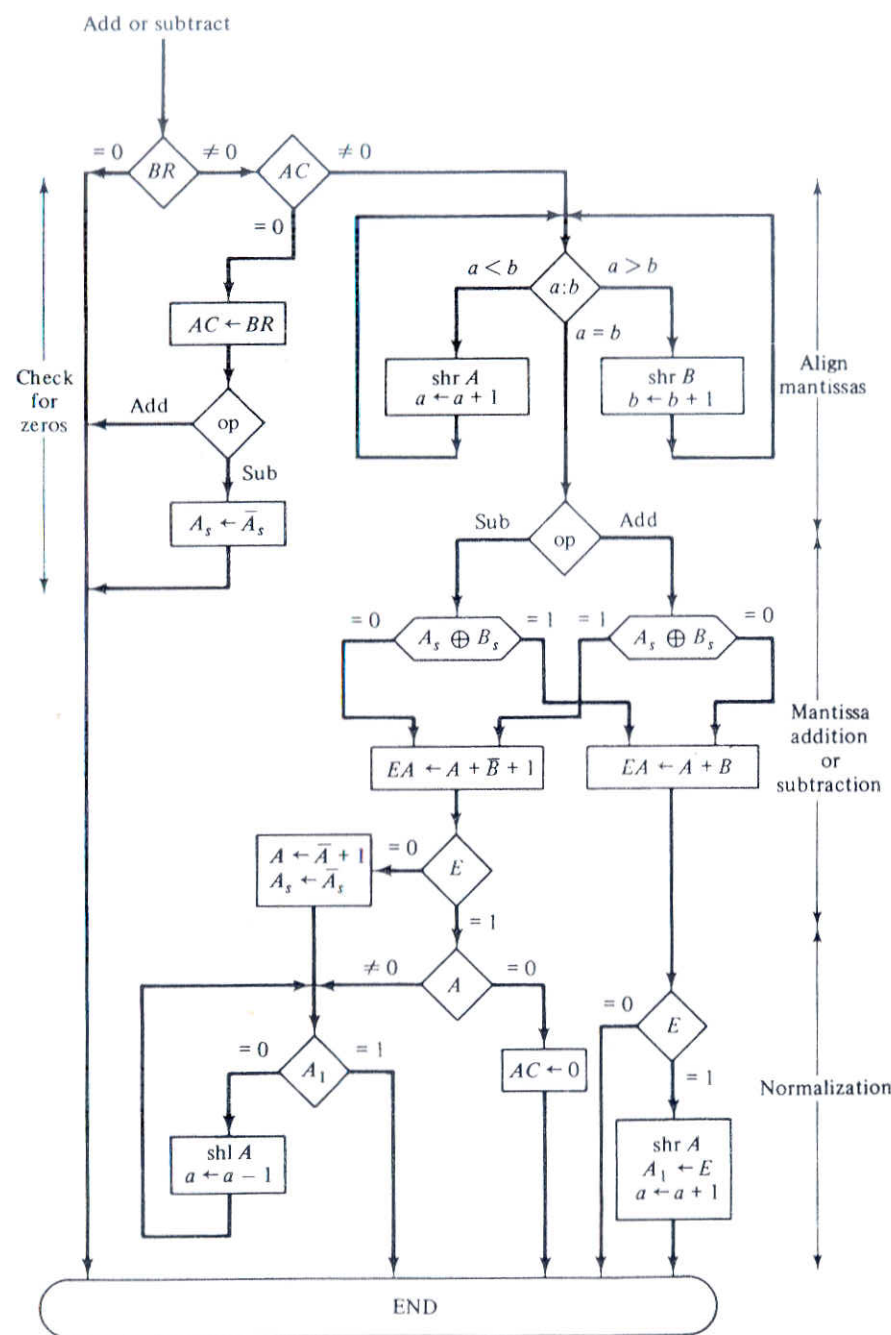
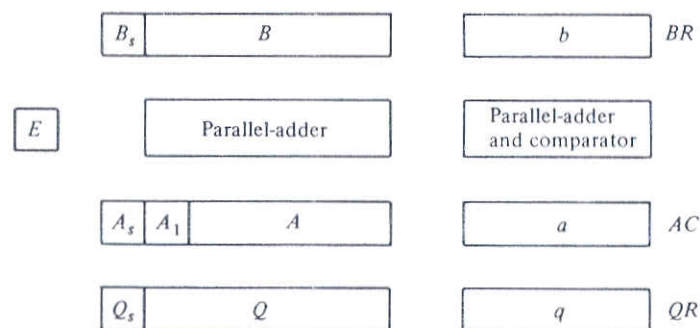


Figure 10-15 Addition and subtraction of floating-point numbers.

Figure 10-16 Multiplication of floating-point numbers.

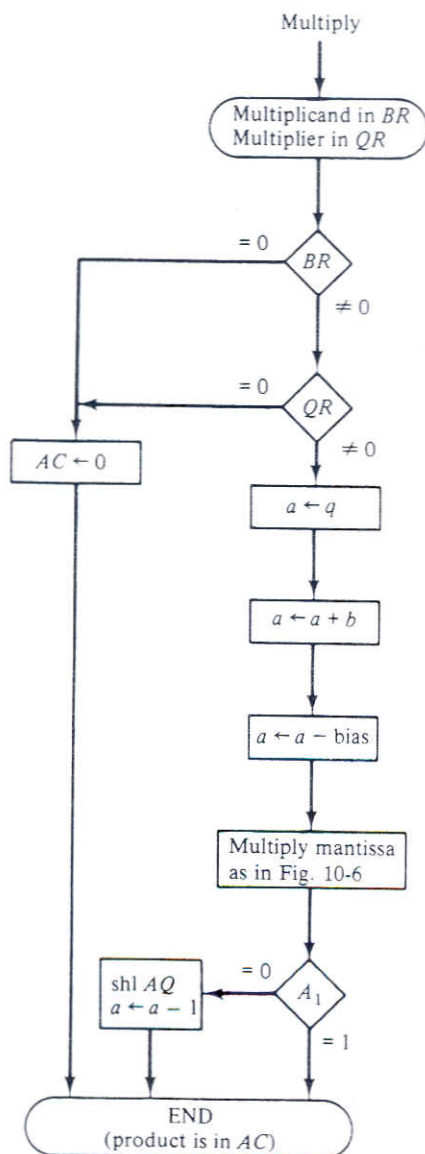


Figure 10-18 Block diagram of BCD adder.

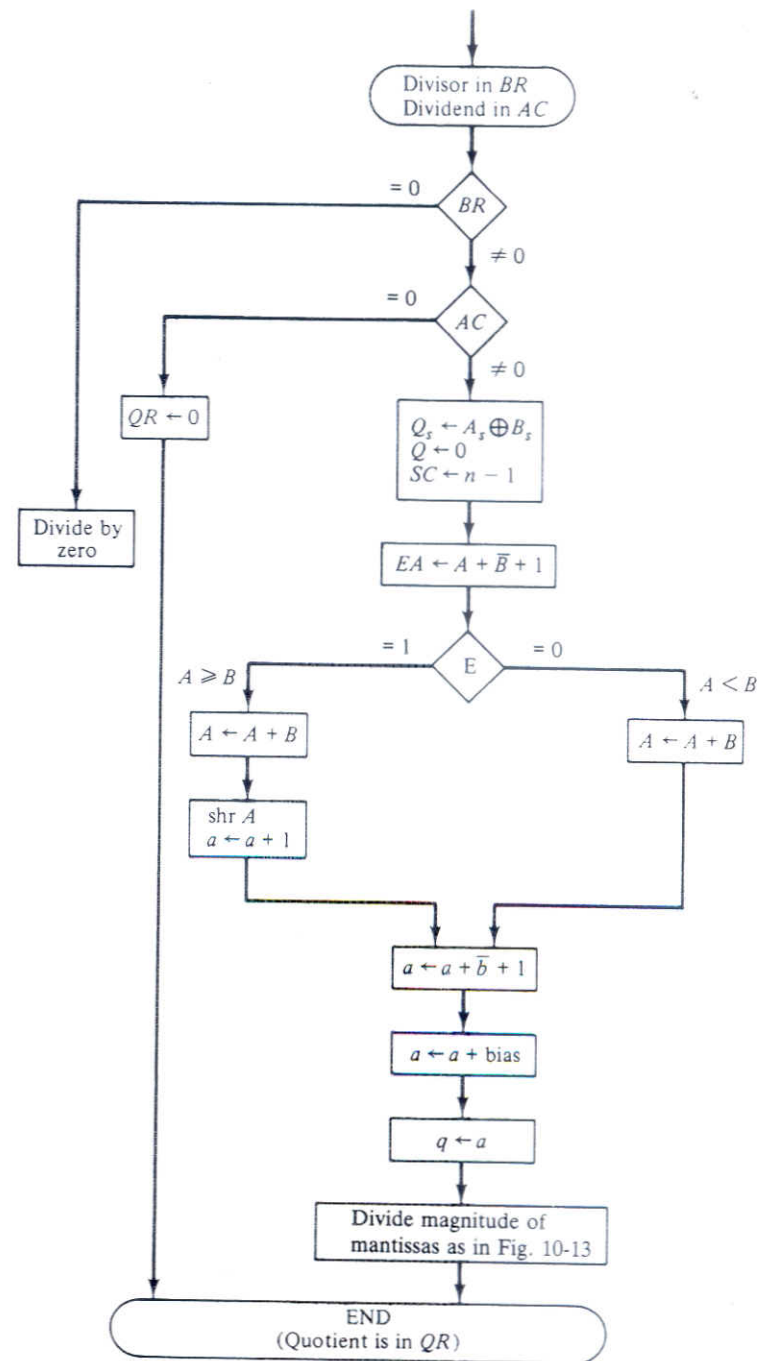
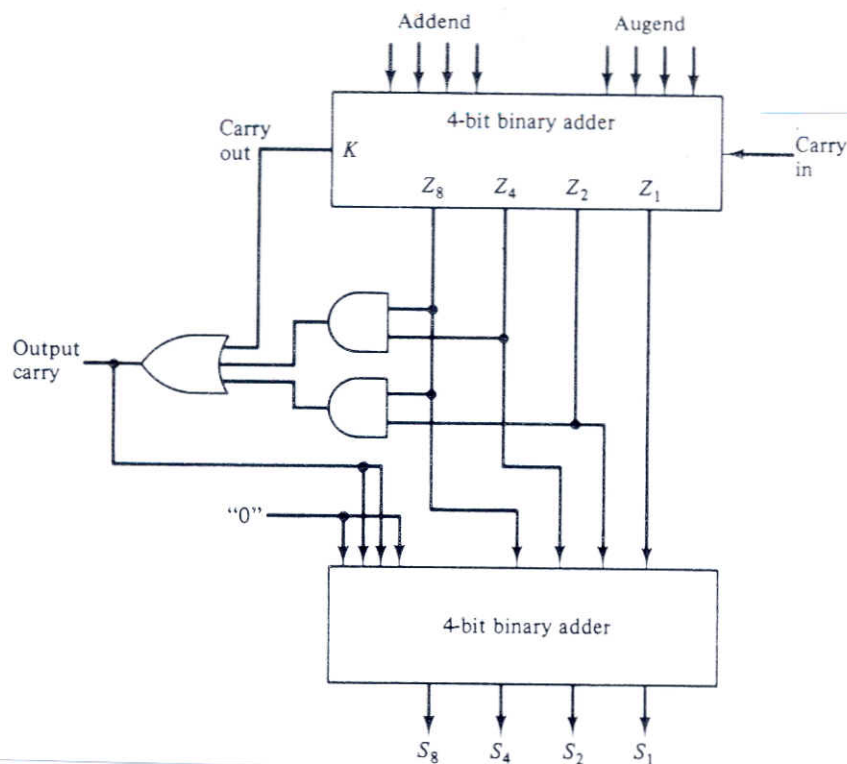


Figure 10-17 Division of floating-point numbers.

Figure 10-19 One stage of a decimal arithmetic unit.

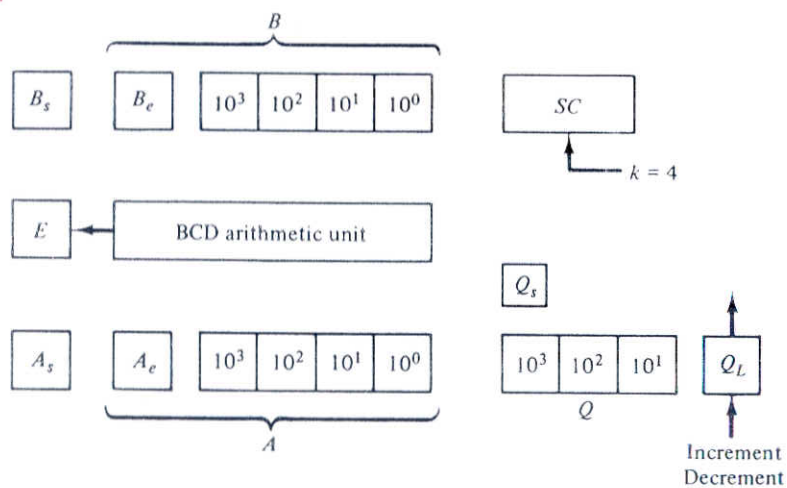
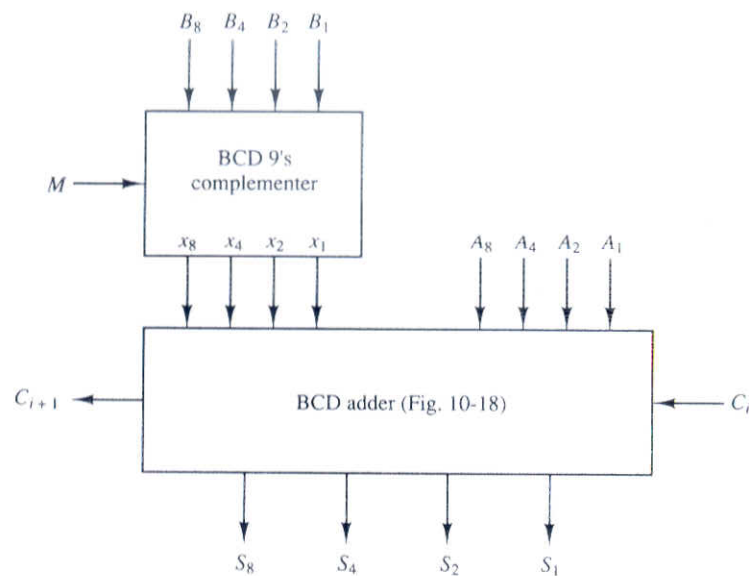
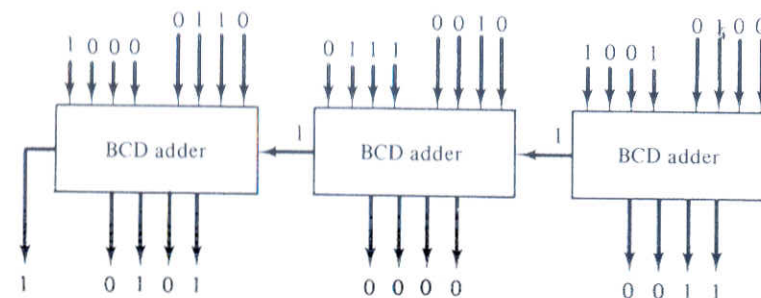
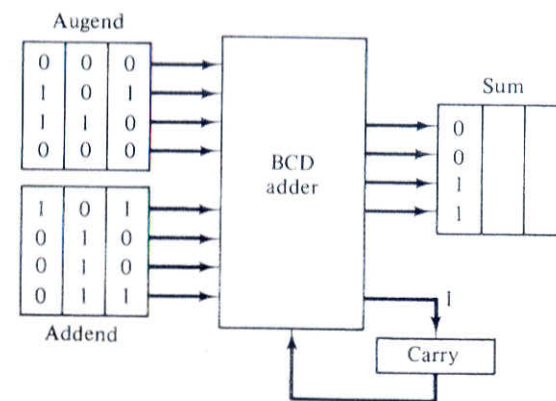
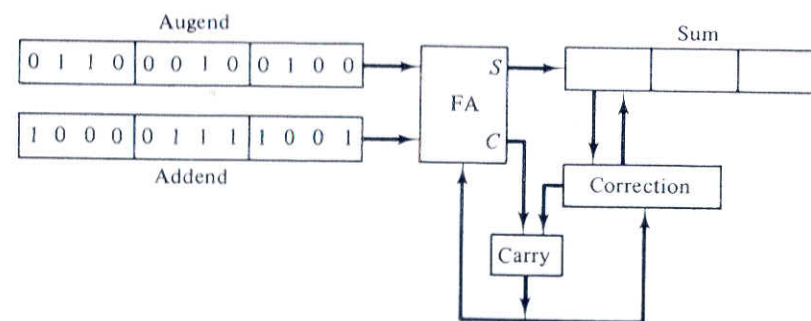


Figure 10-21 Registers for decimal arithmetic multiplication and division.

(a) Parallel decimal addition: $624 + 879 = 1503$ 

(b) Digit-serial, bit-parallel decimal addition



(c) All serial decimal addition

Figure 10-20 Three ways of adding decimal numbers.

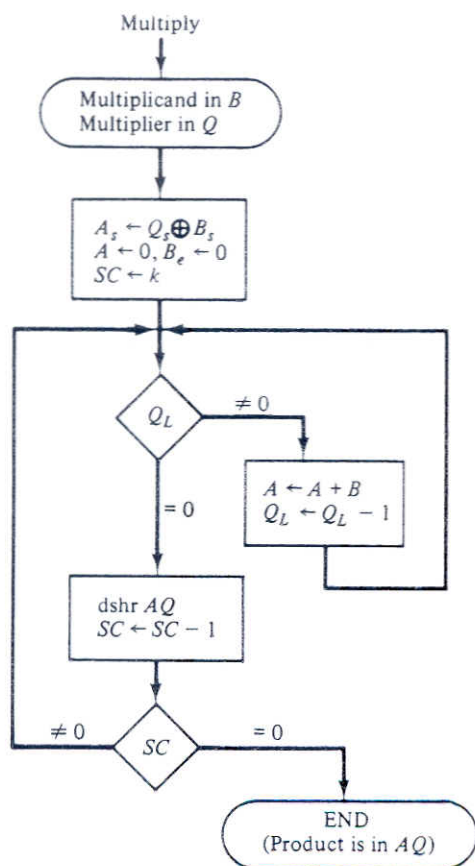


Figure 10-22 Flowchart for decimal multiplication.

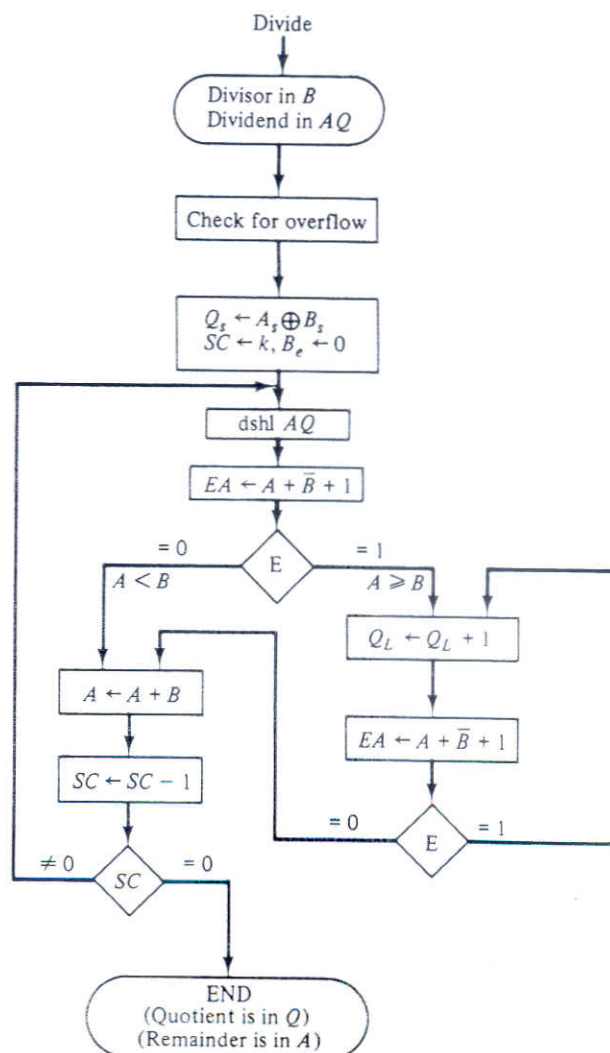
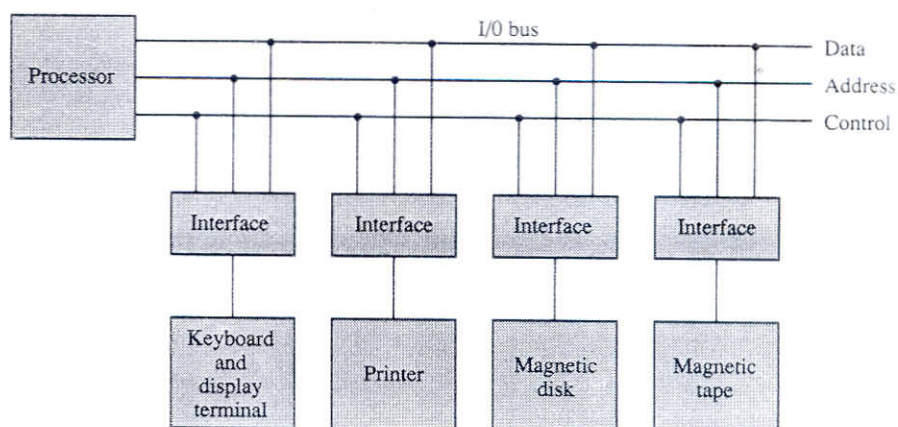
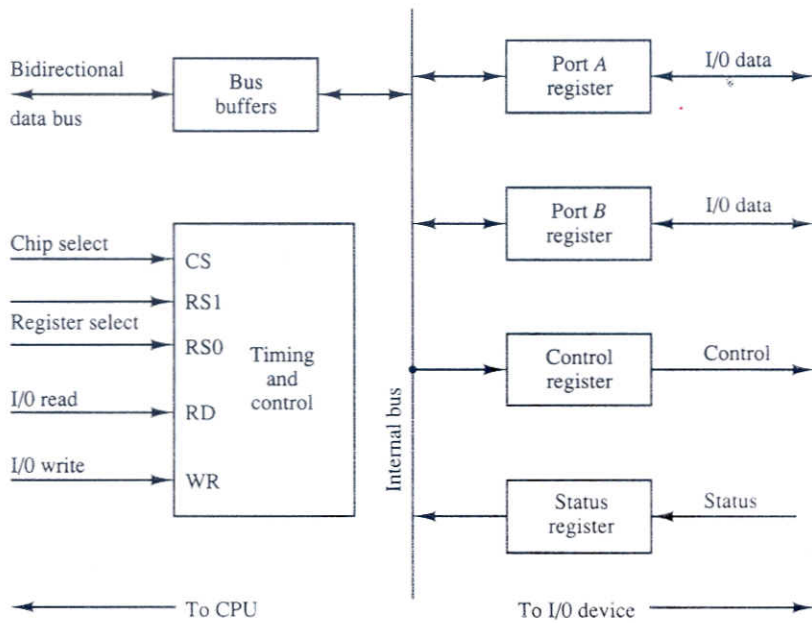


Figure 10-23 Flowchart for decimal division.

Figure 11-1 Connection of I/O bus to input-output devices.





CS	RS1	RS0	Register selected
0	x	x	None: data bus in high-impedance
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

Figure 11-2 Example of I/O interface unit.

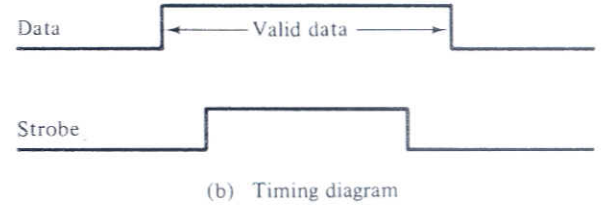
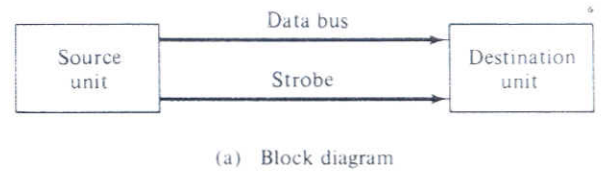


Figure 11-3 Source-initiated strobe for data transfer.

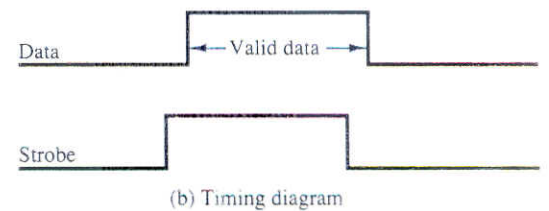
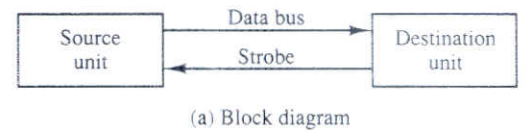


Figure 11-4 Destination-initiated strobe for data transfer.

Figure 11-5 Source-initiated transfer using handshaking.

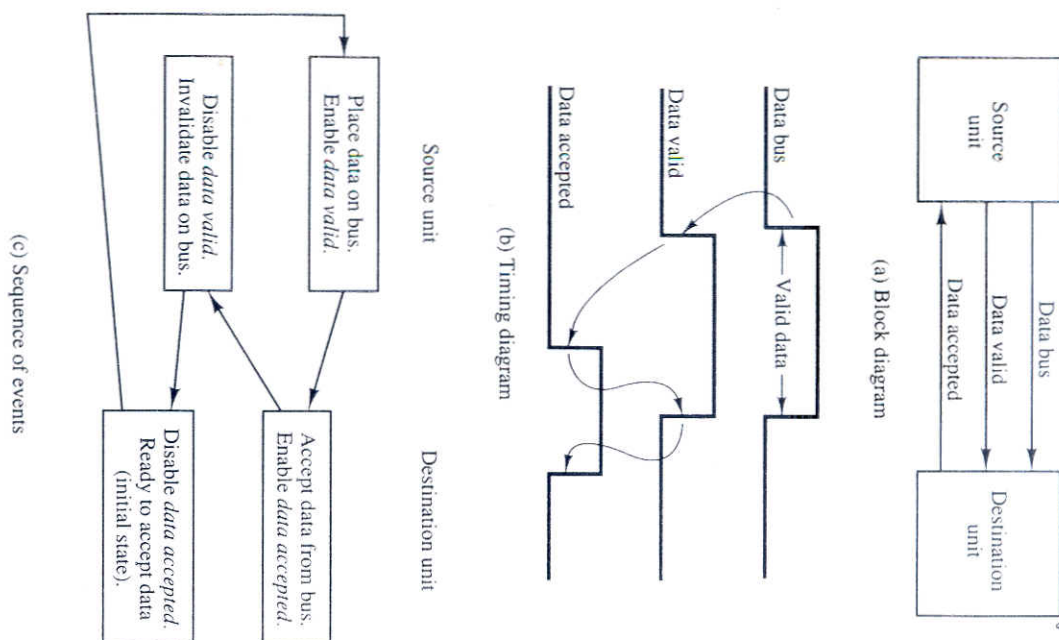


Figure 11-6 Destination-initiated transfer using handshaking.

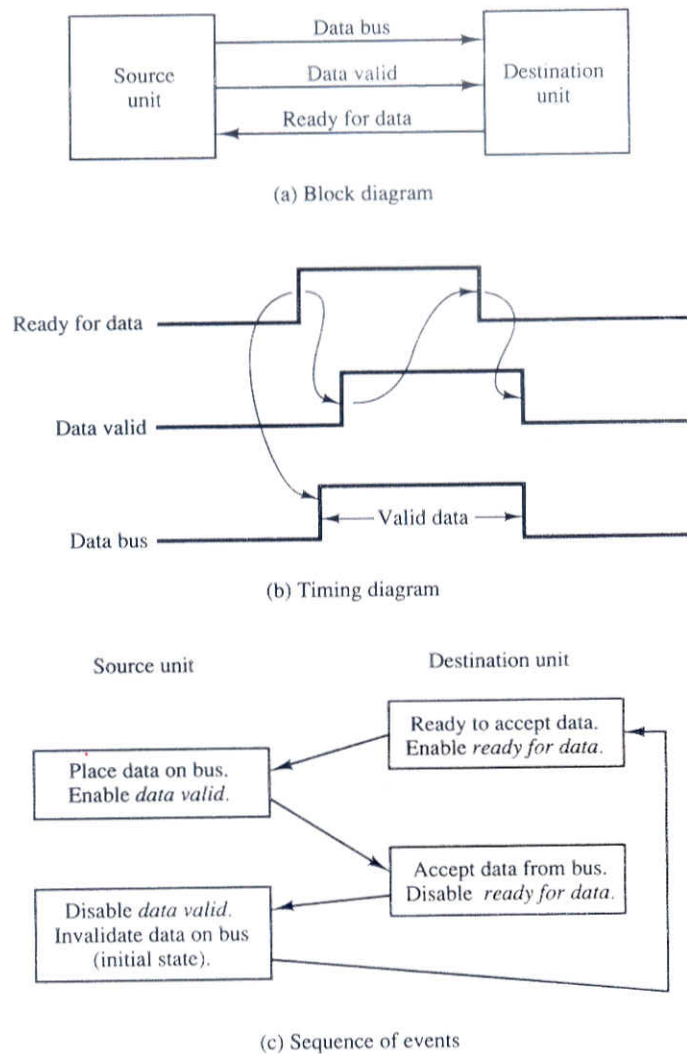
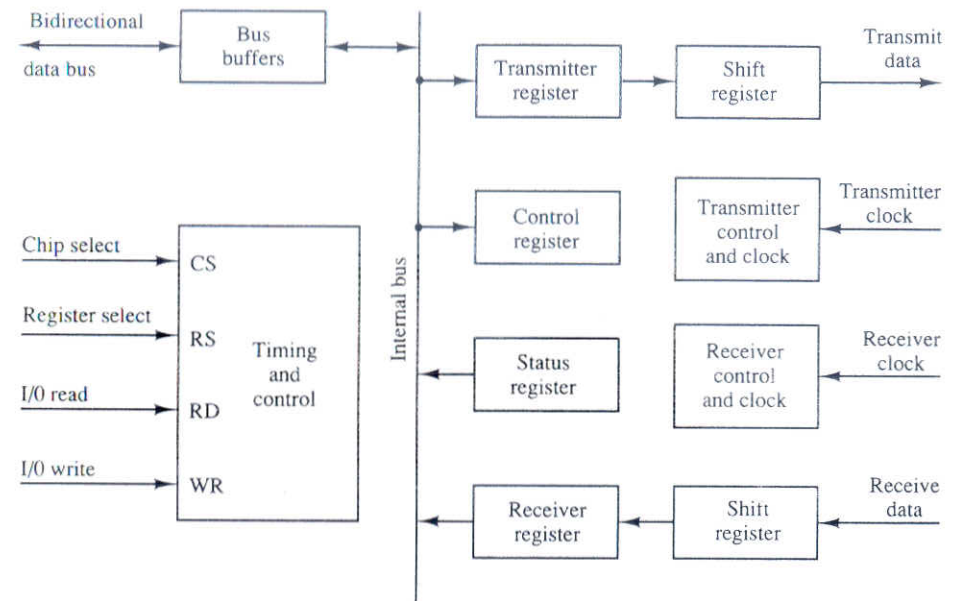
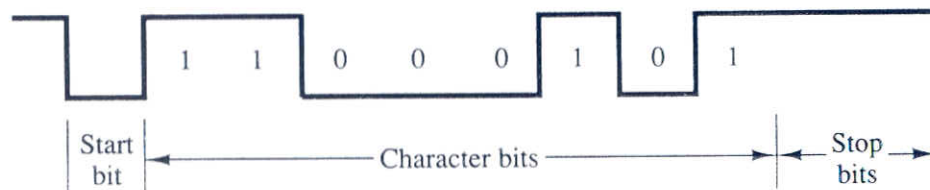


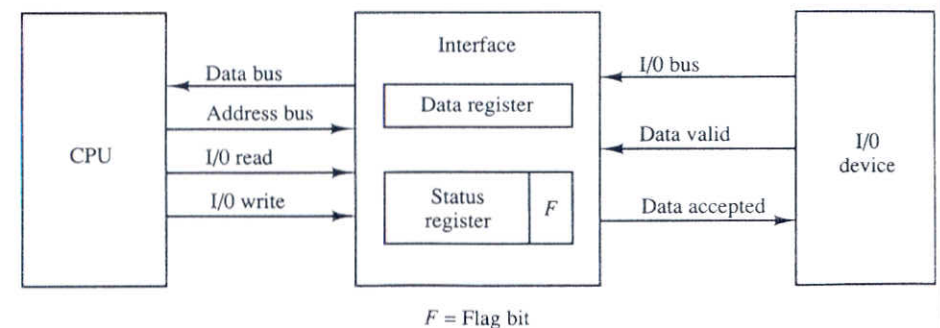
Figure 11-7 Asynchronous serial transmission.



CS	RS	Operation	Register selected
0	x	x	None: data bus in high-impedance
1	0	WR	Transmitter register
1	1	WR	Control register
1	0	RD	Receiver register
1	1	RD	Status register

Figure 11-8 Block diagram of a typical asynchronous communication interface.

Figure 11-10 Data transfer from I/O device to CPU.



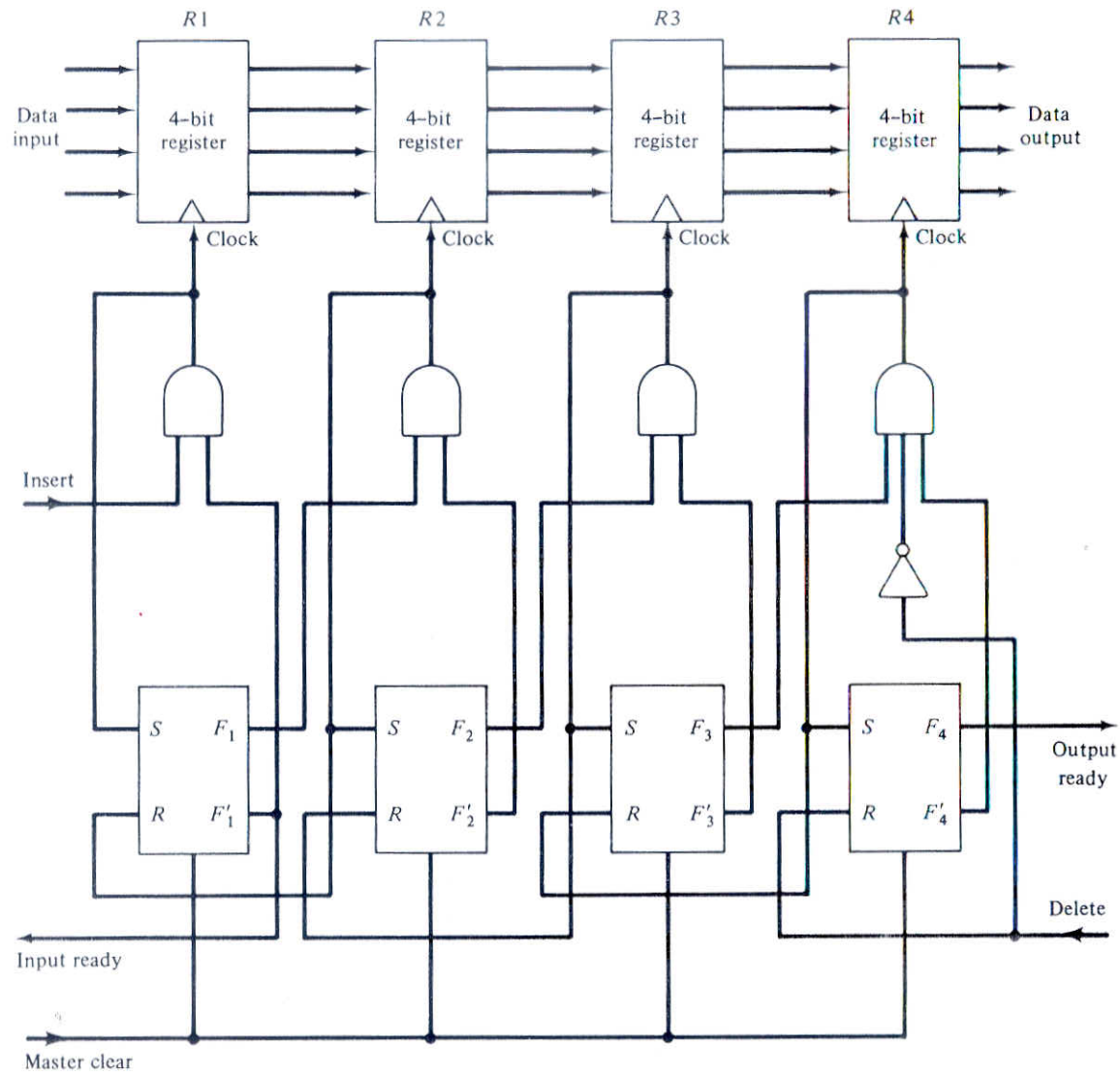
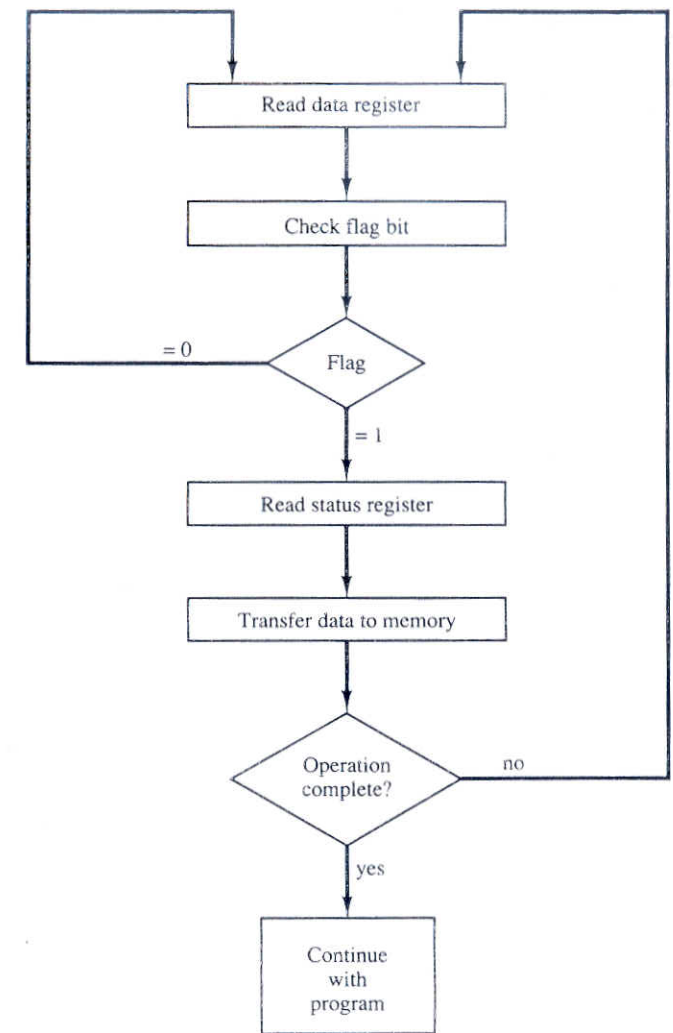
Figure 11-9 Circuit diagram of 4×4 FIFO buffer.

Figure 11-11 Flowchart for CPU program to input data.

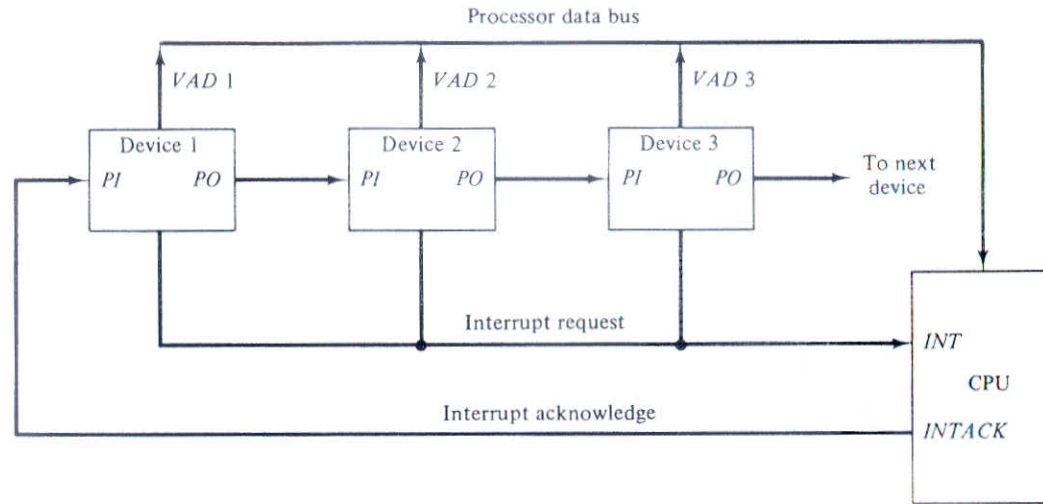


Figure 11-12 Daisy-chain priority interrupt.

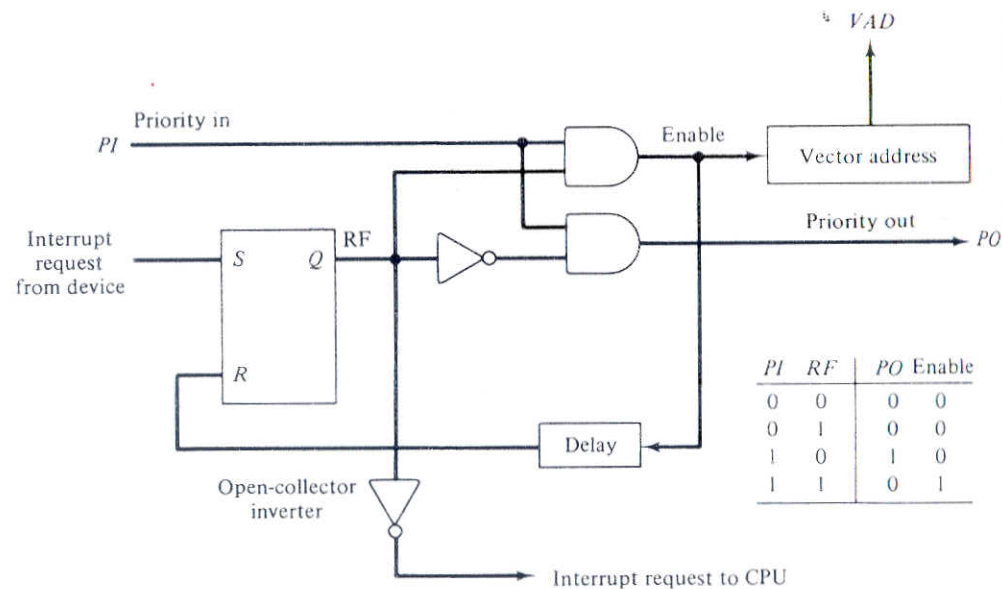


Figure 11-13 One stage of the daisy-chain priority arrangement.

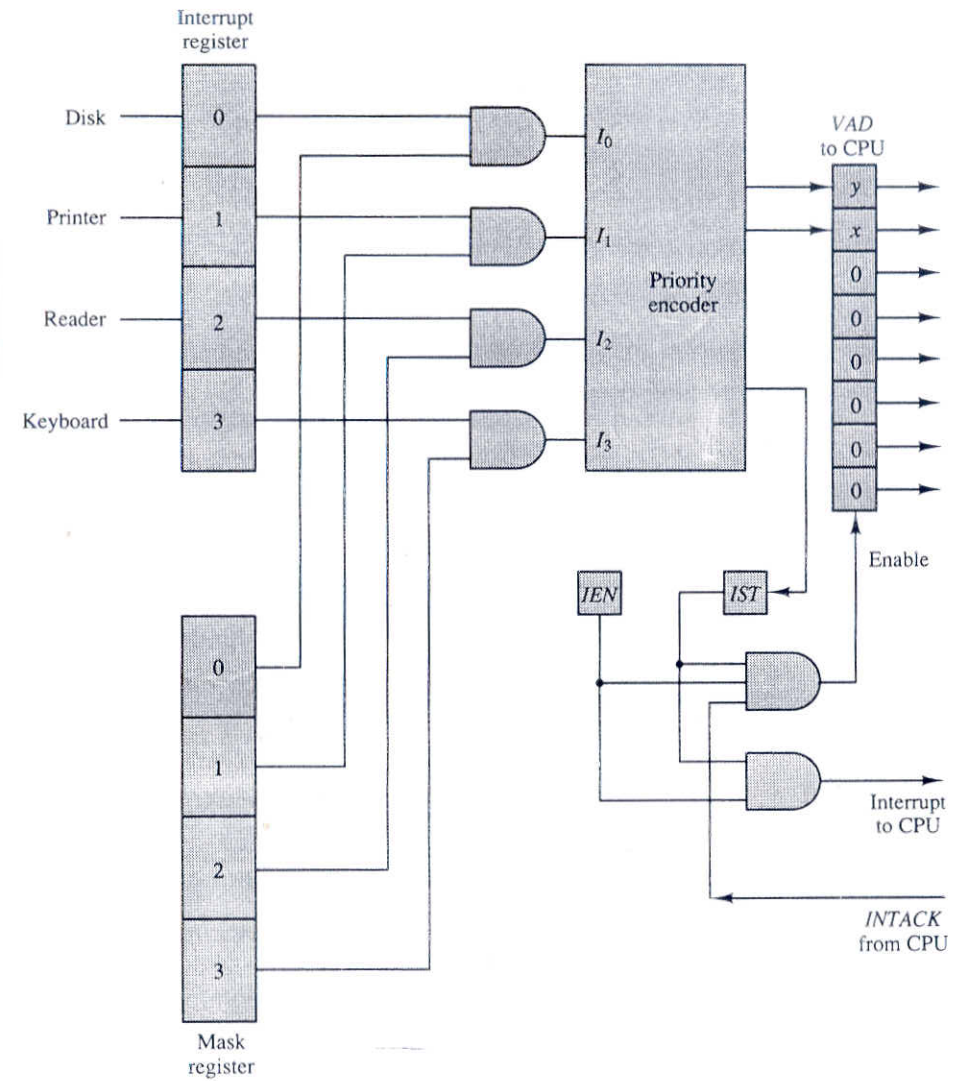


Figure 11-14 Priority interrupt hardware.

Figure 11-15 Programs stored in memory for servicing interrupts.

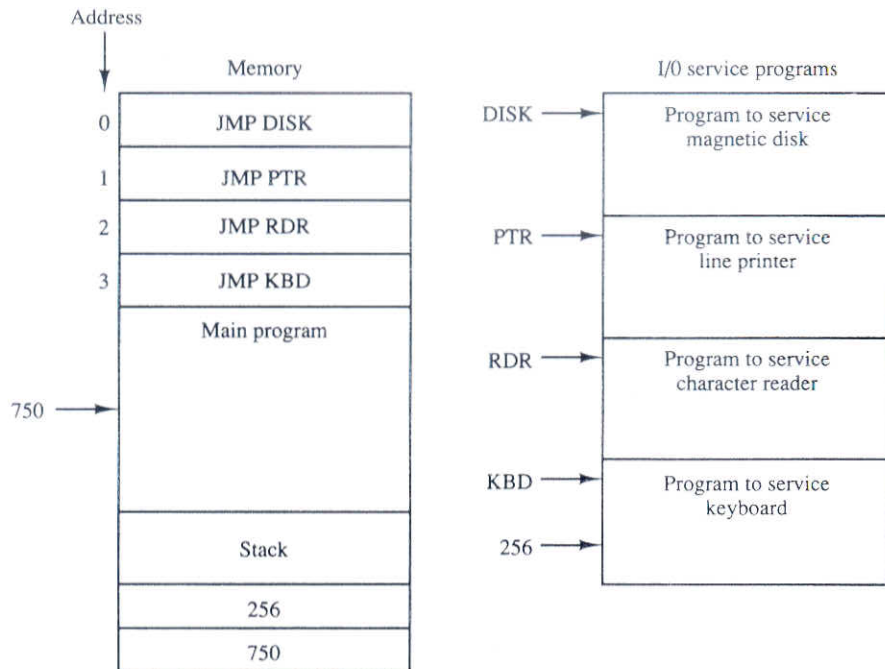


Figure 11-17 Block diagram of DMA controller.

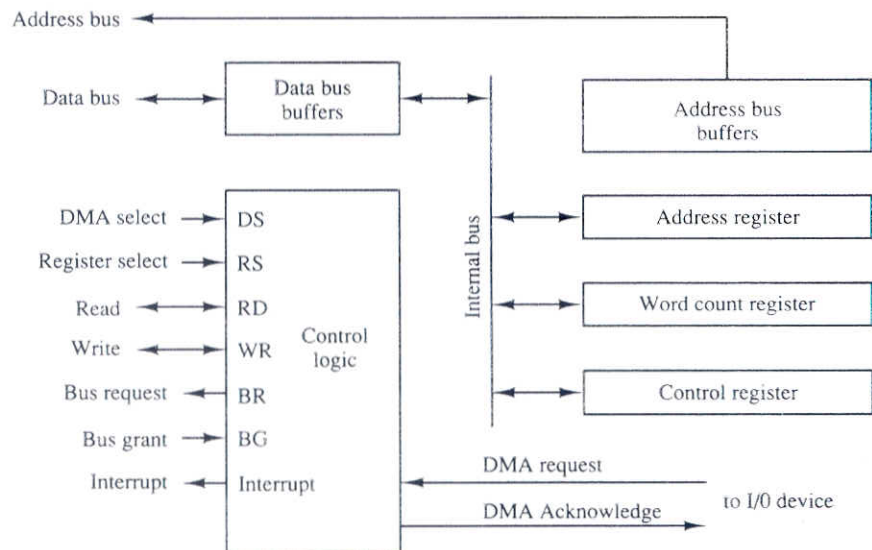


Figure 11-16 CPU bus signals for DMA transfer.

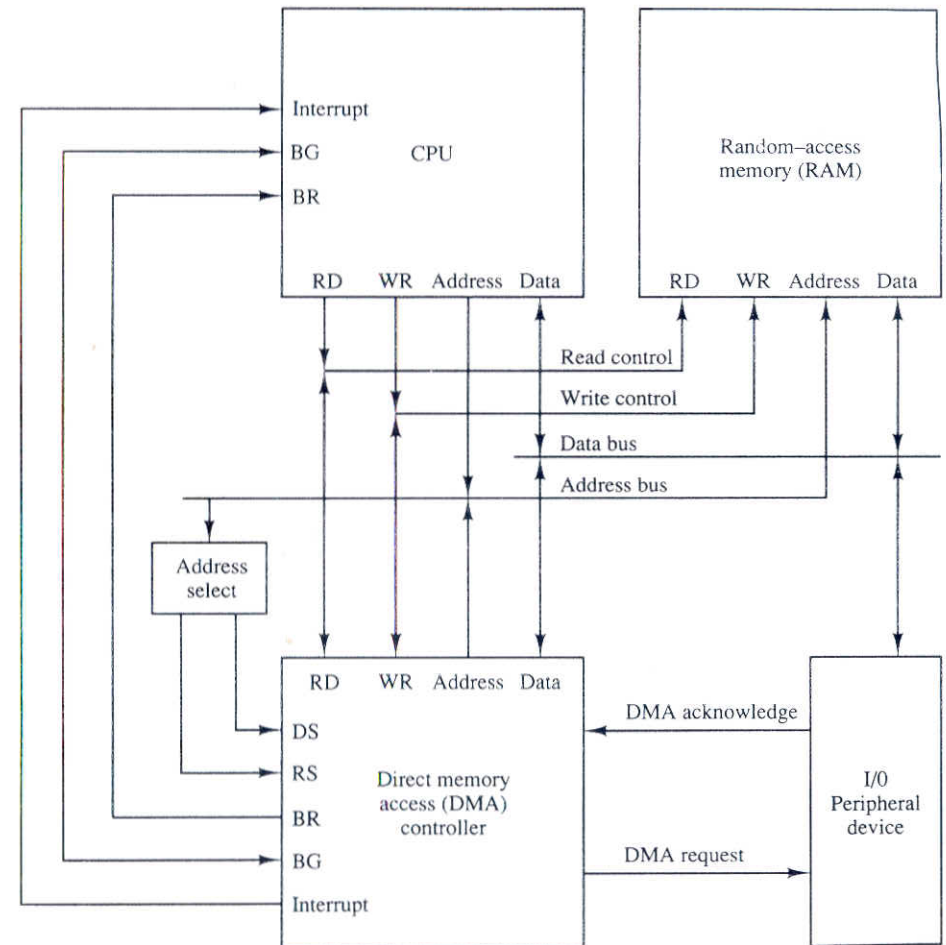
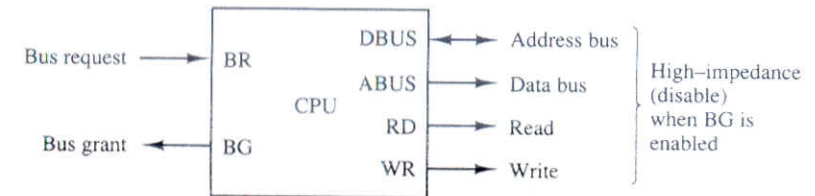


Figure 11-18 DMA transfer in a computer system.

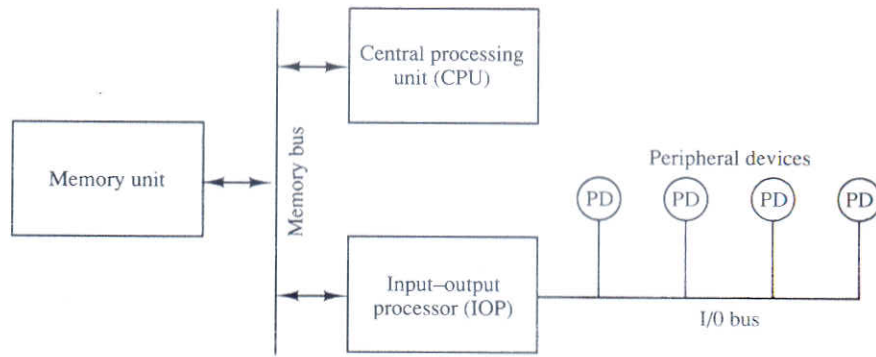


Figure 11-19 Block diagram of a computer with I/O processor.

Figure 11-20 CPU-IOP communication.

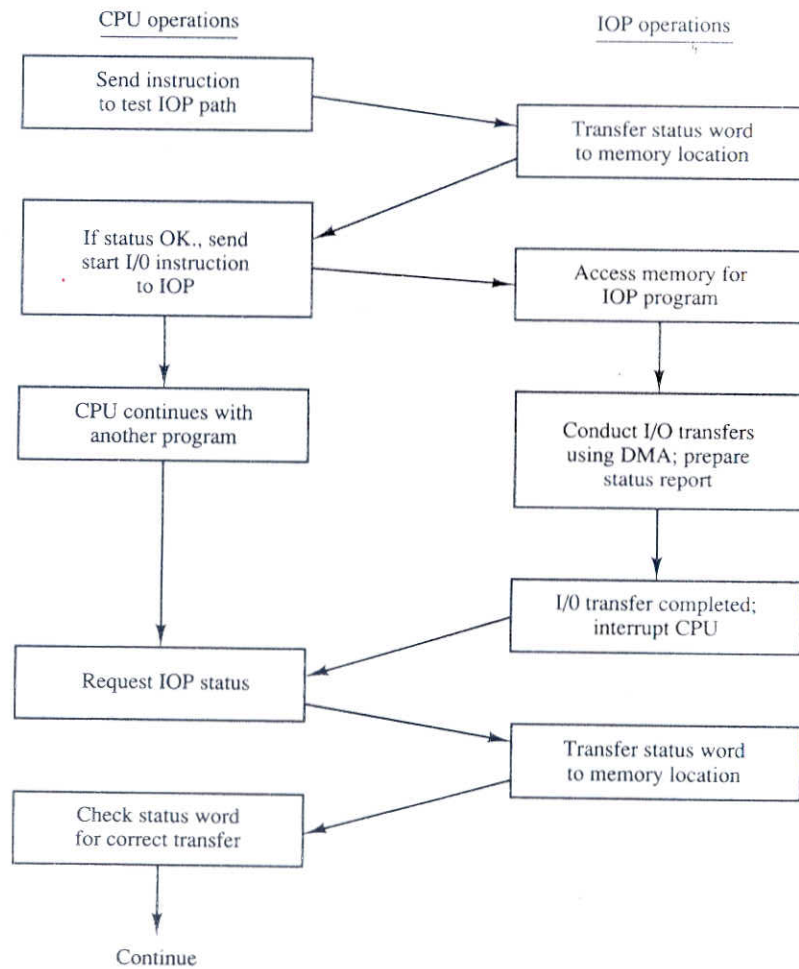
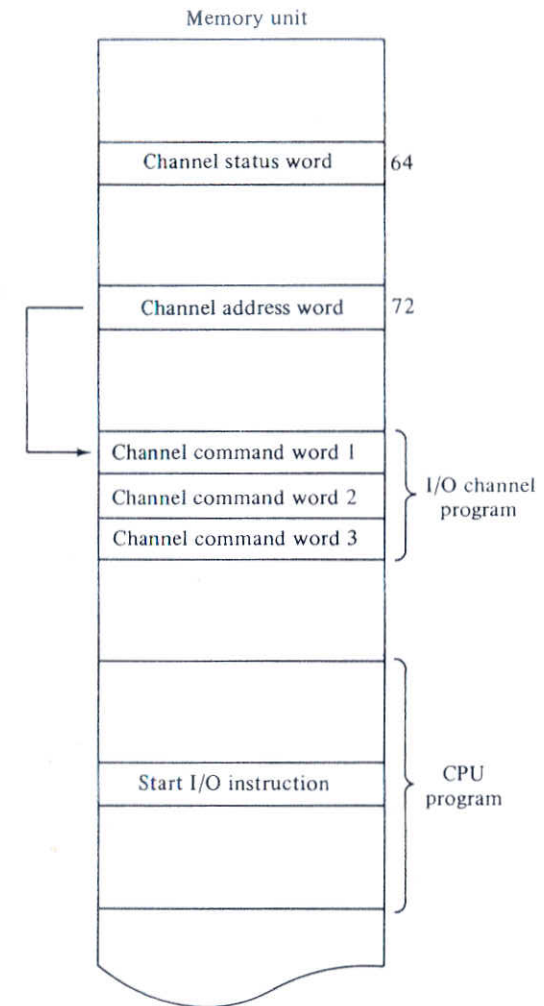
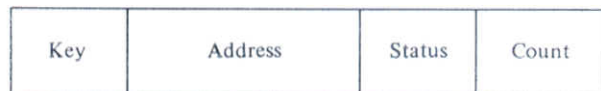


Figure 11-22 Location of information in memory for I/O operations in the IBM 370.

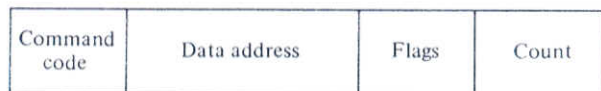




(a) I/O instruction format



(b) Channel status word format



(c) Channel command word format

Figure 11-21 IBM 370 I/O related word formats.

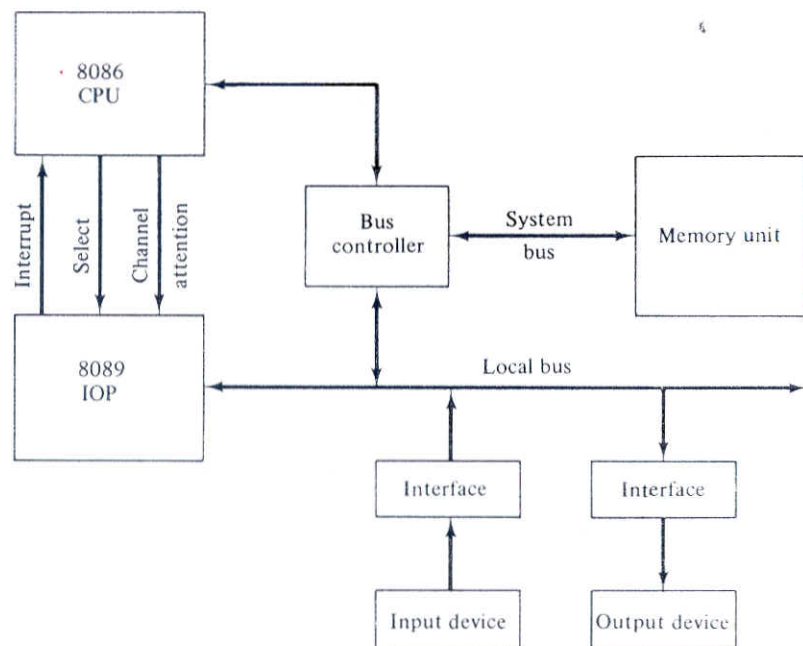


Figure 11-23 Intel 8086/8089 microcomputer system block diagram.

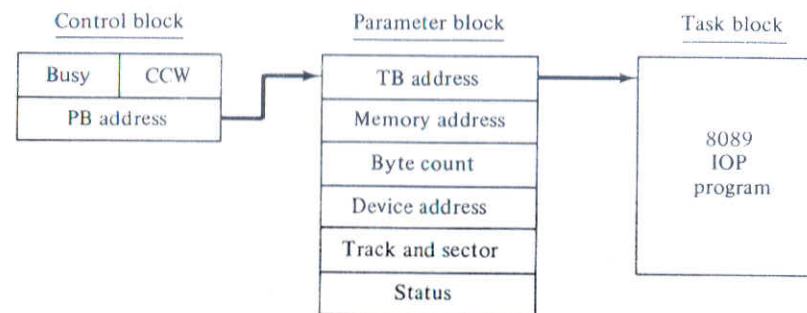


Figure 11-24 Location of information in memory for I/O operations in the Intel 8086/8089 microcomputer system.

Figure 11-25 Typical message format for character-oriented protocol.

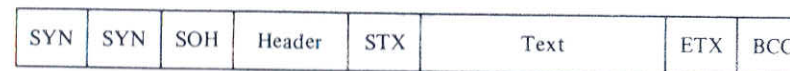


Figure 11-26 Frame format for bit-oriented protocol.

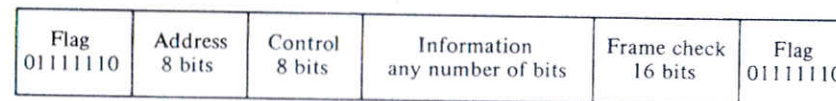
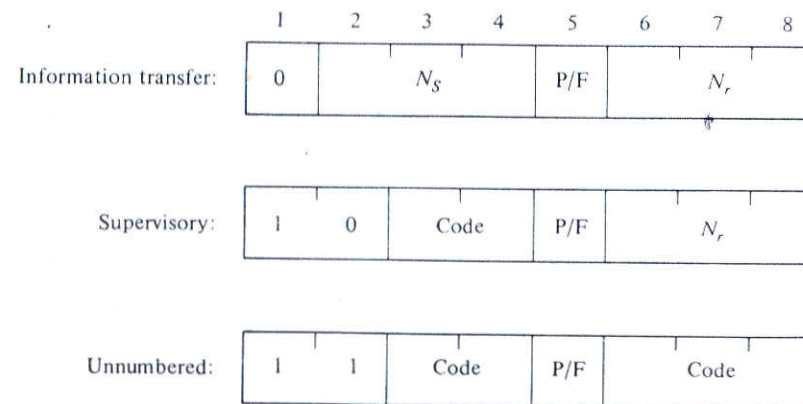


Figure 11-27 Control field format in bit-oriented protocol.



N_s Send count P/F Poll/final
 N_r Receive count Code Binary code

TABLE 11-1 American Standard Code for Information Interchange (ASCII)

$b_4 b_3 b_2 b_1$	$b_7 b_6 b_5$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	'	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Control characters

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

TABLE 11-2 Priority Encoder Truth Table

Inputs				Outputs			Boolean functions
I_0	I_1	I_2	I_3	x	y	IST	
1	x	x	x	0	0	1	$x = I_0' I_1'$ $y = I_0' I_1 + I_0' I_2'$ $(IST) = I_0 + I_1 + I_2 + I_3$
0	1	x	x	0	1	1	
0	0	1	x	1	0	1	
0	0	0	1	1	1	1	
0	0	0	0	x	x	0	

TABLE 11-3 IBM-370 Channel Program Example

Command	Address	Flags	Count
Write tape	4000	100000	60
Write tape	6000	010000	20
Write tape	3000	000000	40

TABLE 11-4 ASCII Communication Control Characters

Code	Symbol	Meaning	Function
0010110	SYN	Synchronous idle	Establishes synchronism
0000001	SOH	Start of heading	Heading of block message
0000010	STX	Start of text	Precedes block of text
0000011	ETX	End of text	Terminates block of text
0000100	EOT	End of transmission	Concludes transmission
0000110	ACK	Acknowledge	Affirmative acknowledgement
0010101	NAK	Negative acknowledge	Negative acknowledgement
0000101	ENQ	Inquiry	Inquire if terminal is on
0010111	ETB	End of transmission block	End of block of data
0010000	DLE	Data link escape	Special control character

TABLE 11-5 Typical Transmission from a Terminal to Processor

Code	Symbol	Comments
0001 0110	SYN	First sync character
0001 0110	SYN	Second sync character
0000 0001	SOH	Start of heading
0101 0100	T	Address of terminal is T4
0011 0100	4	
0000 0010	STX	Start of text transmission
0101 0010		
0100 0101	request	Text sent is a request to respond with the balance of account number 1234
.	balance	
.	of account	
.	No. 1234	
1011 0011		
0011 0100		
1000 0011	ETX	End of text transmission
0111 0000	LRC	Longitudinal parity character

TABLE 11-6 Typical Transmission from Processor to Terminal

Code	Symbol	Comments
0001 0110	SYN	First sync character
0001 0110	SYN	Second sync character
1000 0110	ACK	Processor acknowledges previous message
0001 0110	SYN	Line is idling
.	.	
0001 0110	SYN	Line is idling
0000 0001	SOH	Start of heading
0101 0100	T	Address of terminal is T4
0011 0100	4	
0000 0010	STX	Start of text transmission
1100 0010		
1100 0001	balance	Text sent is a response from the computer giving the balance of account
.	is	
.	\$100.00	
.		
1011 0000		
1000 0011	ETX	End of text transmission
1101 0101	LRC	Longitudinal parity character

Figure 12-1 Memory hierarchy in a computer system.

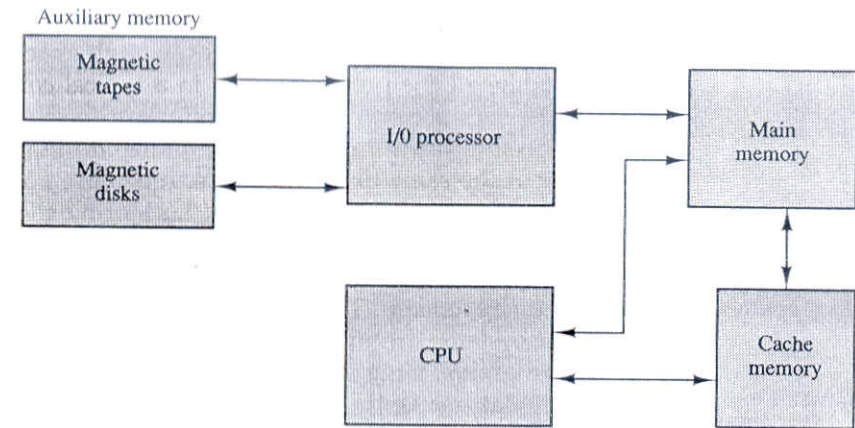
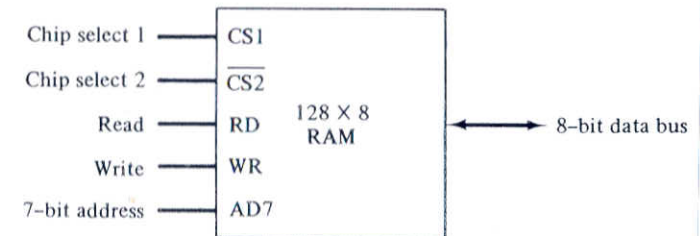


Figure 12-2 Typical RAM chip.



(a) Block diagram

CS1	CS2	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedance
0	1	x	x	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data from RAM
1	1	x	x	Inhibit	High-impedance

(b) Function table

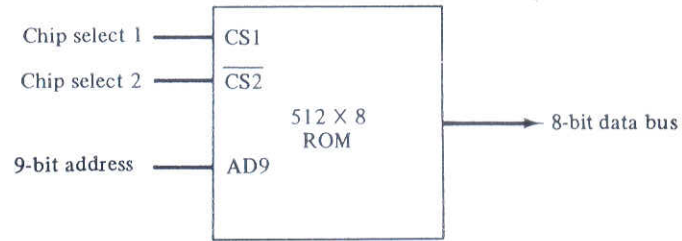


Figure 12-3 Typical ROM chip.

TABLE 12-1 Memory Address Map for Microcomputer

Component	Hexadecimal address	Address bus							
		10	9	8	7	6	5	4	3 2 1
RAM 1	0000-007F	0	0	0	x	x	x	x	x x x x
RAM 2	0080-00FF	0	0	1	x	x	x	x	x x x x
RAM 3	0100-017F	0	1	0	x	x	x	x	x x x x
RAM 4	0180-01FF	0	1	1	x	x	x	x	x x x x
ROM	0200-03FF	1	x	x	x	x	x	x	x x x x

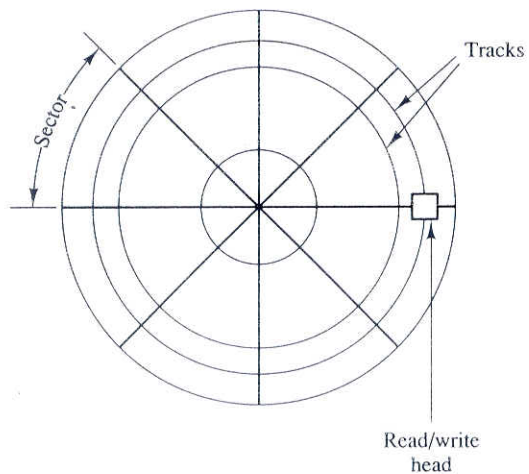


Figure 12-5 Magnetic disk.

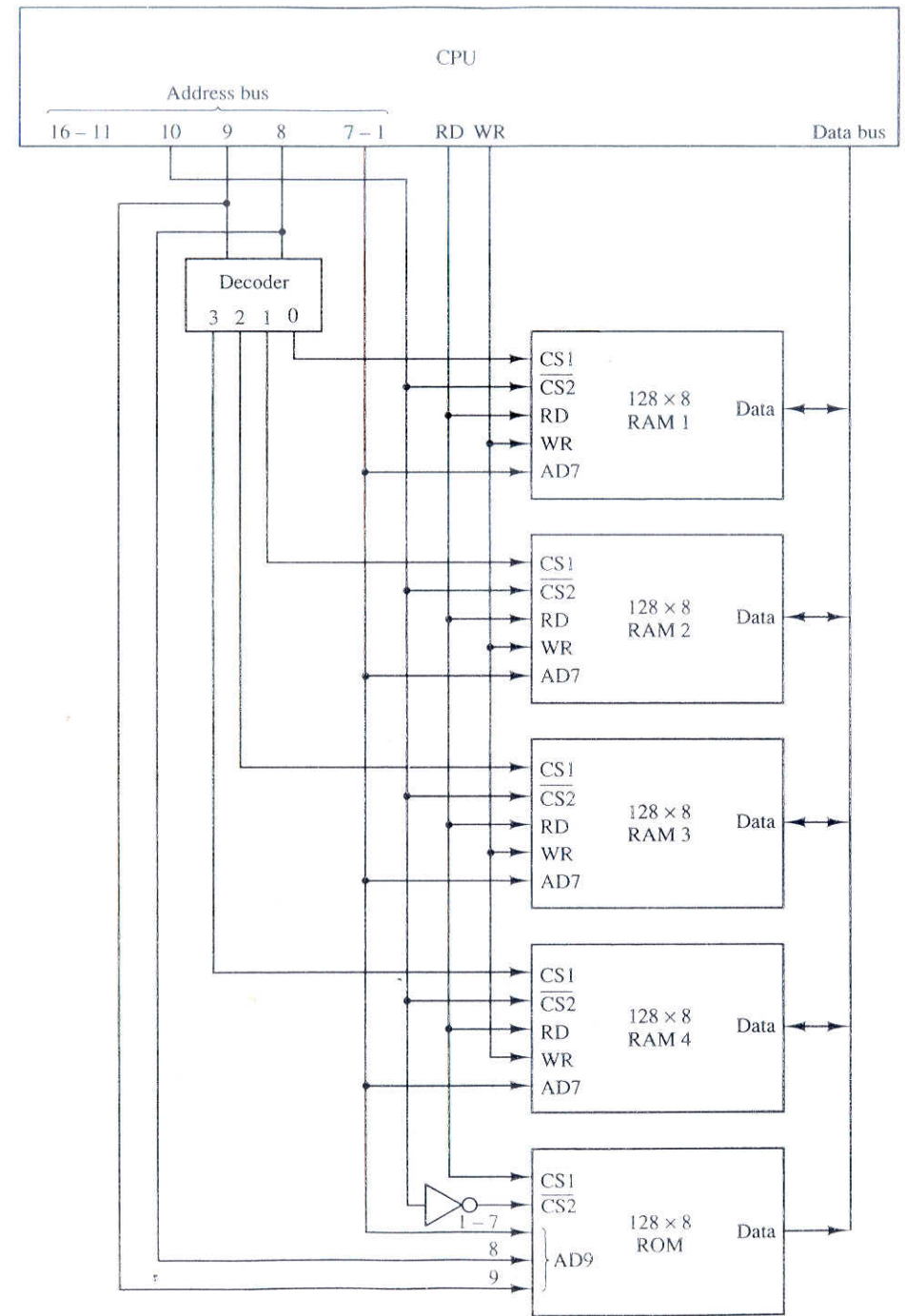


Figure 12-4 Memory connection to the CPU.

Figure 12-6 Block diagram of associative memory.

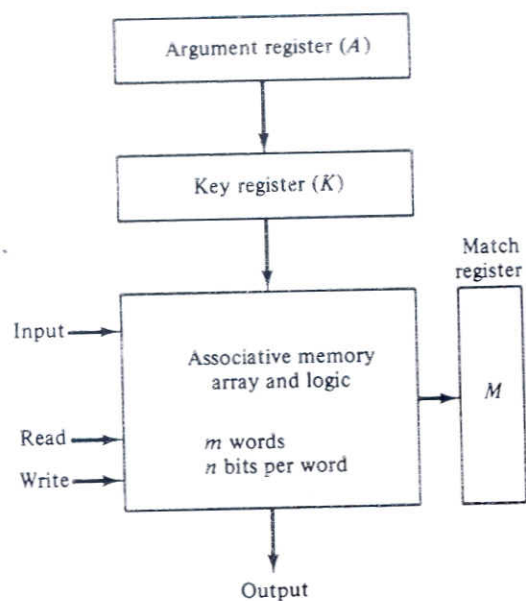
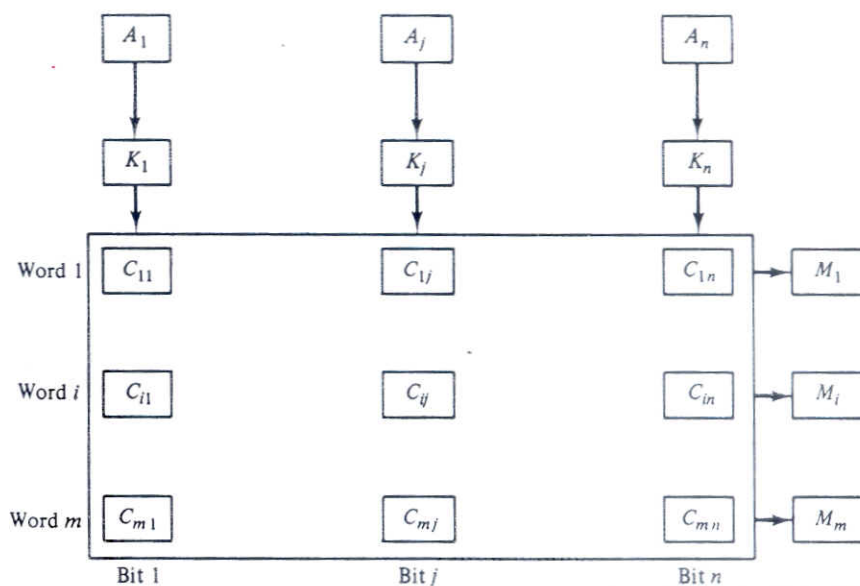
Figure 12-7 Associative memory of m word, n cells per word.

Figure 12-8 One cell of associative memory.

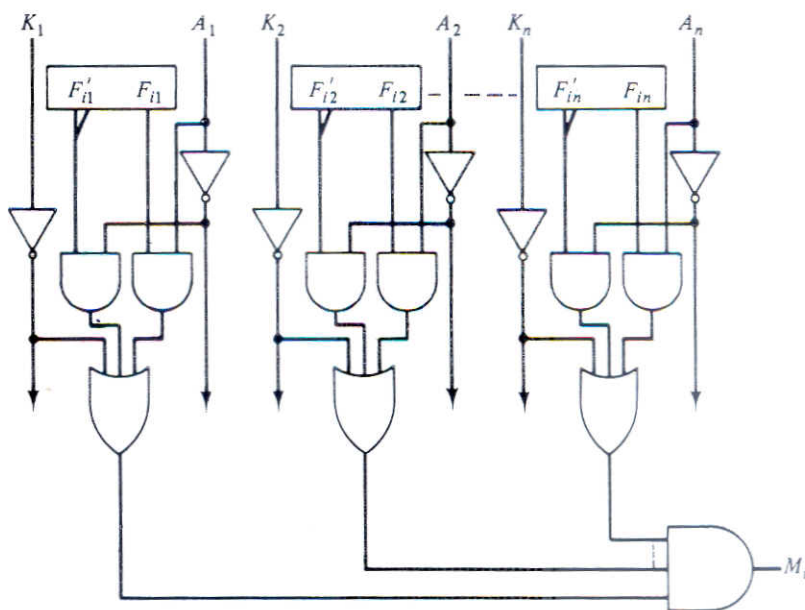
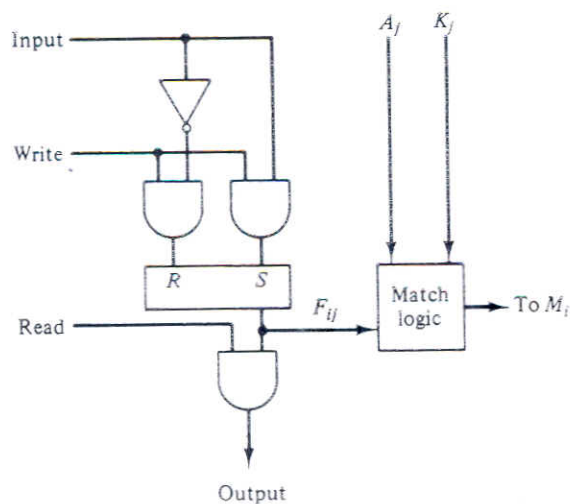


Figure 12-9 Match logic for one word of associative memory.

Figure 12-11 Associative mapping cache (all numbers in octal).

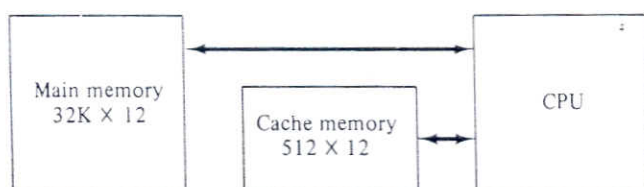


Figure 12-10 Example of cache memory.

CPU address (15 bits)

Argument register	
Address	Data
0 1 0 0 0	3 4 5 0
0 2 7 7 7	6 7 1 0
2 2 3 4 5	1 2 3 4

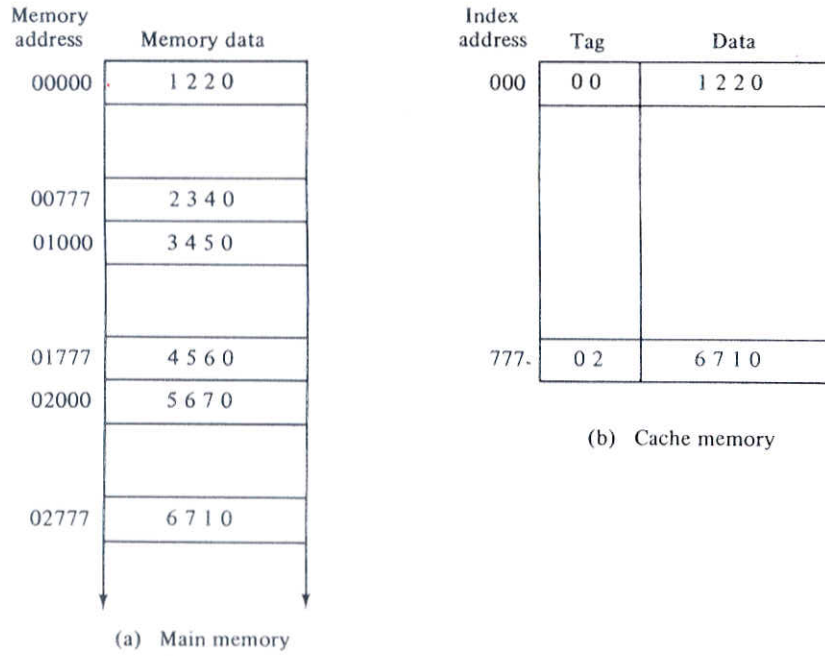


Figure 12-13 Direct mapping cache organization.

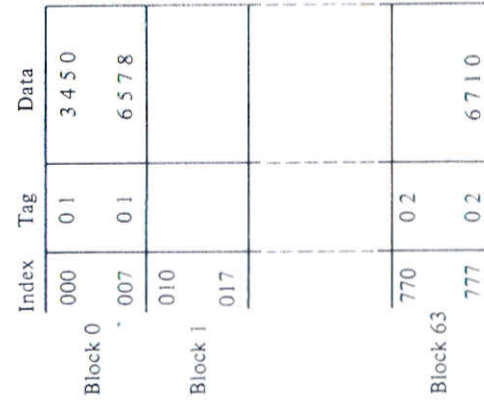
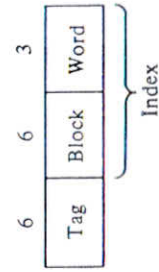
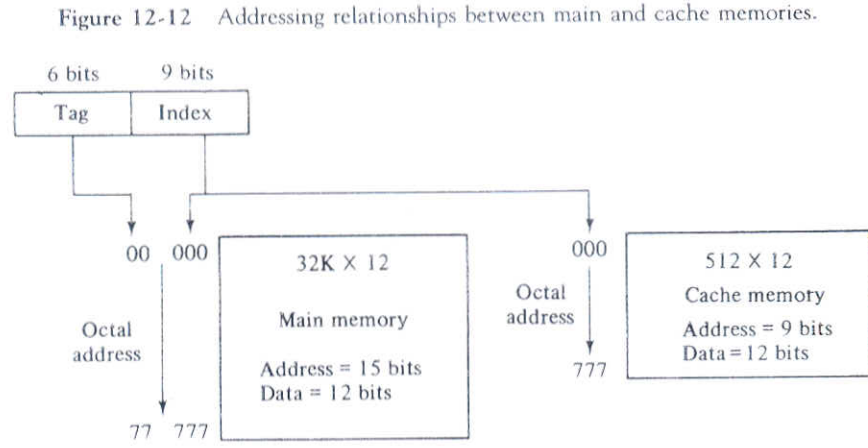


Figure 12-14 Direct mapping cache with block size of 8 words.

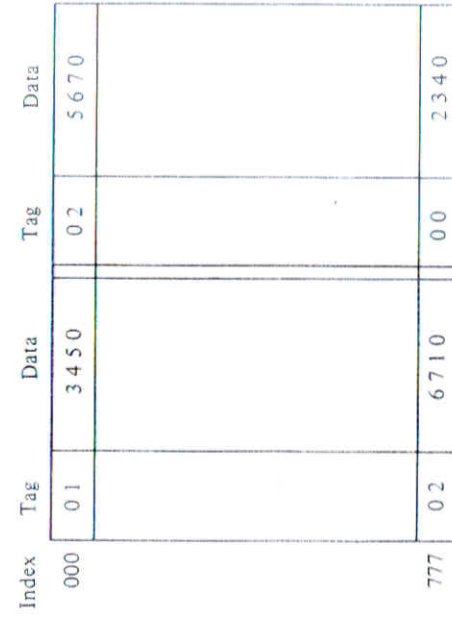
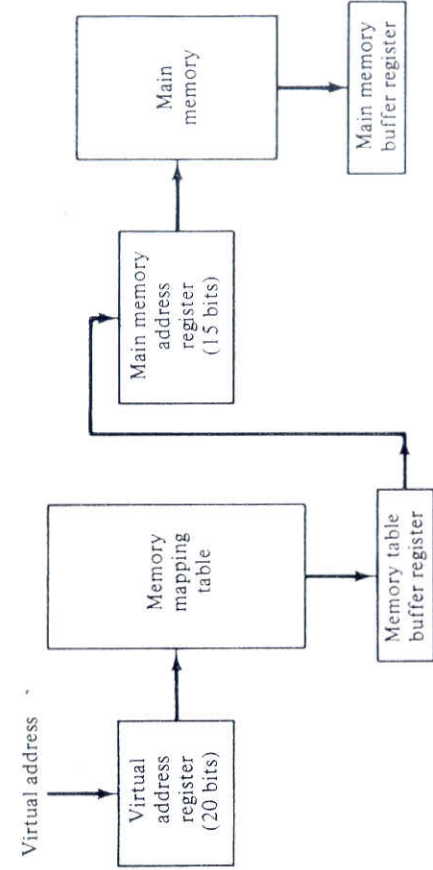


Figure 12-15 Two-way set-associative mapping cache.

Figure 12-17 Memory table for mapping a virtual address.



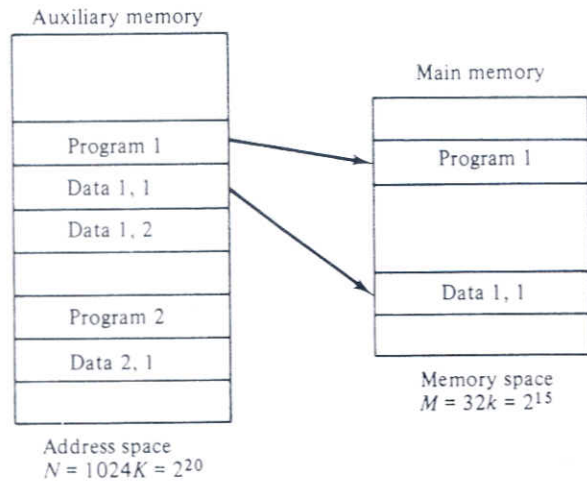


Figure 12-16 Relation between address and memory space in a virtual memory system.

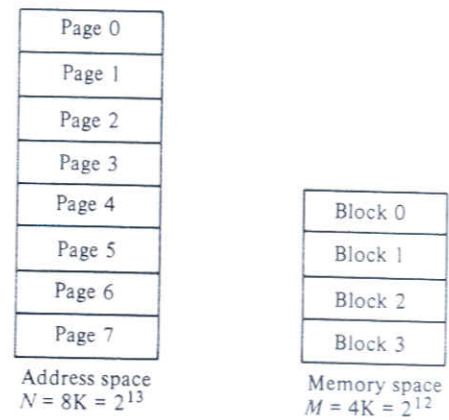


Figure 12-18 Address space and memory space split into groups of 1K words.

Figure 12-19 Memory table in a paged system.

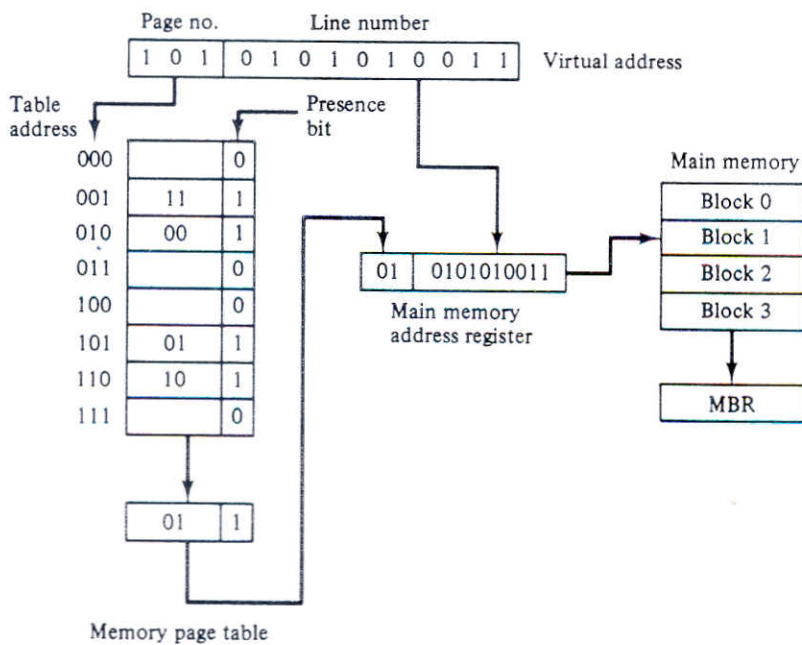


Figure 12-20 An associative memory page table.

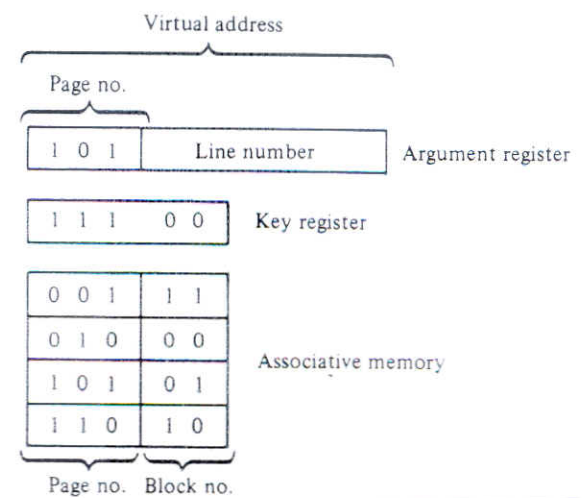
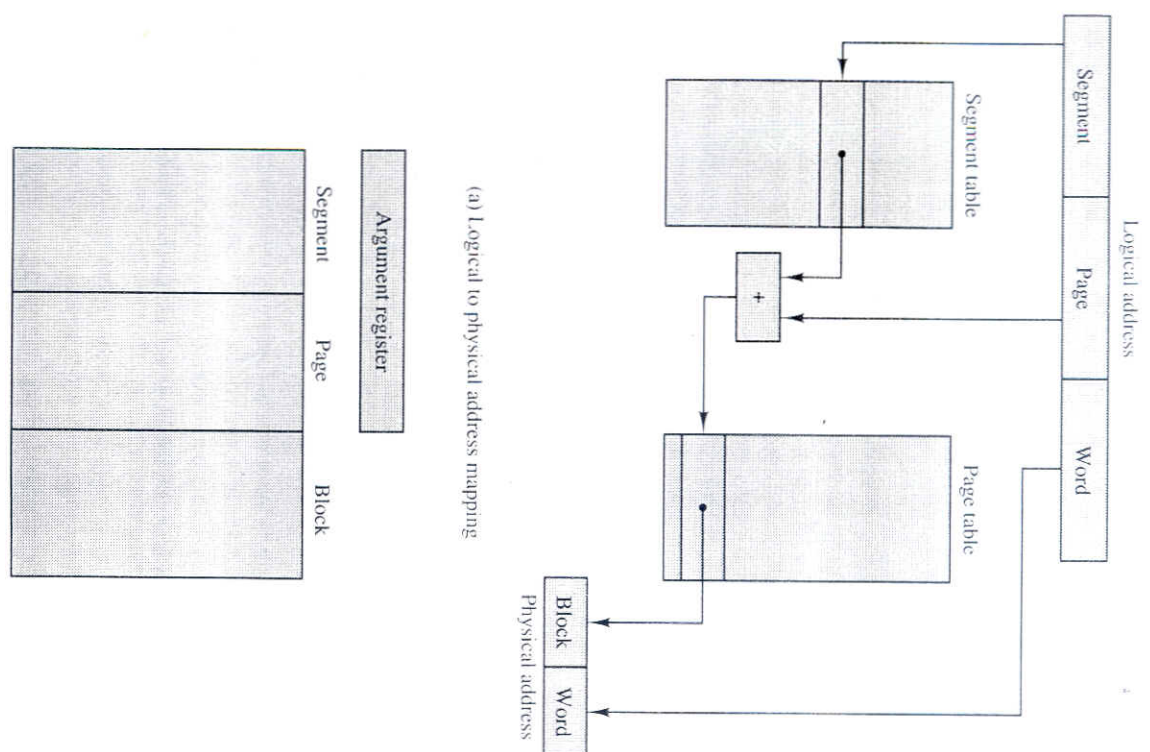


Figure 12-21 Mapping in segmented-page memory management unit.



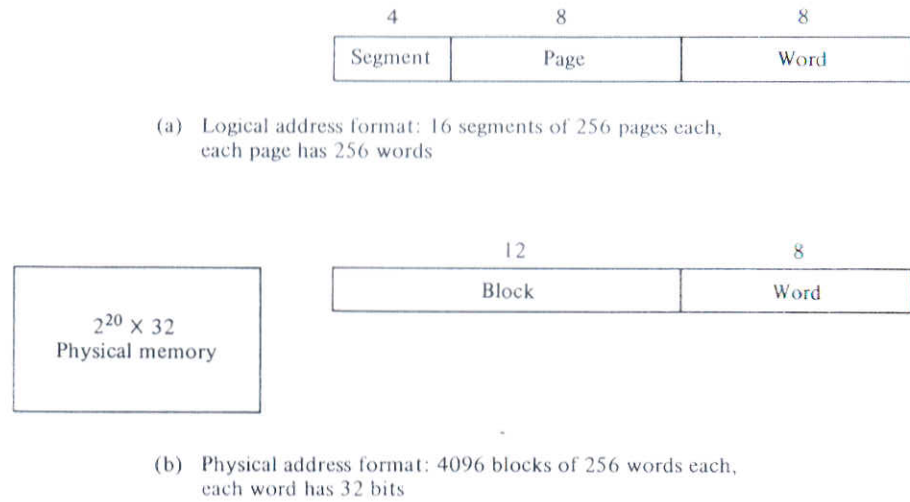


Figure 12-22 An example of logical and physical addresses.

Figure 12-23 Example of logical and physical memory address assignment.

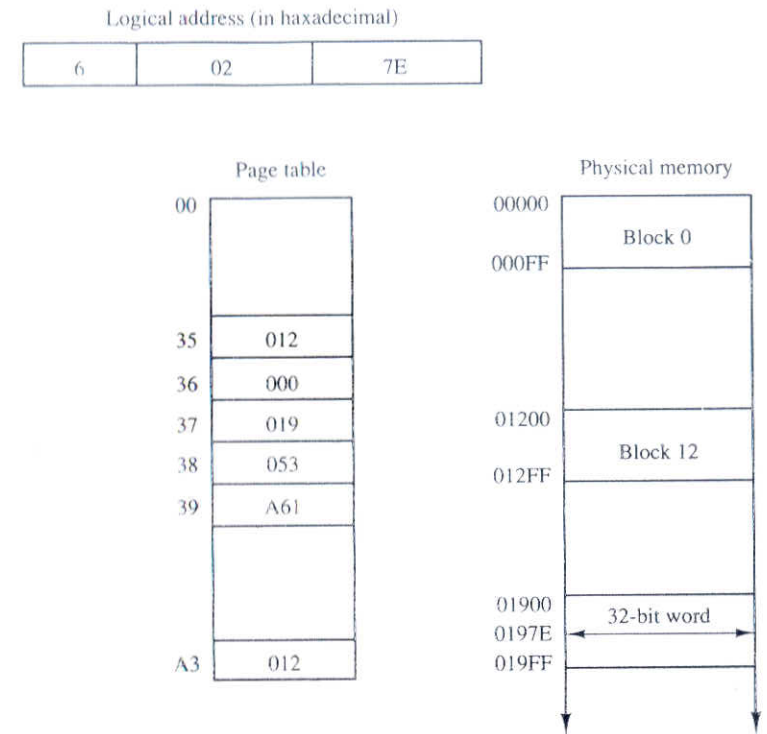
Hexadecimal address	Page number	Segment	Page	Block
60000	Page 0	6	00	012
60100	Page 1		01	000
60200	Page 2		02	019
60300	Page 3		03	053
60400	Page 4		04	A61
604FF				

(a) Logical address assignment

(b) Segment-page versus memory block assignment

Figure 12-25 Format of a typical segment descriptor.

Base address	Length	Protection
--------------	--------	------------



(a) Segment and page table mapping

Segment	Page	Block
6	02	019
6	04	A61

(b) Associative memory (TLB)

Figure 12-24 Logical to physical memory mapping example (all numbers are in hexadecimal).

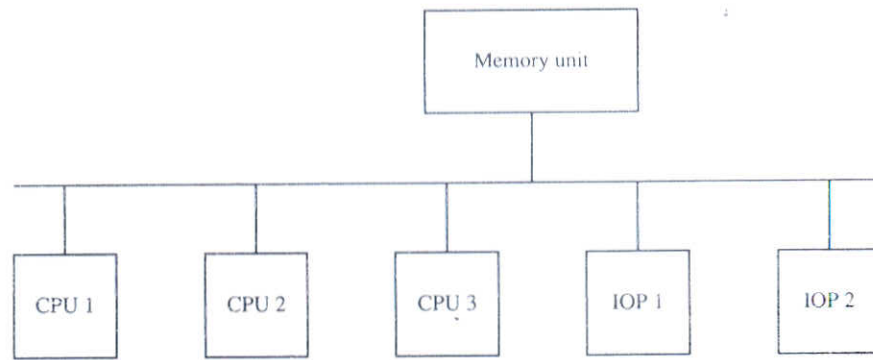


Figure 13-1 Time-shared common bus organization.

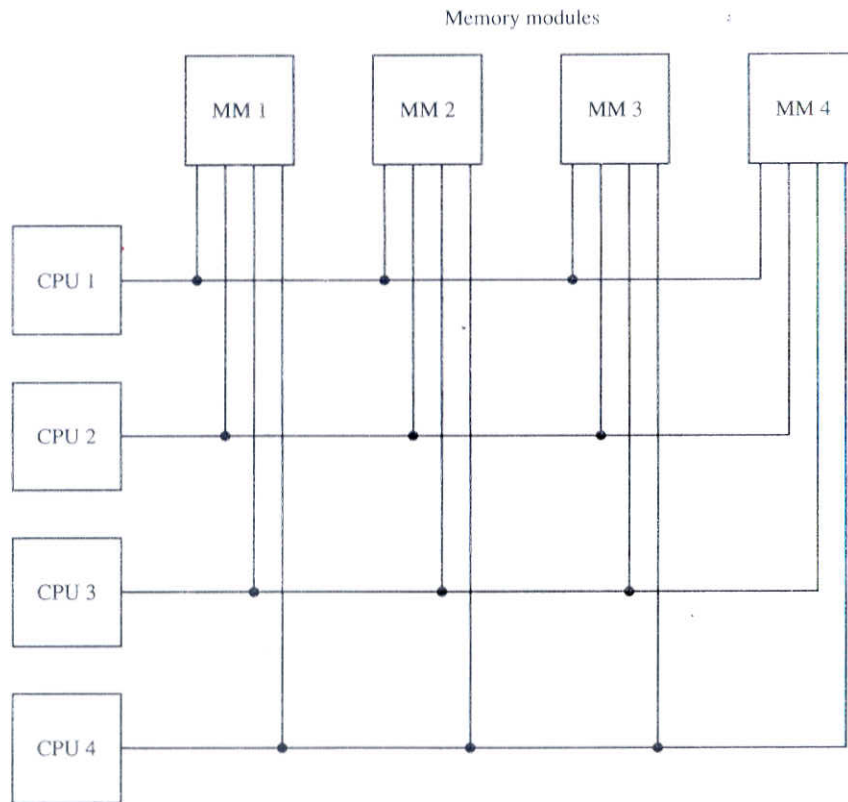


Figure 13-3 Multiport memory organization.

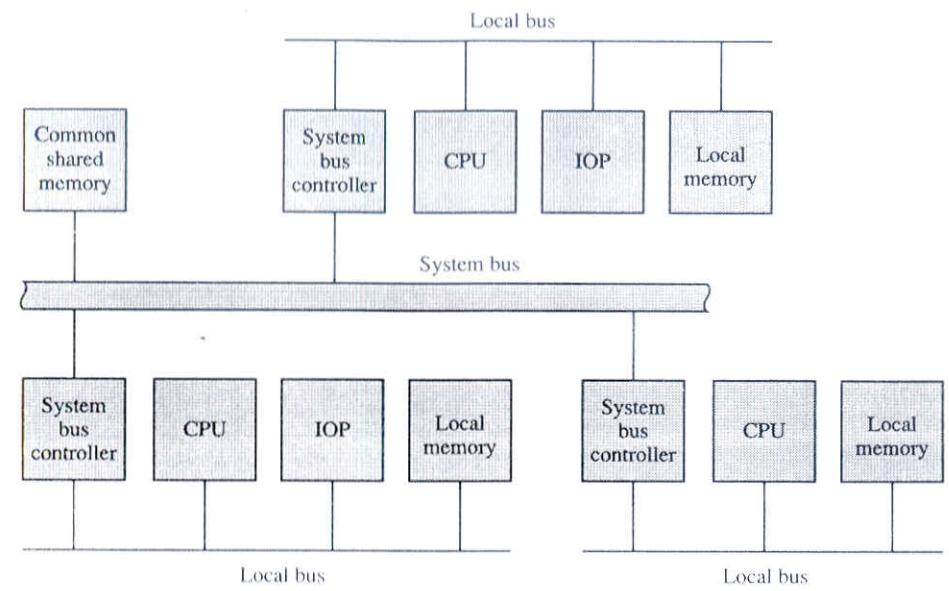


Figure 13-2 System bus structure for multiprocessors.

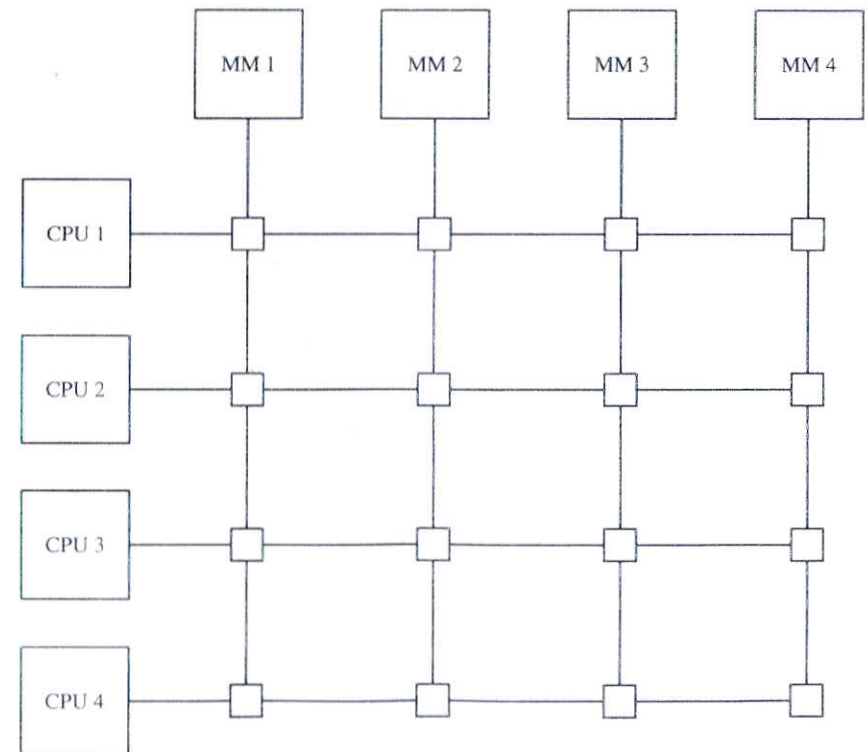


Figure 13-4 Crossbar switch.

Figure 13-5 Block diagram of crossbar switch.

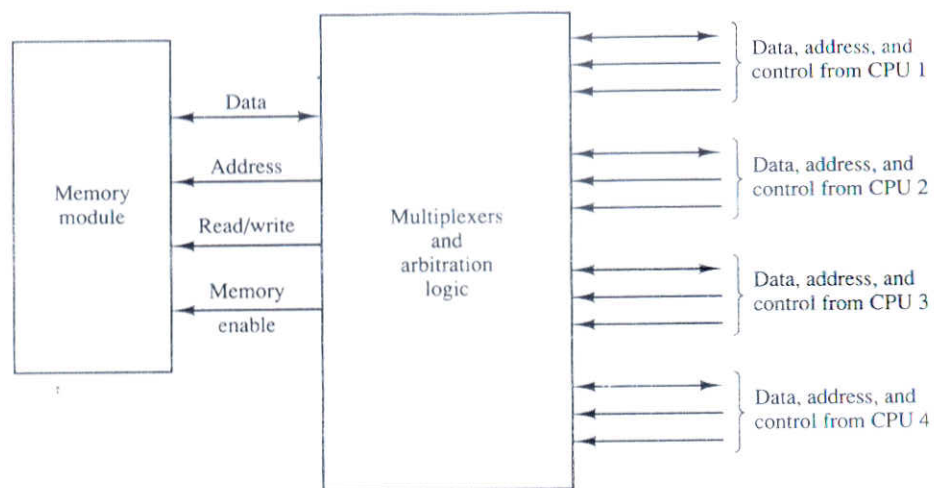
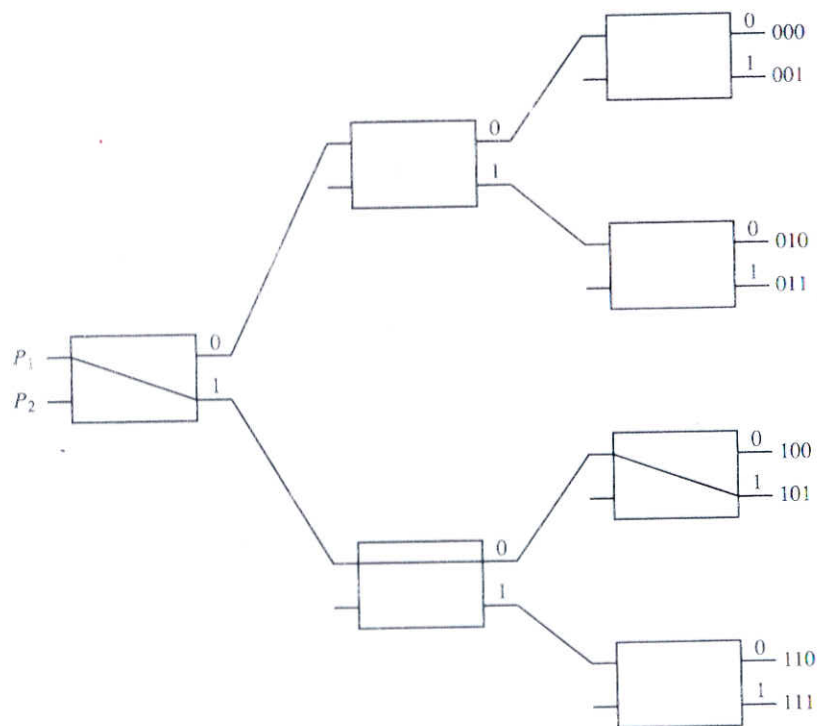
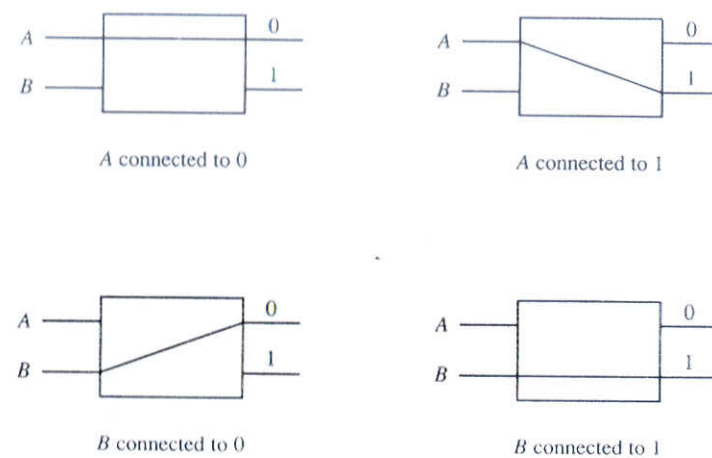
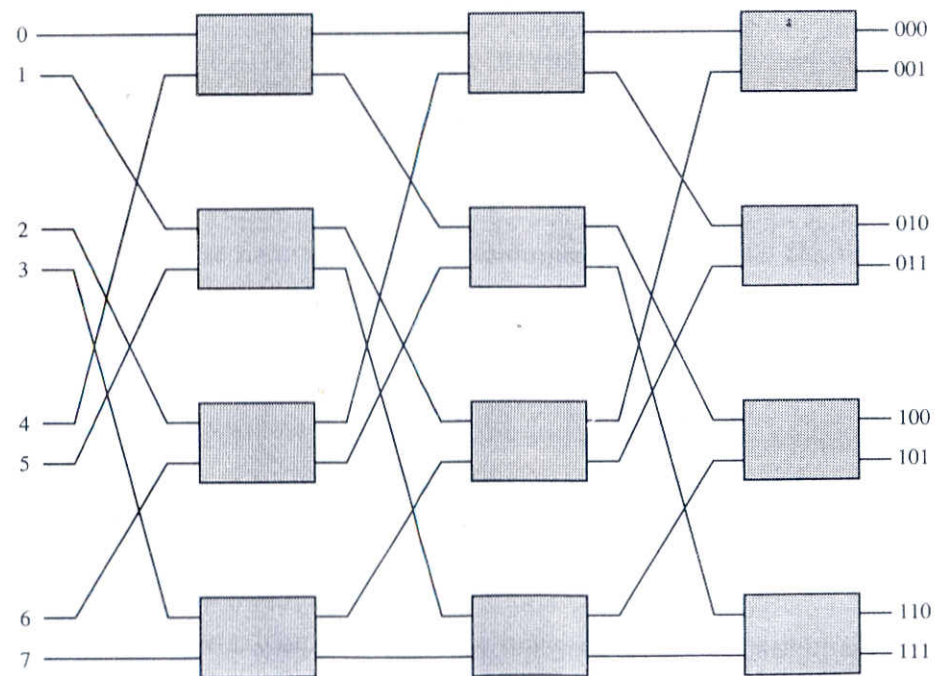
Figure 13-6 Operation of a 2×2 interchange switch.Figure 13-7 Binary tree with 2×2 switches.Figure 13-8 8×8 omega switching network.

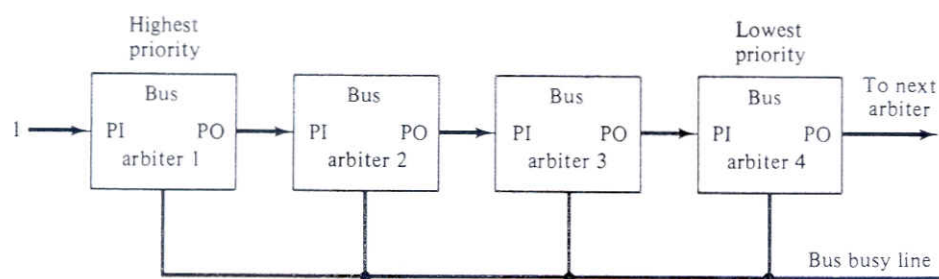
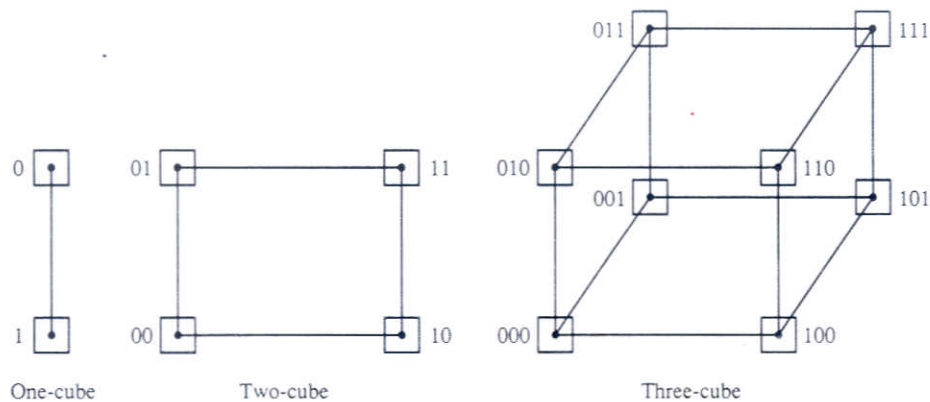
Figure 13-9 Hypercube structures for $n = 1, 2, 3$.

Figure 13-10 Serial (daisy-chain) arbitration.

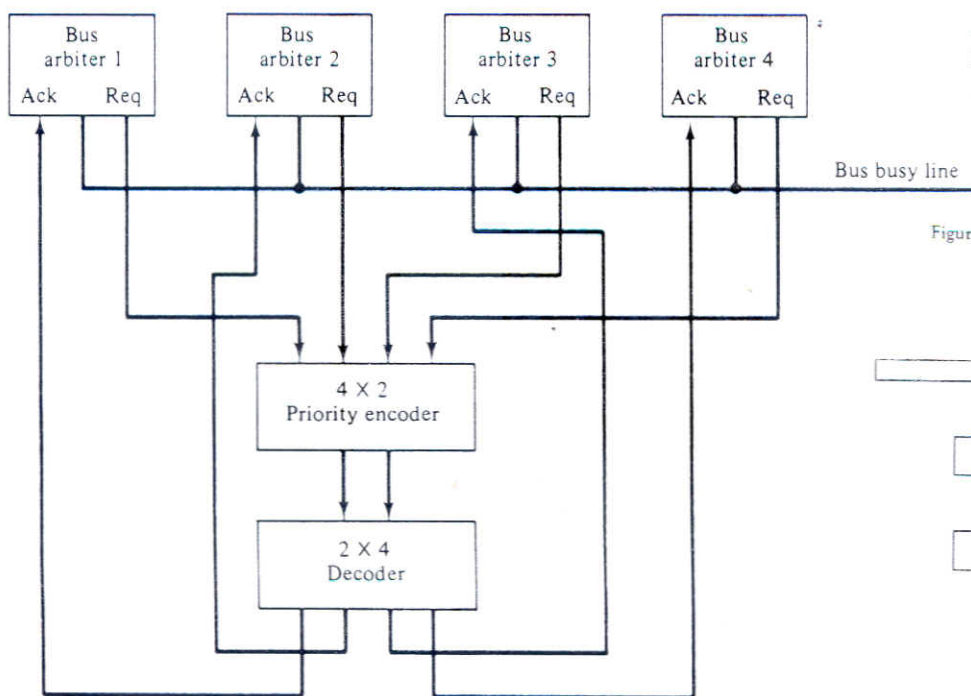


Figure 13-11 Parallel arbitration.

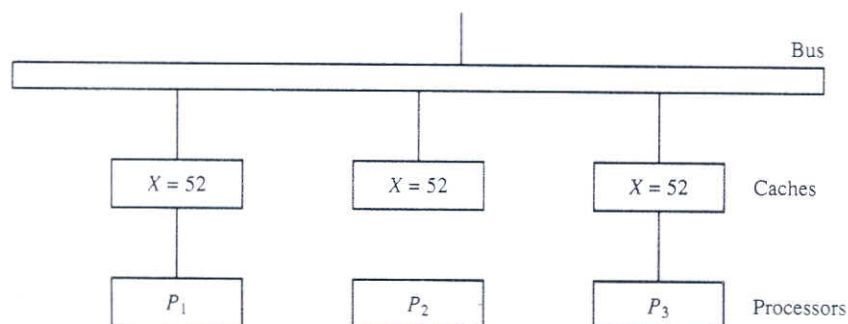
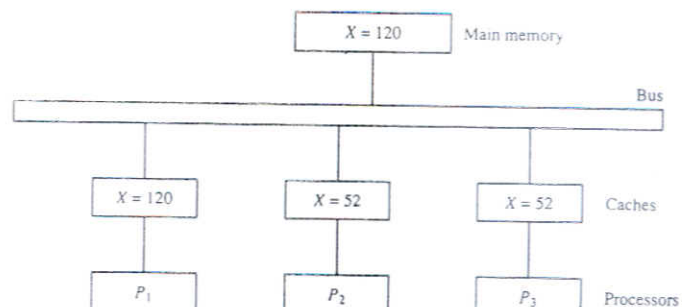


Figure 13-12 Cache configuration after a load on X.

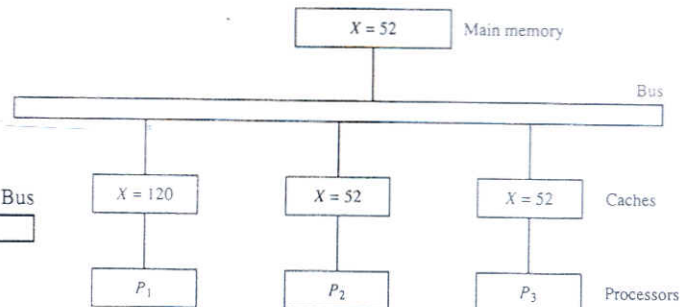
TABLE 13-1 IEEE Standard 796 Multibus Signals

	Signal name
Data and address	
Data lines (16 lines)	DATA0-DATA15
Address lines (24 lines)	ADRS0-ADRS23
Data transfer	
Memory read	MRDC
Memory write	MWTC
IO read	IORC
IO write	IOWC
Transfer acknowledge	TACK
Interrupt control	
Interrupt request (8 lines)	INT0-INT7
Interrupt acknowledge	INTA
Miscellaneous control	
Master clock	CCLK
System initialization	INIT
Byte high enable	BHEN
Memory inhibit (2 lines)	INH1-INH2
Bus lock	LOCK
Bus arbitration	
Bus request	BREQ
Common bus request	CBRQ
Bus busy	BUSY
Bus clock	BCLK
Bus priority in	BPRN
Bus priority out	BPRO
Power and ground (20 lines)	

Reprinted with permission of the IEEE.

Figure 13-13 Cache configuration after a store to X by processor P₁.

(a) With write-through cache policy



(b) With write-back cache policy